Programming Languages and Formal Methods for GPUs

DANIEL SAINATI, University of Pennsylvania, USA
CYNTHIA RICHEY, University of Pennsylvania, USA
PAUL BIBERSTEIN, University of Pennsylvania, USA
YUKA IKARASHI, Cornell University, USA
BENJAMIN C. PIERCE, University of Pennsylvania, USA

Programming modern GPUs is challenging due to both the rapid evolution of their architectural landscape and the need to manage complex parallel and asynchronous hardware to achieve high performance. Many new languages have been designed to address these challenges, employing a wide variety of programming models, but there is little consensus on the best approach. Formal methods for GPUs have progressed more slowly, but the difficulty of writing efficient kernels has generated significant interest in verification and analysis of GPU programs.

In this survey, we aim to familiarize programming-languages and formal-methods researchers both with prior work and with the unique needs and priorities of the GPU community, as these differ in critical and surprising ways from those of the broader computing ecosystem. We open with a concise history of GPU hardware and a summary of the SIMT programming model, outlining major themes and challenges. Next, we genealogize research on programming languages for GPUs, tracing the development of key ideas alongside the advancing hardware and highlighting the state of the art. Finally, we present a taxonomy of formal methods for GPUs, classified by methodology and use case, and identify prominent gaps in the literature.

1 Introduction

GPUs are everywhere, powering applications from video games to physics simulations to LLMs, but they are challenging to program effectively. The dominant programming model—*Single Instruction, Multiple Threads*, or SIMT—requires reasoning about hundreds or thousands of parallel and concurrent threads and is extremely prone to data races and deadlocks [352]. Such errors are notoriously difficult to detect and debug: they are frequently nondeterministic, may appear only for certain program configurations, and often require deep knowledge of GPU hardware to fix [108, 372]. In addition, the constant evolution of this hardware means identical programs may behave differently on different machines [242].

Writing a functionally correct GPU program, or *kernel*, is only the beginning of the story, however—making it fast is even harder. Since GPUs are primarily used for performance-critical applications, developers often spend significant effort optimizing their programs. Moreover, GPUs implement few hardware-level optimizations to hide software inefficiencies [239, 287], so carefully optimized and unoptimized versions of a program can often differ in performance by multiple orders of magnitude [289]. The optimization search space for kernels is vast, with few universally applicable techniques, so achieving "speed-of-light" performance often requires arcane knowledge and heroic effort [136].

Worse yet, recent changes in GPU hardware design have upended the SIMT programming model [243, 245, 247], breaking its fundamental assumptions—such as its per-thread view of program semantics [20]—and throwing the software ecosystem into disarray. GPUs' thin abstraction layers make these changes highly visible to developers, who are forced to reimplement their kernels, often with entirely new algorithms [305], to run efficiently on new hardware.

How do programming languages and formal methods fit into this story?

From the earliest GPUs, programming languages have coevolved with hardware [52, 216, 225, 239], with innovations in language design often having tangible impact on the broader ecosystem [343]. Currently, GPU programming is at an inflection point, with previously dominant programming models struggling to adapt to new hardware [325, 367], while demand for efficient kernels continues to grow rapidly. Some new languages have attempted to bridge the gap by empowering old paradigms with new features [25, 68, 128, 227, 313], while others have taken inspiration from outside the world

of GPUs [149, 369]. Still others propose that no single abstraction can work for all machines and that programmers instead require direct control over compilation to keep up with hardware evolution [163, 164]. Which approach will win out, or whether an entirely different strategy will emerge, is currently unclear.

In formal methods, the situation is similar. No existing tools model the newest hardware features, so none can analyze kernels as they are written in practice. As kernels grow more important and more difficult to write, so the need grows for effective program analysis, a need compounded further as developers turn to LLMs to optimize [30, 358] and generate [70] their kernels.

GPU programming is thus fertile ground for applying techniques from programming languages (PL) and formal methods, both to make correct and efficient kernels easier to write and to give developers more confidence in their correctness and efficiency. For the community's work to have an impact, however, researchers must be familiar both with prior research and with the priorities and concerns of GPU programmers. In this survey, we address these needs, offering the following contributions:

- a historical overview of GPUs and the SIMT programming model, identifying three major themes: incessant demand for increased performance, a vast and complex optimization space, and diverse and rapidly changing hardware (Section 2),
- a genealogy of programming models and languages for GPUs, describing their coevolution with hardware features and highlighting the cutting edge (Section 3), and
- a taxonomy of formal methods for GPU programs, identifying underserved areas and key opportunities (Section 4).

Finally, in Section 5 we conclude and identify potential directions for future research.

2 GPUs and the SIMT Programming Model

Before discussing languages and formal methods for GPUs, we offer a short history of GPUs, describing their basic SIMT programming model and how they evolved from specialized graphics hardware to general-purpose accelerators (Sections 2.1 and 2.2). Then we demonstrate by example how to optimize kernels (Section 2.3) and discuss the main themes of the GPU problem space (Section 2.4). Finally, we outline common challenges that GPU programmers currently face (Section 2.5) and explore how recent hardware developments have upended existing paradigms (Section 2.6).

2.1 Early Graphics Processing Units

The 1980s and 1990s saw an explosion of dedicated graphics processors, initially targeting high-performance computing tasks like computer-aided design (CAD) and 3D modeling but later appearing in personal computers and video game consoles. These early devices were specialized to individual stages of the *graphics pipeline*—the process of transforming a 3D model into a 2D image for display on a screen—and relied on the CPU to connect those stages. When the NVIDIA GeForce 256 [309] launched in 1999, it was the first mass-market chip capable of running the entire graphics pipeline on a single piece of hardware; it was accordingly marketed as the first *graphics processing unit*, or "GPU."

The GeForce 256 was a major step forward in hardware capabilities, but, like its predecessors, it was not programmable and exposed only a limited API. The first steps toward full programmability came with Sony's Emotion Engine [193] and the GeForce 3 [215], which allowed users to customize operations in various stages of the graphics pipeline. Users could write their own *shaders*, small programs specifying how to process a single element of a particular kind (e.g., a vertex in a graphical model or a pixel in an image), which they could then compose into a full pipeline to perform an entire graphics computation. (Figure 1 depicts a representative pipeline of this era.) Images and other data were *streamed* through this pipeline, with the GPU executing each component shader over many elements

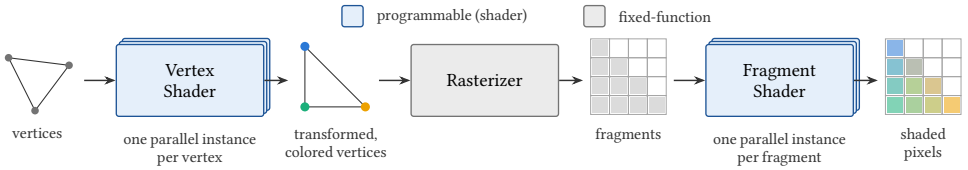


Fig. 1. A simplified graphics pipeline of the early programmable-shader era. A programmable *vertex shader* transforms each vertex and assigns it a color; a fixed-function rasterizer converts a triangle into the *fragments* it covers; and a programmable *fragment shader* computes each pixel's final color from the interpolated vertex colors. Every vertex and fragment is processed independently, so each shader stage is inherently data-parallel.

simultaneously. This model, known as *Single Instruction, Multiple Data* (SIMD) [116], had appeared previously on other hardware and was a natural fit for the parallel nature of graphics workloads.

The additional flexibility offered by programmable hardware allowed users to more easily write custom shaders using specialized *shading languages*. It also opened the door to other uses of GPUs beyond graphics. In 2001, Larsen and McAllister [197], building on prior work by Hoff et al. [154], described a matrix-multiplication procedure on the GeForce 3 that "tricked" the hardware into processing the input matrices by representing them as images. Before long, GPUs were being used in a variety of domains, including robotics [200], numerical and physical simulation [46, 143, 211], and Boolean satisfiability [342]. As general-purpose GPU (GPGPU) programming continued to proliferate, processors tailored for graphics workloads no longer sufficed for the full range of applications; demand grew for hardware that could support both graphics (which still dominated) and these emerging use cases.

2.2 GPU Hardware and the SIMT Model

In 2006, NVIDIA released the Tesla microarchitecture [216], with AMD's TeraScale [157] following shortly after. These devices implemented a "unified shader model," wherein every component of GPU hardware became equivalent in capability. In practice, this meant that GPUs became homogeneous collections of directly programmable parallel processors, no longer tethered to the basic structure of graphics workloads, dramatically increasing their flexibility. This change proved successful; while hardware has continued to evolve (as we will see in detail in Section 2.6), modern GPUs still share the same fundamental design [6, 18, 245, 247].

Despite its success, the unified shader model, as realized by Tesla and its descendants, presents a very thin abstraction over the hardware, making GPUs extremely challenging to program. Programmers are forced to reckon directly with factors like the performance characteristics of RAM and manual cache management to write efficient code.

To better understand these challenges, the rest of Section 2.2 explores the GPU hardware and programming model at a particular point in time: NVIDIA's Pascal [241] microarchitecture. This is a good starting point for understanding the basics of GPU programming, since Pascal is complex enough to reveal many of the subtleties of the SIMT programming model but does not yet include any of the newer features that would later complicate and ultimately undermine it.

2.2.1 The Streaming Multiprocessor. A Pascal-era GPU consists of many *streaming multiprocessors* (SMs), hardware units roughly analogous to CPU cores that execute independently and in parallel. The number of SMs on a GPU varies between architectures but has grown over time. Each consists of many *CUDA Cores*—basic compute units for processing integer or floating-point operations—and *warp schedulers*, which decide at each clock which computations to assign to which Cores. Each warp scheduler is responsible for a group of 32 CUDA Cores. Pascal's SMs feature two warp schedulers for a total of 64 Cores, although SMs on newer architectures have four. Each warp scheduler has an associated program counter, meaning that CUDA Cores administered by the same warp scheduler

always execute the same instruction at the same time. Each warp scheduler is tied to a register file, which stores data private to its associated CUDA Cores; one Core cannot access the parts of the register file belonging to another. In the literature, the term "warp scheduler" sometimes refers to both the actual scheduler and the Cores for which it is responsible, but we distinguish the two.

In addition to their CUDA Cores, registers, and warp schedulers, SMs contain fast SRAM that is accessible by all their CUDA Cores but not by other SMs. As on the CPU, this SRAM can be used as an automatically populated cache; unlike on the CPU, however, it can also be used as manually managed scratchpad memory, giving programmers control over when data is loaded into and evicted from it. Beyond the SRAM is DRAM, the GPU's main memory, which is accessible by all SMs. The DRAM is vastly larger than the SRAM, but it is not located on the same physical die as the SMs, so requests to it are very slow relative to registers or SRAM.

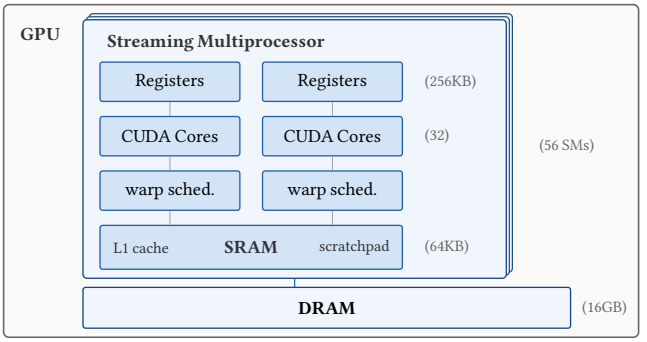


Fig. 2. Simplified SIMT hardware organization, annotated with NVIDIA Pascal P100 capacities. A GPU contains many streaming multiprocessors (SMs); each SM has two warp schedulers, two groups of CUDA Cores, a large partitioned register file, an L1 cache (SRAM), and programmer-managed shared memory (scratchpad). All SMs share the GPU's off-chip DRAM.

Pascal's DRAM and SRAMs have particular access behaviors designed to maximize data-movement speed in common GPU applications. For instance, reads and writes to DRAM occur at a 128-byte granularity; that is, when data is requested from DRAM, the hardware returns an entire 128-byte chunk of data, starting at the 128-aligned address containing the request. When multiple CUDA Cores under the same warp scheduler issue simultaneous requests to DRAM, the hardware groups these requests to minimize the number of chunks that must be loaded; this process is known as *memory coalescing*. In the common case, this allows each of the 32 CUDA Cores attached to a warp scheduler to simultaneously load one 4-byte word[1] or float in a single request to DRAM; typically this occurs when each Core requests a consecutive word. This coalescing process extends to larger data types; if, for example, each Core requested a vector of four floats (for a total request size of 512 bytes across all 32 Cores), the hardware would attempt to coalesce those requests into four chunks and thus four DRAM loads. Accesses that can be perfectly packed into the minimal number of chunks for the requested amount of unique data (1 chunk for 128 bytes, 4 chunks for 512 bytes, etc.) are called *coalesced* [250], while accesses that span more chunks are called *uncoalesced*.

SRAM, on the other hand, is divided into 32 *memory banks*, with its address space striped across these banks: consecutive 4-byte words map to consecutive banks, so address `0x00` would be in bank 0, address `0x04` in bank 1, address `0x08` in bank 2, and so on. Every 128 bytes, the bank numbers wrap around; address `0x7C` (i.e., 124) maps to bank 31, but `0x80` (128) maps to bank 0. This structure is designed to allow the 32 CUDA Cores associated with a warp scheduler to each read consecutive words from SRAM in parallel. Each memory bank acts independently of the others, but accesses to different addresses within the same bank (e.g., `0x00` and `0x80`) must be handled sequentially: an attempt by two CUDA Cores to simultaneously request different words from the same SRAM memory bank results in a *bank conflict* [264], which delays data movement. Figure 3 illustrates coalesced, uncoalesced, and bank-conflicting access patterns.

[1]GPUs are primarily designed for 32-bit, not 64-bit, computation.

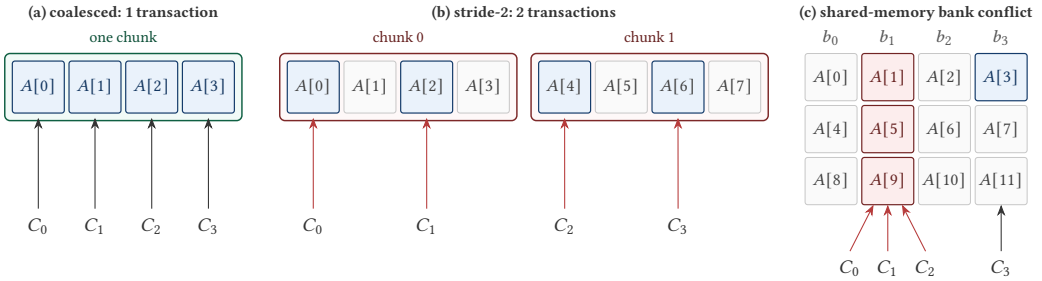


Fig. 3. Access patterns on a fictional GPU with 4 Cores per warp scheduler, 4-word chunks, and 4 shared-memory banks (real NVIDIA GPUs use 32 of each). In (a), consecutive loads coalesce into one transaction; in (b), loads touch two chunks and fetch twice the data they need; in (c), word $A[i]$ lives in bank ($b_{i \bmod 4}$), so Cores C_0, C_1, and C_2, which access $A[1]$, $A[5]$, and $A[9]$, serialize on bank 1 while C_3 reads $A[3]$ in parallel.

2.2.2 The SIMT Programming Model. While understanding the hardware is important for performance, certain of its details are abstracted away when actually writing code for GPUs. In particular, programmers primarily interact with GPUs via the SIMT programming model, which first appeared in CUDA [239] alongside Tesla but has since been adopted by most major GPU manufacturers. In this survey, we use the terminology associated with CUDA, but HIP [8], Metal [19], and OpenCL [322] use different terms for many concepts. These differences are summarized in Table 1.

In the SIMT model, programmers control the behavior of their programs, or *kernels*, by specifying the behavior of individual *threads*. Threads constitute the lowest level of the GPU's *execution hierarchy* (also called the *compute hierarchy*), representing the most basic compute resource available on the GPU. Threads each have their own ID and *local memory*, which is inaccessible to other threads; local memory sits at the bottom of the *memory hierarchy*, mirroring the execution hierarchy. Threads execute in lockstep with others in the same *warp*—a group of 32 consecutive threads whose first thread has an ID that is a multiple of 32—while threads in different warps operate independently.

Programmers often assign the same instructions to most or all of the threads executing a kernel, leveraging the fact that each thread sees different local data to perform computations over large amounts of data in parallel. One common intuition for this sort of parallelism is to think of a thread's ID as an index into a "parallel `for` loop" wrapping a program; when executing a statement like `out[threadIdx.x] = in[threadIdx.x]`, each thread writes a different cell of `in` into `out`.

However, threads can make control decisions based on their IDs or local data, leading to situations in which different threads execute entirely different instructions. When two such threads belong to different warps, they can proceed in parallel, but threads within the same warp must execute synchronously. If two threads in the same warp take different branches, execution is *masked*, or *predicated* [158]: threads are selectively enabled or disabled based on whether they pass or fail a guard condition. In Figure 4, only the first 16 threads of a warp pass the guard on line 1, and the other

Table 1. Terminological differences between CUDA, HIP, Metal, and OpenCL. The size of a warp equivalent varies across manufacturers; a CUDA warp and a Metal SIMD group, for example, have 32 threads, while a HIP wavefront has 64 work-items.

CUDA [239] — NVIDIA	**HIP** [8] — AMD	**Metal** [19] — Apple	**OpenCL** [322] — Cross-platform
Thread	Work-item	Thread	Work-item
Warp	Wavefront	SIMD group	Sub-group
(Thread) Block	Work-group	Threadgroup	Work-group
Grid	Grid	Grid	NDRange (N-Dimensional Range)
Local Memory	Local Memory	Thread Memory	Private Memory
Shared Memory	Shared Memory	Threadgroup Memory	Local Memory
Global Memory	Global Memory	Device Memory	Global Memory

16 are disabled, effectively sitting idle while the success branch executes. After that branch is done, the situation reverses: the first 16 threads are disabled while the others are enabled, and the `else` branch proceeds. This pattern, wherein threads in the same warp are assigned different instructions, is called *warp divergence*; it causes unnecessarily sequential execution of otherwise parallel code and is a common source of performance degradation in GPU kernels.

```
1 if (threadIdx.x < 16) {
2   x++;
3 } else {
4   x--;
5 }
```

Fig. 4. Divergent control flow based on thread ID.

One level up the execution hierarchy, the threads and warps executing a kernel are organized into *blocks*. The number of threads per block is configurable at each invocation, or *launch*, of a kernel, up to a hardware-dependent maximum. This size is usually a multiple of 32, however, so that the threads within each block can be evenly distributed into warps. Threads in the same block but not the same warp execute independently (i.e., in parallel), but they are guaranteed to see the same *shared memory*—the level of the memory hierarchy matching the block's place in the compute hierarchy. Shared memory (SHMEM) is the primary medium for intra-block communication; to avoid data races, programmers must use barrier instructions to synchronize all the threads in a block after a shared-memory operation.[2] Every thread in the block must reach these barriers, or the kernel will experience *barrier divergence*, which is undefined behavior.

Different blocks have separate shared memory and must instead communicate through *global memory*, which sits at the top of the memory hierarchy and is shared by the entire collection of blocks executing a kernel, called the *grid*. Like the number of threads per block, the number of blocks in the grid is configurable when the kernel is launched by the CPU, or *host*. In contrast to the host, the GPU executing a kernel is often called the *device*.

2.2.3 Latency Hiding and Occupancy. Having discussed both the SIMT model and structure of GPU hardware, we can now consider how one maps onto the other. One might expect that CUDA performs a naïve one-to-one mapping of threads to CUDA Cores, warps to warp schedulers, and blocks to SMs, but the reality is more complicated. In fact, each CUDA Core hosts multiple threads, each warp scheduler multiple warps, and each SM multiple blocks; at each clock cycle, the warp scheduler's primary function is to select which logical warp is active on its CUDA Cores. The memory hierarchy is similar: the register file corresponds to the local memory of the CUDA Cores' threads, SRAM to the shared memory of each SM's blocks, and DRAM to global memory for the grid.

The benefit of this many-to-one mapping is that when a warp stalls on a high-latency operation like a DRAM load, its warp scheduler can dynamically decide to pause that warp, leaving its execution state stored in its register file. A new warp can then be loaded onto the scheduler's Cores, executing until it too encounters a long instruction, at which point the old warp can be resumed from registers to pick up where it left off. In this way, the latency associated with slow operations is *hidden* by scheduling other computations concurrently on the same hardware. This dynamic scheduling capability is one of the GPU's most important hardware optimizations, allowing the grid to contain more blocks than the GPU has SMs and individual blocks to contain more warps than SMs have warp schedulers. To meet the SIMT model's mandate that warps within the same block have the same view of SHMEM, the hardware must simply ensure such warps are always scheduled on the same SM. No guarantees are made about when or where one block will execute relative to another.

The *occupancy* [259] of an SM is the ratio of active warps on that SM to the maximum number of warps it could schedule concurrently; the greater the occupancy of a GPU's SMs for a given kernel, the better it is at hiding latency. Thus, to maximize occupancy, GPU programmers frequently *oversubscribe* the hardware, launching kernels with far more blocks and threads than the GPU can

[2]Use of barriers rather than locks means that much of the existing research on lock-based synchronization does not translate to GPUs [213, 380].

execute simultaneously. This minimizes the frequency with which a warp is stalled with no other warps eligible to be scheduled in its place.

2.3 A Sample of GPU Programming and Kernel Optimization

Having discussed the SIMT model abstractly, we now turn to implementing and optimizing kernels within it. To optimize a kernel, one must first understand how its speed is limited, or *bound*: a kernel is *compute-bound* if it is bottlenecked by the speed at which CUDA Cores can perform arithmetic operations and *memory-bound* if it is bottlenecked by how quickly the GPU can move data to and from global memory. The roofline model [363] visualizes this notion by plotting a kernel's performance against its *arithmetic intensity*, or its ratio of arithmetic operations to memory operations. The "roofline," as depicted in Figure 5, is formed by intersecting lines representing the GPU's maximum memory and arithmetic bandwidths; as these parameters differ across architectures, different GPUs have different roofline profiles. The two lines intersect at the *ridge point*; kernels to the left of the ridge point are memory-bound, while kernels to its right are compute-bound.

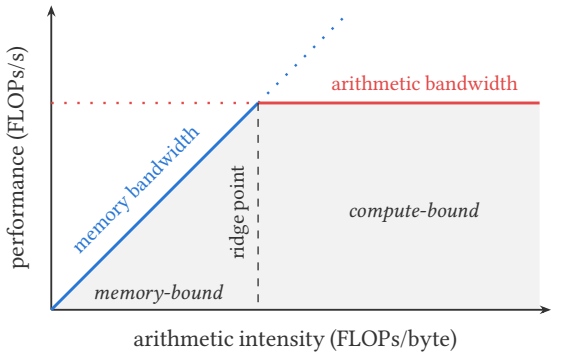


Fig. 5. The roofline model [363]. A kernel's maximum attainable performance is capped by the lower of two ceilings.

In practice, kernels rarely reach the roofline [68], but the model can still tell us where our optimization effort is best spent. We begin with matrix transpose, an operation that switches the row and column indices of a matrix. It performs little arithmetic, so it is a memory-bound kernel; accordingly, our optimization effort will focus on improving memory-access patterns.

2.3.1 Optimizing Transpose. The transpose algorithm is easy to understand and specify: the element at row i and column j is written to row j and column i. Figure 6 depicts a straightforward implementation of this algorithm in CUDA, which surfaces many details of the SIMT model. The body of a `__global__` function describes the work of a single thread, and launching a kernel runs that function on every thread in the grid. Identifiers like `threadIdx.x` and `blockIdx.x` associate each thread and block with a unique ID, which they use to determine which part of the input they should process. The static constant `SIZE` refers to the dimensions of the matrix being transposed, here assumed to be square for simplicity. The matrix is further divided into square *tiles* with dimensions `TILE_DIM`; each block is responsible for one tile and transposes its portion of the input matrix independently. For now, the main benefit of tiling is to organize our code by breaking the transpose into small, separate chunks, mirroring the compute hierarchy; later, we will see how tiling can improve performance by exploiting data locality.

```
1  __global__ void transposeNaive(float *odata, const float *idata) {
2    const int block_x = blockIdx.x % (SIZE / TILE_DIM);
3    const int block_y = blockIdx.x / (SIZE / TILE_DIM);
4
5    const int thread_x = threadIdx.x % TILE_DIM;
6    const int thread_y = threadIdx.x / TILE_DIM;
7
8    const int col = block_x * TILE_DIM + thread_x;
9    const int row = block_y * TILE_DIM + thread_y;
10
11   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
12    // reads are coalesced, writes are not
13     odata[col * SIZE + row + j] =
14       idata[(row + j) * SIZE + col];
15 }
```

Fig. 6. A naïve matrix transpose in CUDA.

Depending on a kernel's launch configuration and tile size, a block may not have enough threads to process every element of its tile at once. The loop on line 11 handles this: the first `BLOCK_ROWS`

rows of the tile are transposed in parallel, followed by the next `BLOCK_ROWS`, and so on until the end of the tile. Each thread thus transposes several elements of its column, spaced `BLOCK_ROWS` apart.

Now, consider our kernel's memory-access patterns. An easy way to do this is to choose constants for the various values—say `SIZE = 1024`, `TILE_DIM = 32`, and `BLOCK_ROWS = 8`—and examine the addresses that each thread reads from and writes to. For the first three threads in block 0, the read addresses on line 14 are 0, 1, and 2, but the write addresses on line 13 are 0, 1024, and 2048. This points to a coalescing problem: reads are coalesced, but writes are *strided* by 1024, meaning the index into `odata` differs by 1024 between consecutive threads. Thus, while a warp's reads all fit within a single 128-byte chunk—one trip to DRAM—writes require 32 separate trips, one for each thread. This uncoalesced write results in an unacceptable performance penalty, and, worse, the problem is inherent to the transpose algorithm: one might attempt to swap the reads and writes so that writes are coalesced, but this causes an uncoalesced read pattern instead! The only solution is to eliminate the strided global access pattern entirely.

Shared memory can be used for exactly this purpose. Although it is much smaller than global memory, shared memory is quicker to access, and accesses do not need to be chunked. Here we see another benefit of decomposing our input matrix into 32×32 tiles: we can load our tile into shared memory using a coalesced read, do the strided work on-chip, and eventually perform a (coalesced) write back to global memory. The code in Figure 7 implements this process: each block reads its tile from `idata` into `tile` using a coalesced read, storing it in its original orientation. The transposition happens when the tile is read back out on lines 23 and 24 using the `row` and `col` values that were swapped on lines 19 and 20. Crucially, this write to `odata` uses the same coalesced pattern that the read from `idata` did; every global access is coalesced, and the strided steps are confined to SHMEM, increasing data-movement speed. This pattern works with many different tile sizes, but in practice, these sizes are chosen to fit precisely into shared memory.

```
1  __global__ void transposeCoalesced(float *odata, const float *idata) {
2    __shared__ float tile[TILE_DIM * TILE_DIM];
3
4    const int block_x = blockIdx.x % (SIZE / TILE_DIM);
5    const int block_y = blockIdx.x / (SIZE / TILE_DIM);
6
7    const int thread_x = threadIdx.x % TILE_DIM;
8    const int thread_y = threadIdx.x / TILE_DIM;
9
10   int col = block_x * TILE_DIM + thread_x;
11   int row = block_y * TILE_DIM + thread_y;
12
13   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
14     tile[(thread_y + j) * TILE_DIM + thread_x] =
15       idata[(row + j) * SIZE + col];
16
17   __syncthreads();
18
19   col = block_y * TILE_DIM + thread_x; // transpose block offset
20   row = block_x * TILE_DIM + thread_y;
21
22   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
23     odata[(row + j) * SIZE + col] =
24       tile[thread_x * TILE_DIM + thread_y + j];
25 }
```

Fig. 7. A coalesced matrix transpose.

Unfortunately, this implementation still is not as efficient as we'd like, due to memory bank conflicts. Recall that shared memory is divided into 32 *banks*, each of which can be accessed in parallel, with

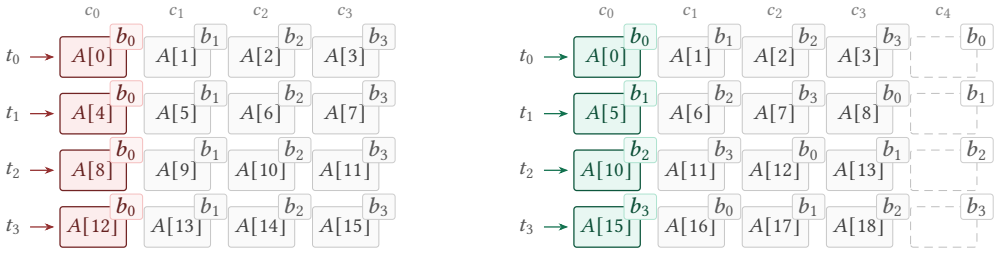


Fig. 8. Bank conflicts occur when threads access data in the same memory bank. In this example, we consider a 4×4 tile of a matrix, stored in `A`, and use a warp size of four threads for simplicity. On the left, naïvely filling `A` with the tile's entries causes values in the same column to be stored in the same bank, so threads accessing such values must serialize. On the right, adding a column of padding to `A` ensures that there is always one more element per row than threads per warp. Threads thus access different banks and can access SHMEM in parallel.

consecutive words assigned to consecutive banks. When our tile data is initially written on line 14, each thread writes to consecutive elements of its tile, so no bank conflicts occur. However, on line 24, each thread accesses an element staggered 32 words from the previous thread, so all the threads in each warp access the same bank. These threads must thus access their data serially, diminishing the benefits of using shared memory.

Luckily, this issue can be resolved straightforwardly: we can add one extra element of "padding" to each row of our tile. Increasing the row size by 1 causes the access pattern on line 24 to be strided by 33 instead of 32; since 33 is coprime with 32, thread 0 accesses element 0 at bank 0, thread 1 accesses element 33 at bank 1, thread 2 accesses element 66 at bank 2, and so on. This technique is illustrated in Figure 8, and the resulting kernel appears in Figure 9, where the size of the `tile` allocation on line 2 and the index expressions on lines 14 and 24 have changed to accommodate the column of padding.

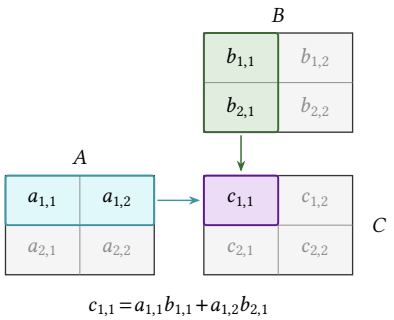
```
1  __global__ void transpose(float *odata, const float *idata) {
2    __shared__ float tile[TILE_DIM * (TILE_DIM + 1)];
3
4    const int block_x = blockIdx.x % (SIZE / TILE_DIM);
5    const int block_y = blockIdx.x / (SIZE / TILE_DIM);
6
7    const int thread_x = threadIdx.x % TILE_DIM;
8    const int thread_y = threadIdx.x / TILE_DIM;
9
10   int col = block_x * TILE_DIM + thread_x;
11   int row = block_y * TILE_DIM + thread_y;
12
13   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
14     tile[(thread_y + j) * (TILE_DIM + 1) + thread_x] =
15       idata[(row + j) * SIZE + col];
16
17   __syncthreads();
18
19   col = block_y * TILE_DIM + thread_x;
20   row = block_x * TILE_DIM + thread_y;
21
22   for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS)
23     odata[(row + j) * SIZE + col] =
24       tile[thread_x * (TILE_DIM + 1) + thread_y + j];
25 }
```

Fig. 9. A bank-conflict-free, coalesced transpose.

2.3.2 Optimizing Matrix Multiplication. We have discussed optimizing matrix transpose in detail, but this kernel is simple compared with many others commonly run on GPUs. We now consider a more complex kernel: matrix multiplication, or "matmul." This is arguably the most fundamental operation in modern GPU programming; in graphics, for example, matrix multiplication may be used to rotate 3D objects, while in machine learning, matrices often hold weights associated with a neural network [267]. A thorough treatment is not feasible here, as high-performance implementations, such as those found in cuBLAS [252], may span hundreds of lines of code; instead, our goal is to continue building intuition for GPU optimization techniques.

Like transpose, matrix multiplication is a simple algorithm: it takes two input matrices and produces a single output matrix, the elements of which are the dot product of each row of the first input with each column of the second. General matrix multiplication (GEMM, or SGEMM when performed on single-precision floating-point numbers) is a standard form of matrix multiplication that computes $C = \alpha AB + \beta C$, where A, B, and C are matrices and α and β are scalars. Figure 10 shows the process of producing an element of the output matrix, along with a naïve implementation of GEMM.

Unsurprisingly, the GEMM in Figure 10 is inefficient, achieving barely 1% of cuBLAS's performance [45]. Some of its shortcomings resemble those in our naïve transpose, but it has an additional wrinkle:




```
1  __global__ void gemm(float *C, const float *A, const float *B,
2                       float alpha, float beta) {
3
4    const int thread_id = blockIdx.x * blockDim.x + threadIdx.x;
5    const int x = thread_id % SIZE;
6    const int y = thread_id / SIZE;
7
8    if (y < SIZE && x < SIZE) {
9      float sum = 0.0f;
10     for (int k = 0; k < SIZE; k++) {
11       sum += A[x * SIZE + k] * B[k * SIZE + y];
12     }
13     const int i = x * SIZE + y;
14     C[i] = alpha * sum + beta * C[i];
15   }
16 }
```

Fig. 10. Left: how rows and columns of the input matrices combine to produce the output matrix. Right: a naïve GEMM in CUDA. A simple matmul is a special case of GEMM where $\alpha = 1$ and $\beta = 0$.

in an $n \times n$ matmul, each input value is used in n different dot products, whereas each value in a transpose is used once. Tiling matmul's input matrices therefore improves performance by allowing these reused values to be stored together in local or shared memory.

Many simple tiling strategies use a single shared-memory buffer for each matrix; the example in Figure 11 loads a pair of tiles—one from each matrix—into SHMEM, then performs the relevant arithmetic. However, this arithmetic cannot begin until both tiles are loaded, and it must complete before the next pair can load. Since memory units like DRAM can process loads in parallel with the GPU's compute units, this approach leaves performance on the table.

```
__global__ void gemm(float *C, const float *A, const float *B,
                     float alpha, float beta) {
  __shared__ float aTile[TILE_DIM * TILE_DIM];
  __shared__ float bTile[TILE_DIM * TILE_DIM];

  const int block_x = blockIdx.x % (SIZE / TILE_DIM);
  const int block_y = blockIdx.x / (SIZE / TILE_DIM);
  const int thread_x = threadIdx.x % TILE_DIM;
  const int thread_y = threadIdx.x / TILE_DIM;
  const int col = block_x * TILE_DIM + thread_x;
  const int row = block_y * TILE_DIM + thread_y;

  float sum = 0.0f;
  for (int tile_k = 0; tile_k < SIZE; tile_k += TILE_DIM) {
    aTile[thread_y * TILE_DIM + thread_x] =
      A[row * SIZE + tile_k + thread_x];
    bTile[thread_y * TILE_DIM + thread_x] =
      B[(tile_k + thread_y) * SIZE + col];
    __syncthreads();

    for (int k = 0; k < TILE_DIM; k++)
      sum += aTile[thread_y * TILE_DIM + k] *
        bTile[k * TILE_DIM + thread_x];
    __syncthreads();
  }

  const int i = row * SIZE + col;
  C[i] = alpha * sum + beta * C[i];
}
```

Fig. 11. A single-buffered, tiled GEMM implementation in CUDA.

We can regain this lost performance via *double buffering*, a technique in which two pairs of tiles—four in total—are maintained in shared memory. After the first pair of tiles is loaded, compute units begin calculating the dot product for that pair; simultaneously, memory units load the next pair of tiles. By the time the first dot product is complete, the second load will have finished, and work can begin on that pair while the next one loads. This process, illustrated in Figure 12, ensures that the hardware is always busy. Double buffering is but one of many complex ways to optimize matrix multiplication; we will not attempt to enumerate every optimization strategy here. Instead, we direct interested readers to Boehm [45]'s matmul-optimization worklog and Hijma et al. [152]'s survey for further study.

2.4 Main Themes of GPU Programming

So far, we have discussed the SIMT programming model, but SIMT has not always been the dominant mode of GPU programming, and it is unlikely to remain so forever. Accordingly, we now broaden our view beyond SIMT and identify three main themes of GPU programming that transcend the details of its hardware and software model.

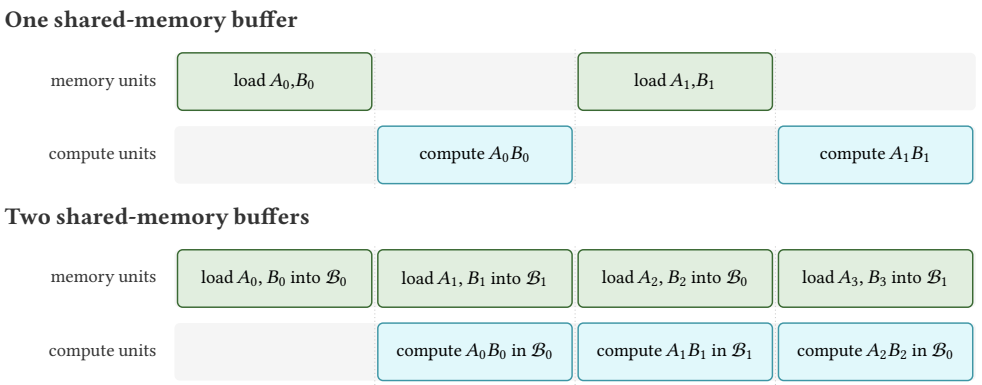


Fig. 12. Top: one SHMEM buffer means that either memory units (e.g., DRAM) or compute units (e.g., CUDA Cores) are idle. Bottom: double buffering ensures that the hardware is always busy. $\mathcal{B}_0$ is buffer 0; $\mathcal{B}_1$ is buffer 1.

Theme I: Slow Code Is Wrong

In many domains, performance takes a back seat to productivity. Many high-level languages and programming patterns are designed to make code easier to read, write, or understand; often this comes at a nontrivial performance cost. Depending on the application, this trade-off may or may not be worth it; for example, one might use a garbage-collected language for writing a website but not for writing an operating system. GPU applications, however, rarely prioritize ergonomics over efficiency: **performance is a first-order concern, in many cases on par with correctness.**

One reason for this is that individual SMs are significantly less powerful than CPU cores, despite their conceptual similarities. Data-parallel workloads by definition consist of many tasks that can be performed independently, and economic factors mean that hardware consisting of many simple processors usually achieves higher throughput for such workloads than fewer, faster processors at the same price point [287]. For this reason, many hardware optimizations (e.g., branch prediction, memory prefetching, and out-of-order execution) that we take for granted on CPUs are entirely absent on GPUs, the chip space and silicon they would require instead being used to house more cores. Without these hardware features, software optimizations are commensurately more important, often improving performance by multiple orders of magnitude [289].

Another reason is that GPUs are expensive specialty equipment, finely tuned for performance-sensitive use cases like graphics, deep learning, and scientific computing. They are thus unlikely to be used for other kinds of workloads, and developers who don't care about performance are similarly unlikely to target a GPU in the first place. Many GPUs are also rented by their users [15, 123], who pay per unit of time or compute; slow code thereby incurs a direct monetary cost to its author or their employer, further disincentivizing GPU usage for performance-insensitive tasks.

GPU programmers care deeply about performance, and this care has shaped the dominant programming languages and paradigms throughout the GPU's history. While *mass-market* [324] domain-specific languages (DSLs) like PyTorch [272] and TensorFlow [2] are popular for specific use cases, low-level languages like CUDA remain the default choice for general-purpose kernel implementation [25]. Languages that see wide use often sacrifice things like expressiveness [83, 343] or simplicity [239, 289] rather than performance. Numerous performance-engineering worklogs [45, 329], educational materials [94, 139, 141, 142], and microbenchmarking research papers [3, 171, 173, 209, 219] also evince the premium that GPU programmers place on performance.

Theme II: Specification Is Easy, Optimization Is Hard

As we saw in Section 2.3, kernels like matrix transpose or multiply have straightforward naïve implementations that quickly balloon in complexity as we optimize them. These examples are instances of a more general pattern: **most kernels are simple to describe and specify but complex to implement efficiently.** For example, a popular tutorial [45] on optimizing GEMM uses only 14 lines of CUDA for its initial, naïve kernel but 213 lines for its most optimized version. NVIDIA's cuBLAS [252] library of linear-algebra operations—the standard benchmark for GEMM performance—is over 100 MB in size and contains over 5,000 implementations of GEMM, the best of which is selected at run time based on the characteristics of the matrices being multiplied [302].

As we discuss in greater detail in Sections 3 and 4, a common goal in GPU research is building better tools for managing this complexity, balancing the desire for elegant abstractions with the need for efficiency. Low-level languages like CUDA expose the structure of the GPU and give programmers precise control over how programs are mapped to hardware; in turn, they place a commensurate burden on programmers to perform that mapping correctly and efficiently. Kernel performance in these languages is more often limited by poor implementation than by the hardware itself [68]—that is, most kernels do not reach the roofline.

Higher-level approaches can automate this mapping, but they must contend with the fact that the GPU optimization space is vast and few optimizations are applicable in all circumstances. Even simple program transformations can have dramatically different effects on performance depending on the kernel in question [31]. Consider loop unrolling, which decreases the number of instructions necessary to execute a loop but increases *register pressure*—the number of registers in use by each thread. This can result in lower maximum occupancy on SMs, since each thread now requires more space devoted to storing its state when suspended [350]. Whether this is a worthwhile trade depends on the performance characteristics of the kernel in question: compute-bound kernels will benefit from faster loop execution, while memory-bound kernels will suffer from reduced occupancy.

This is an example of a common pattern: general-purpose optimizing compilers for GPUs struggle to find and apply the appropriate optimizations in all circumstances [136]. Instead, GPU compilers often limit themselves to a particular domain with well-understood performance characteristics—like graphics [270, 283], tensor algebra [73, 184, 353, 368], or deep learning [69, 112, 195, 297]—or leave much of the decision-making about optimization in the hands of the programmer [137, 163, 288, 369]. Compilers that do try to optimize general-purpose code frequently rely on *autotuning* [359] or *iterative compilation* [44] to search for the best results [135, 179, 289]. These strategies, popularized by the ATLAS linear-algebra library [360], try various optimizations with different parameters and evaluate the resulting programs to find the ideal compilation path. While useful, these approaches are limited by the optimizations the compiler knows about, so they are usually paired with other methods to identify potential improvements [136].

Conversely, kernels' simple logic often makes them more amenable to analysis. In particular, many kernels—up to 60% [213] or 70% [201]—feature an implicit separation between control and data: their control flow and memory-access patterns do not depend on their input data, a property we refer to as "control-data independence." The idea that one can reliably distinguish values carrying control information (also called *integrity* [201] in some literature) from those that carry input data is a fairly common simplifying assumption when writing, analyzing, and optimizing GPU programs [163]. We can see control-data independence in practice in the examples in Section 2.3: all the access expressions into the input or output matrices or the shared-memory buffers depend only on static constants and thread indices. However, the validity of this assumption is highly domain-dependent; in dense tensor algebra and graphics, it is applicable very often, while it is violated frequently in specific applications like ray tracing [10, 361] and chemical process modeling [33].

Theme III: Hardware Is Rapidly, Visibly Evolving

Another consequence of GPUs' thin abstraction layer is that programmers are significantly more exposed to hardware changes than they would be on a CPU. The specifics of CPU hardware change frequently, but, thanks to hardware optimizations, CPUs can present users with a programming model that has not fundamentally changed since the 1970s [287]. By contrast, every generation of NVIDIA GPU since the Volta [243] microarchitecture has introduced features that altered its programming model. We discuss these changes in detail in Section 2.6 and focus here on their impact.

The introduction of Tensor Cores [258] in Volta and asynchronous data movement in the Ampere [244] and Hopper [245] microarchitectures completely changed how performant GPU kernels were written. Kernels that were efficient on one generation became useless on the next, with migration commonly requiring dramatic restructuring of code. As an extreme example, FlashAttention [90, 91, 305, 375]—a popular machine-learning kernel—required an entirely new algorithm to achieve good performance on Hopper [305], resulting in over a year of lag time between Hopper's release and the development of a usable attention kernel [313]. The disruption caused by hardware evolution affects low- and high-level languages both; kernels that exercise direct control over hardware features must be rewritten for new architectures, while those written in more hardware-agnostic languages

must wait for the compiler, library, and tooling ecosystem to catch up before they can be used. As a result, **one of the central questions of modern GPU language research has been whether the compiler or the programmer should be responsible for the performance of code**: relying too much on the compiler makes languages brittle as hardware evolves [367], while leaning too much on the programmer makes them challenging and labor-intensive to use [128, 163].

Along with differences between generations of GPUs from the same manufacturer, programmers are exposed to variance in design across manufacturers. While AMD's TeraScale [157] and NVIDIA's Tesla [216] architectures were quite similar, their more recent designs have diverged significantly. AMD's newest chips [6, 7], for example, feature fewer fixed-function units than NVIDIA's but come with an extra L3 cache layer for programmers to consider. AMD's Matrix Cores [312] and Intel's XMX [165] engines likewise began similarly to NVIDIA's Tensor Cores, but their programming models on current hardware have become distinct.

Architectural variation also has consequences for tools that analyze the behavior of GPU kernels. These tools commonly make simplifying assumptions about the programs they target to make their analyses more tractable, but these assumptions are frequently invalidated by the release of new microarchitectures [38, 39, 191, 373] or are only applicable to certain manufacturers [41, 99]. Formal-methods tools are thus often forced to specialize on a handful of particular hardware targets.

2.5 Major Challenges of GPU Programming

Having discussed the chief priorities and concerns of GPU programmers, we now turn to some specific difficulties they face, identifying two classes of challenge: *implementation challenges* and *engineering challenges*. Implementation challenges are small-scope difficulties encountered when writing or reasoning about code, like debugging errors or optimizing underperforming kernels. Engineering challenges are larger in scope, encompassing questions like how to balance performance with portability or how to adapt kernels and pipelines to newly released hardware.

2.5.1 Implementation Challenges. Yang et al. [372] and van den Haak et al. [352] analyzed posts on Stack Overflow [317] by topic and content, identifying common challenges faced by GPU programmers; due to the nature of the data set, most of these challenges are small-scope. Many issues, such as poorly documented APIs, unexpected run-time errors, or confusion about data-type conversion, overlap with issues common on the CPU, suggesting that the usual solutions there would also find success on GPUs. However, other classes of problems are unique to GPUs and have little precedent in other domains [108], so we will focus on these.

Debugging correctness errors in GPU kernels is difficult because error reporting is unusually complex; kernels execute asynchronously relative to the host that launched them, so handling errors from the host requires blocking until the device code finishes. This is expensive and typically not an option in production [140], making static bug-finding methods commensurately more valuable.

Common bugs include out-of-bounds or uninitialized access—further complicated by the multilevel memory hierarchy—along with more GPU-specific concerns like improper barrier synchronization. For example, if we had forgotten the `__syncthreads()` on line 17 of Figure 7, it would have been possible for a thread to read uninitialized data out of `tile` on line 24 before the correct data was written to it by a different thread on line 14. The precise behavior of GPU hardware is also often underspecified, creating confusion about details like floating-point rounding modes [124, 212, 351], performance characteristics [171, 187, 209], and memory consistency [12, 314, 345].

Beyond correctness, developers struggle to identify both why their kernels underperform and how to fix performance issues once found [372]. Within individual kernels, challenges include coalescing global-memory accesses, eliminating bank conflicts in shared-memory accesses, efficiently buffering data in shared memory, and avoiding warp divergence. However, performance also depends on

kernel-configuration choices like grid and block dimensions, how to divide work among threads (e.g., tiling), and how many resources to devote to different tasks in a pipelined kernel.

In addition to such broadly applicable optimizations, developers must grapple with optimization techniques particular to certain kernels or domains. *Swizzling* optimizations, for example, involve mapping data and computation onto hardware resources using more complex patterns than the usual approach, in which consecutive threads process consecutive data. While a typical kernel would distribute 64 bytes over a warp by assigning the first thread in the warp the first 2 bytes, the second thread the next 2, and so on, a swizzled kernel might assign bytes 0 and 32 to the first thread, bytes 1 and 33 to the second, and so forth. Figure 13 depicts an example swizzle, which permutes data as it is copied from global memory to SHMEM. For certain kernels in domains like cryptography [35] and linear algebra [60], a creative choice of swizzle can improve data locality or result in superior memory-access patterns.

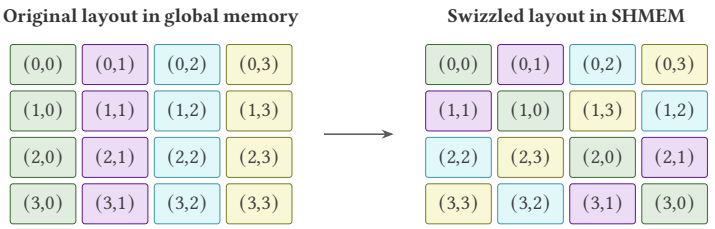


Fig. 13. A swizzle of a 4×4 tile. Labels (r,c) identify the original location of elements. The swizzle moves each entry to column $c \oplus r$ without changing its row, where $\oplus$ is bitwise XOR.

Another important optimization technique is *warp specialization*, in which different tasks in a kernel are assigned to different warps. Warp specialization has its roots in tools like CudaDMA [32], an API for improving kernels' memory bandwidth, and Singe [33], a domain-specific compiler for combustion chemistry. Chemistry kernels are unusual because they frequently lack the usual control-data independence property; the "control flow" of a chemical reaction often depends on the quantity and configuration of the reagents involved, and so the input data to a chemical simulation can flow into branch guards and array accesses. Additionally, data about reaction rates is required to compute the results of reactions, but there is far too much such data to store in shared memory; it must be loaded on demand at each stage of the reaction. Singe solves this problem by specializing the warps undertaking its computations: producer warps are responsible for loading reaction-rate data into SHMEM before it is required by the consumer warps that compute the reactions, reducing the time these warps spend waiting on global-memory loads. Producer-consumer specialization has proven effective for a variety of kernels, achieving up to a 4× performance improvement over usual data-parallel approaches [306], and has grown in importance with the evolution of GPU hardware. We will explore this further in Section 2.6.

Beyond individual kernel optimizations, developers must also consider the performance of entire pipelines in which several kernels run in sequence. A common optimization is *kernel fusion*, in which multiple sequential kernels are combined into one; the fused kernel exhibits equivalent behavior with improved data locality, lower loop overhead, and fewer expensive transfers between global and shared memory. Kernel fusion is closely related to stream fusion [88], further cementing the connection between GPUs and stream processors. However, fusion is complex, and developers must contend not only with how to best fuse their kernels but also with whether to fuse them at all: like most GPU optimizations, fusion can either improve or degrade performance depending on the kernels in question. In particular, fusing high-occupancy kernels with low-occupancy ones can limit the latency-hiding ability of the resulting fused kernel, lowering overall performance. Consider an example where two kernels—call them k_1 and k_2—that compute large outputs are fused with another (k_3) that reduces over the results of k_1 and k_2 to produce a scalar; both k_1 and k_2 must complete before k_3 can run. Without fusion, the result of k_1 will be stored off-chip while k_2 is being evaluated, allowing both kernels to achieve high occupancy. If all three are fused, however, k_1's result must be

held on-chip while k_2 is running, avoiding data movement between global and shared memory but also potentially reducing occupancy by increasing shared-memory usage [115]. Whether avoiding this extra transfer outweighs the reduced occupancy depends on both the kernels and their inputs, so heuristic optimizers often struggle to decide whether to fuse.

On the topic of data transfers, programmers must also ensure that movement between GPUs and their host CPU is both efficient and correct. There are two main ways to perform a host-device data transfer: using a unified address space between the two machines (such as CUDA's Unified Memory [266]), which is simpler but can have worse performance [187], or manually sending data back and forth. In the former case, because the CPU and GPU have different memory models, programmers must take care to use design patterns that respect a compound consistency model that unifies the machines [377]. In the latter case, most languages require programmers to manually track the provenance of pointers; attempting to dereference a pointer to host memory from device code is a common bug. Developers must also respect the implicit assumptions made by kernels about how they will be launched; improperly configured kernels can suffer from poor performance or run-time errors, but few languages feature ways to manage configuration requirements [194].

Finally, programmers often struggle with translating CPU code into GPU code or understanding why such translations do not result in the expected performance gains. These difficulties manifest at many granularities, from low-level concerns like implementing a specific sequential loop nest as a parallel loop in the SIMT model to high-level questions about how to parallelize entire algorithms. Many of these challenges are highly domain-specific, as different application areas of GPU programming all have different performance characteristics, and no one-size-fits-all solution exists.

2.5.2 Engineering Challenges. Beyond questions of implementation, developers must also contend with larger software-engineering concerns. One common problem is verifying that the behavior of an optimized kernel is both correct and equivalent to its original naïve implementation; as we saw in Section 2.3, these two versions of the same kernel can look quite different. Formal-methods tools can help with this task, but, as we will see in Section 4, these can present their own difficulties, and few support the advanced GPU features typically needed for optimal performance.

Another important question is how to strike the right balance of portability and performance for a given kernel or pipeline; GPUs' thin abstraction layers frequently put these factors at odds, as optimizing a kernel typically requires specializing it to a particular architecture. Basic properties of GPUs and their programming model—like the number of threads in a warp or the size of the register file—differ across manufacturers, so naïvely transporting code between them results in subpar performance. Portability also has a correctness dimension: different brands of GPU often disagree on critical details like floating-point rounding modes and the precision of numerical operations, leading identical kernels to yield subtly different results on different machines [124].

Manufacturer differences are not the only dimension of portability programmers must consider; they must be prepared to migrate kernels forward or backward between different GPU generations as well. As discussed in Theme III, GPU hardware is constantly evolving, and a kernel that is efficient on one chip generation may be unacceptably slow on another. Sometimes a rewrite is unavoidable, as in the case of the FlashAttention kernel that required an entirely new algorithm for the release of Hopper [305], but in other cases, kernels can be migrated; the question is how to do so effectively and whether the migration can be done automatically.

As GPUs increasingly handle sensitive data (e.g., zero-knowledge proofs [221]), an emerging challenge has been the security and privacy of GPU computation. GPUs' unique architectures expose them to different kinds of vulnerabilities than those common in traditional research [354]; for example, GPU processes are not isolated from one another, so they can peek into each other's memory [280]. Yandrofski et al. [371], meanwhile, describe a procedure for designing adversarial

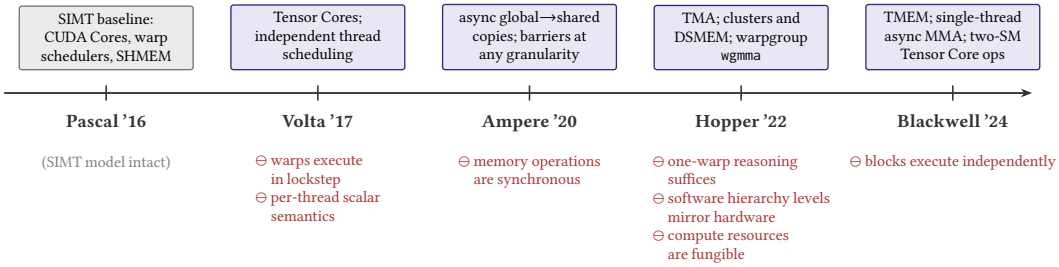


Fig. 14. Five generations of NVIDIA microarchitectures. Above the timeline: the major hardware capabilities each generation introduced. Below: the SIMT assumptions that those capabilities removed.

kernels that starve others for compute and memory resources when executing concurrently on the same device. This topic is too wide to cover in detail here—we direct readers to Mittal et al. [231]'s and Wang and Oswald [357]'s surveys for more information—but we note that programming-languages- and formal-methods-based approaches to these problems are scarce [147].

By far the largest challenge in recent years, however, has been the shift toward deep learning as the predominant mode of GPU programming. For most of their history, GPUs have been used primarily for graphics, and their hardware and programming model reflect this. As we have discussed, SMs were designed to efficiently implement shaders, and the SIMT model itself descends from stream processing [51, 238]. But, while these design decisions were a natural fit for programming the graphics pipeline, they may not be appropriate for workloads in other domains. Moreover, as GPUs have evolved to favor deep learning, their hardware has changed in ways that have eroded the most fundamental assumptions of the SIMT model, leaving developers scrambling to reconcile old programming paradigms with the reality of new hardware [68, 149, 325, 337]. The poster child of this erosion is the Tensor Core.

2.6 Tensor Cores: Beyond SIMT

The release of the Volta [243] microarchitecture in 2017 brought two changes to GPU hardware that would dramatically impact the SIMT programming model. The first, *Independent Thread Scheduling* (ITS) [256], removed the hardware restriction requiring all CUDA Cores managed by the same warp scheduler to execute synchronously. In SIMT terms, this means that threads within the same warp no longer need to execute in lockstep; warp divergence still incurs a performance penalty [364], but the cost is not nearly so large as it was prior to Volta. To allow the restoration of warp-synchronous behavior after divergent branching, Volta also added new barrier primitives to synchronize all the threads within a warp. Kernels written before Volta had not needed such synchronization, however, and became vulnerable to data races and barrier divergence on new hardware—a major backward-incompatible change, with many more soon to follow, as Figure 14 summarizes.

Volta's second, more influential change was the introduction of *Tensor Cores* [258], which broke another fundamental assumption of the SIMT model: that the behavior of a kernel can be specified at a per-thread granularity. Tensor Cores are fixed-function units for fast matrix multiplication based on systolic arrays [192]; every warp scheduler on Volta GPUs has an associated pair of Tensor Cores [20], which operate in parallel with their associated CUDA Cores. The programming model for Tensor Cores requires them to be invoked *collectively* by groups of threads (quarter-warps—or *quad-pairs*—on Volta [40], but this would change with later architectures [261, 262]). If a Core is invoked with the wrong number of threads, execution can hang or produce invalid output; Tensor Cores therefore require programmers to reason about groups of threads at a time, violating SIMT's usual view of each thread as one independent iteration of a parallel `for` loop.

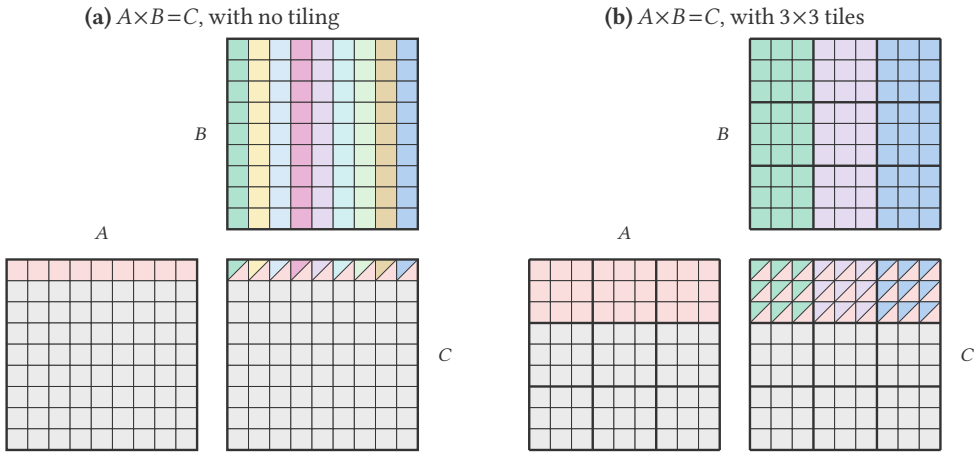


Fig. 15. Two approaches to matrix multiplication. In (a), threads compute C one scalar cell at a time; each cell is the sum of products of a cell-row of A and a cell-column of B. In (b), warps cooperate to compute C a tile at a time using Tensor Cores; one tile of C is the sum of products of a tile-row of A and a tile-column of B.

Unlike CUDA Cores, which operate on scalars, Tensor Cores operate on tensors, requiring a fundamentally different programming model. Their inputs are small—8×4 and 4×8 on Volta [40] and up to 256×32 and 32×256 on later GPUs [171, 262]—but kernels often operate over matrices many times larger or with more dimensions. Tiling, previously one optimization among many, thus became a prerequisite for using the Cores; we illustrate the approach visually in Figure 15. Additionally, Tensor Cores expect their input tiles to be laid out in specific ways—corresponding to low-level details of their underlying hardware—that are different from how tensors are typically arranged; rather than being contiguous in memory, inputs must be distributed across the threads invoking the Tensor Core [40]. Accordingly, matrix multiplication became a process of swizzling inputs into Core-compatible tiles; these tiles, rather than scalars, became the basic unit of computation.

Upon release, the only way to program Tensor Cores in CUDA was via the unstable `wmma` (Warp Matrix Multiply Accumulate) preview library [20]; APIs like CUTLASS [334] were developed later with simpler, more reliable interfaces, but these do not—and, in CUDA, cannot [25]—guarantee proper collective execution of Tensor Core operations, so their ergonomics remain lacking. Despite the complexity of Tensor Cores, their performance could not be ignored, and other manufacturers like AMD [312] and Intel [165] soon followed with their own systolic units. Today, as much as 90% of a modern GPU's throughput capacity lies in these units [68], so programmers wishing to produce performant kernels must learn to wield them effectively.

Another sea change for the SIMT model came with the release of the Ampere [244] microarchitecture in 2020. Ampere changed the structure of Tensor Cores slightly, combining Volta's two Tensor Cores per warp scheduler into one larger Core. More significantly, though, Ampere introduced hardware support for asynchronous data movement between global and shared memory. Instead of requiring a full warp to request coalesced chunks of data from DRAM, blocking further execution of that warp until the data arrives, individual threads on Ampere can issue asynchronous requests for chunks from global memory and continue with other work until the request completes. As a result, latency hiding alone is no longer sufficient for maximum performance; one needs asynchronous data movement as well [209]. To support these new asynchronous operations, Ampere also introduced fine-grained synchronization barriers, allowing arbitrary groups of threads to coordinate without synchronizing an entire warp or block.

As Tensor Cores continued to process larger matrices faster, asynchronous programming became increasingly important for supplying Cores with data. By the fourth generation of Tensor Cores, they had become so fast that the primary bottleneck for the performance of tensor kernels was no longer the speed of the Cores themselves but rather the ability of the rest of the SM to keep them fed. So, with Hopper [245] in 2022, NVIDIA equipped each SM with a *Tensor Memory Accelerator* (TMA) [265], a new hardware unit designed to satisfy the Tensor Cores' insatiable appetite for data. The TMA expands Ampere's asynchronous data-movement innovations by issuing bulk asynchronous transfers between global and shared memory, accelerating the flow of data into Tensor Cores while freeing the CUDA Cores to do other work [245, 369]. A less obvious benefit of the TMA is that its large requests require less addressing math than many smaller ones, reducing register pressure [181] and improving efficiency by allowing more blocks to be scheduled on each SM.

Hopper also loosened the relationship between the software hierarchy and hardware structure through the introduction of *clusters* of SMs and *distributed shared memory* (DSMEM), which enabled SMs in the same cluster to access each other's SRAM. DSMEM obviated the need for inter-block communication to pass through global memory, functionally adding a level to the SIMT hierarchy—clusters of blocks—that lies between the block and the grid. Unlike the others, however, this new level is not backed by its own hardware resource; SHMEM and DSMEM both map to SRAM under the hood, so programmers must be cognizant of how allocations in one affect data in the other.

Hopper also further evolved the programming model of Tensor Cores by changing the granularity of their execution. On Ampere, Tensor Core operations are invoked by individual warps via a `wmma` [263] instruction; on Hopper, to support larger matrix sizes, they are invoked by a *warpgroup*: four consecutive warps that execute cooperatively. As of Volta, SMs possessed four warp schedulers rather than two (each with one Tensor Core after Ampere), so a warpgroup-level `wgmma` (Warpgroup Matrix Multiply Accumulate) [261] effectively treats each SM's four physical Tensor Cores as a single logical Core [369]. This change invalidated a common assumption resulting from the SIMT model; when reasoning about the correctness (as opposed to the performance) of kernels, one could usually assume that all thread blocks consisted of a single warp [38, 39, 373]. Analysis of kernel behavior post-Hopper, however, must contend with the full complexity of inter-warp interactions.

The increasing heterogeneity of SMs on Hopper gave rise to a warp-specialized programming paradigm—in particular, pipelines in which one producer warp manages the TMA while consumer warps issue work to Tensor Cores. As discussed in Section 2.5.1, warp specialization predates Hopper, but so many heterogeneous hardware units operating in tandem on Hopper's SMs make it practically mandatory for optimal performance [369]. It is so important, in fact, that support for warp specialization was introduced at the hardware level [68], violating yet another SIMT assumption: that all compute resources are fungible.

In 2024, NVIDIA continued to heterogenize the SM in its Blackwell [247] microarchitecture by introducing another kind of memory, *Tensor Memory* (TMEM), designed to give Tensor Cores a low-latency place to accumulate output [171]. Each Blackwell SM has one bank of Tensor Memory, which lies outside the usual SIMT memory hierarchy and has a separate set of instructions for interfacing with it, issued cooperatively by a warp [262]. TMEM further encourages a pipelined programming style, as full utilization of Blackwell SMs requires processing multiple tiles simultaneously; Tensor Cores compute over the current tile, the TMA loads the next tile, and the previous tile is written from Tensor Memory to the output.

However, the performance characteristics of Tensor Memory are not yet fully understood; whether it outperforms the traditional memory hierarchy depends on both matrix size and whether the multiplication is pipelined or performed in isolation. Microbenchmarks indicate that TMEM achieves peak throughput when processing matrices of size 64×64 [171], even though Blackwell's Tensor Cores themselves support operations with outputs of size 256×256 [262]. As a result, efficient kernels

targeting Blackwell must vary the degree to which they rely on TMEM depending on their specific dimensions and data-usage characteristics, resulting in different code from case to case.

Blackwell once again changed the programming model of Tensor Cores. Rather than being issued cooperatively by a warpgroup, each Tensor Core operation is issued asynchronously by a single thread, with results accumulated in Tensor Memory. However, while this operation is issued by one thread, it conceptually occurs at the granularity of a block because each SM has only one logical Tensor Core for use by its resident blocks. Despite this block-level programming model, TMEM and the TMA still require warp- and warpgroup-level operations that occur below the block level, so warp specialization remains necessary for optimal throughput.

At the same time, it became possible for two SMs to collaborate to perform a multi-SM Tensor Core operation, requiring two threads in adjacent blocks in a cluster to synchronously issue the operation [262]—it had not previously been possible for threads in different blocks to cooperate in this way. In fact, before Hopper's clusters, it had not even been possible to assume that any given pair of blocks would be executing at the same time or would be scheduled on different SMs. These dramatic changes to the programming model, along with the fact that Hopper GPUs still see extensive use alongside Blackwells [329], mean that kernels that wish to be portable between architectures require radically different compilation procedures for each target.

At the time of writing, two years into the life cycle of Blackwell, the programming model for GPUs looks vastly different than it did pre-Volta. Many of SIMT's most basic assumptions have been broken, and some [68, 149, 325, 337] have argued that the SIMT paradigm is fundamentally unsuited to the new, heterogeneous, asynchronous reality of GPU programming. Accordingly, we must wonder: what programming model will arise to replace it?

3 A Genealogy of GPU Programming Languages

To design the languages that will support the future of GPUs, it is helpful to understand the languages that were the foundation of their past. In this section, we present a genealogy of programming paradigms for GPUs, tracing the evolution of their software model in tandem with the hardware. Figure 16 depicts this genealogy, starting from early shading DSLs that predate the unified shader model and continuing on to languages for modern hardware; later figures in this section present zoomed-in slices of the overall graph. We identify languages as inheriting from one another when a later language uses or develops ideas that originally appeared in an earlier one; in some cases, this relationship is well known (e.g., Brook [52] and CUDA [51, 238]), while in others, we have identified conceptual connections that have not been previously documented (e.g., the Real-Time Shading Language [283] and user-schedulable languages).

To make this survey tractable, we limit discussion in this section according to two criteria. First, languages' original or primary compilation targets should be GPUs. Many high-performance or scientific computing languages like OpenMP [89], OpenACC [362], Legion [34], Chapel [55], Realm [9], Sequoia [110], and MATLAB [339] can be compiled to GPUs but were primarily designed for other architectures, and full exposition of these languages is beyond the scope of this survey. Likewise, general-purpose languages like Java [127, 145, 146, 167, 202, 281, 370, 376] and Lime [22, 23, 100] that have been extended to target GPUs also fall outside the scope of this survey. Second, languages should be novel or make influential contributions to the GPU ecosystem. Many DSLs exist besides those we describe here—including, but certainly not limited to, Opt [96] for least-squares optimization; OptiML [324] and Latte [347] for machine learning; StencilGen [293] for general stencil computations; and Spark [117], Forma [292], Gator [121], and CoolerSpace [67] for graphics programming—but we focus on those of particular interest to a programming-languages audience.

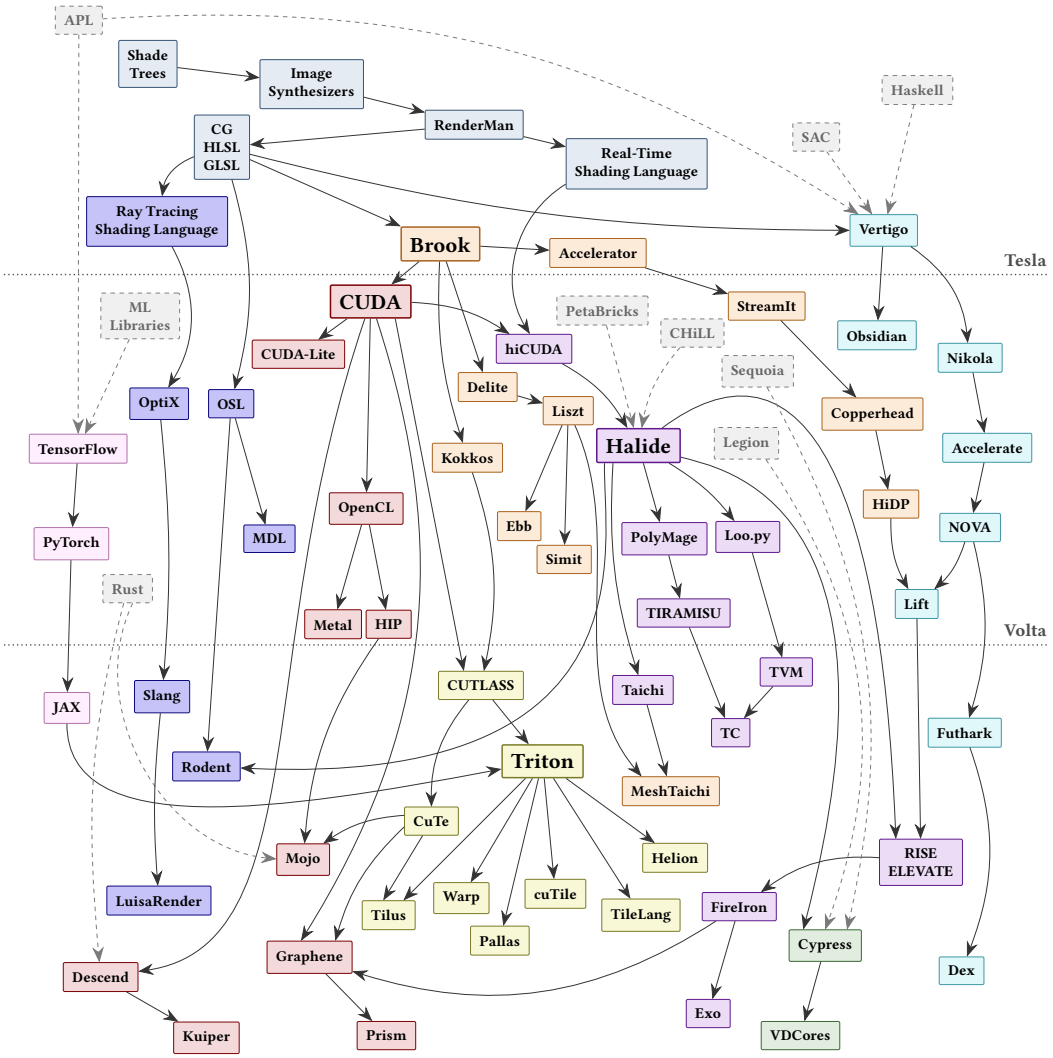
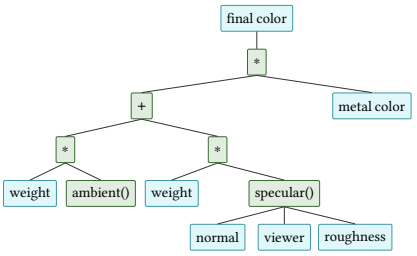


Fig. 16. A genealogy of GPU programming languages, connected by arrows when one inherits its ideas or goals from another. Languages sharing a color are discussed in the same subsection; dashed gray boxes indicate languages primarily used outside the GPU space. Languages are (roughly) temporally sorted, with the horizontal dotted lines indicating the release of major new architectures. In the rest of this section, we present several sub-genealogies, highlighting particular language families in more detail.

3.1 Early Shading Languages

Our genealogy begins with two early graphics frameworks—**shade trees** [83] and **image synthesizers** [278], depicted in Figure 17—from which all modern shading languages descend [117]. These languages predate the modern GPU, but they embody a dichotomy between abstraction and control that will reappear in various guises throughout our discussion of its programming languages.

The shade trees framework allows users to describe the structure of their shaders declaratively as data-flow graphs, or shade trees. Compound shaders, such as lighting or atmosphere shaders, are described by multiple individual trees, which are joined (or "grafted") together by the framework's




```
float a=0.5, s=0.5;
float roughness=1;
float intensity;
color metal_color = (1,1,1);
intensity = a * ambient() +
  s * specular(normal, viewer, roughness);
final_color = intensity * metal_color;
```

```
function turbulence(p)
  t = 0
  scale = 1
  while (scale > pixelsize)
    t += abs(Noise(p / scale) * scale)
    scale/= 2
  return t
```

Fig. 17. Early graphics shading frameworks. Left: a shade tree, from Cook [83]. Top right: the shader program corresponding to the tree, also from Cook [83]. Bottom right: an image synthesizer, from Perlin [278].

compiler into a combined tree. Shade trees' abstraction is simple and modular; shaders need not know each other's implementation details to be grafted together. The trade-off, however, is expressiveness—shade trees do not support any kind of nonlinear control flow or mutable state.

Image synthesizers, meanwhile, are a procedural framework; shaders are imperative programs and can handle the kinds of complex state and control flow that shade trees cannot. Consequently, they impose less structure on shading programs and must rely on weaker abstractions: primarily function signatures at procedure boundaries. Image synthesizers thus lack shade trees' modularity, and building composite shaders from smaller ones typically requires interprocedural reasoning.

The conceptual abstraction-control spectrum delimited by shade trees and image synthesizers shaped the development of many early graphics languages, with many frequently describing themselves in relation to these two poles of design. Pixar's **RenderMan** [138] language, for example, positioned itself toward the center of this spectrum; RenderMan is based on C and is procedural like image synthesizers, but it also imposes additional requirements on program structure to recover some of shade trees' modularity. In particular, RenderMan shaders are specified as classes (in the object-oriented sense), and common graphics objects like surfaces and lights have distinct types and operations. Shader composition occurs via specialized control operators, guaranteeing a uniform abstraction for building modular shaders.

RenderMan's compiler targets a SIMD array-processing architecture but gives programmers the illusion of a sequential semantics, automatically parallelized by the compiler. RenderMan exposes the structure of the graphics pipeline (as opposed to the hardware itself) by allowing programmers to directly describe how often values change during computation. Variables in RenderMan are decorated with *computation rates*: `uniform` variables are constant throughout a shader (although they can differ between multiple instances of the same shader and so are not necessarily constant for an entire program), while `varying` variables change during execution. These rates allow RenderMan to compute the values of expressions that rely only on `uniform` variables when shaders are instantiated, rather than recomputing them on each application of the shader. Programmers had previously performed scheduling optimizations like this by hand, so reifying these details of the shading process as variable annotations represents an early example of graphics languages leveraging domain-specific information to produce faster code automatically.

The **Real-Time Shading Language** (RTSL) [283] built upon the idea of computation rates by exposing even more of the structure of the graphics pipeline. In RTSL, developers write *pipeline shaders*, specifying high-level behavior declaratively for the entire graphics pipeline, rather than for its individual components. Values in a shader can be annotated with *computation frequencies*, which are similar to RenderMan's computation rates but have finer granularity: instead of two rates for uniform and varying computations, frequencies directly describe where in the pipeline an operation

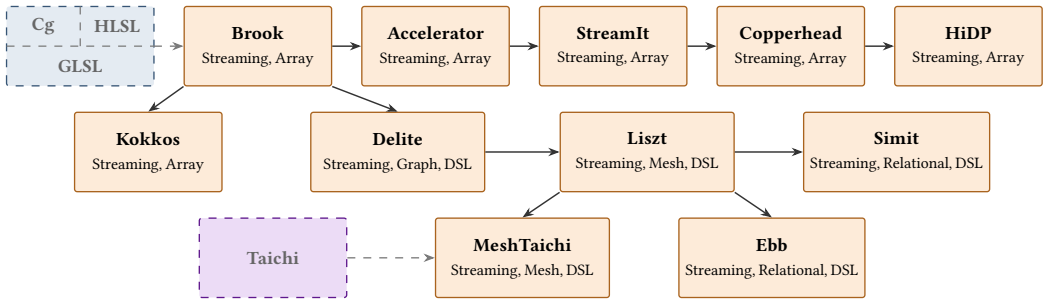


Fig. 19. Descendants of Brook [52]. Dashed boxes and lines indicate connections to or from languages outside this immediate slice of the genealogy in Figure 16.

occurs. For example, the `perlight` annotation on line 2 of Figure 18 indicates that `NdotL` should be computed once for each light source. RTSL's compiler uses these frequencies to decompose and map shaders into compositions of individual stages that could actually execute on a graphics pipeline. The idea behind RTSL's frequencies—that users should specify a program's behavior separately from its mapping to hardware—would prove enormously influential, leading to a later line of research on *user-schedulable languages*, which we discuss in Section 3.7.

While RTSL sat between shade trees and RenderMan on the abstraction-control spectrum, other languages moved further toward image synthesizers to support applications beyond graphics. In contrast to RTSL's whole-pipeline approach, **Cg** [225], **GLSL** [298], and **HLSL** [230] adopted a *shader-per-stage* paradigm, wherein programmers write individual shaders for each programmable stage of the graphics pipeline. The primary benefit of this approach is expressiveness: these are full programming languages that can express computations outside the graphics domain. The trade-off is that certain graphics operations are not easily separable into distinct pipeline stages; for such cross-cutting operations, shaders must either implement redundant functionality across stages or divide the implementation of these operations across the pipeline in ways that do not correspond to natural divisions of their logic [117].

```
1 surface float4 lightmodel_diffuse (float4 ka, float4 kd) {
2     perlight float NdotL = max(0, dot(N,L));
3     return ka * Ca + integrate(kd * NdotL * Cl);
4 }
```

Fig. 18. A light model in RTSL, from Proudfoot et al. [283].

Although specialized to graphics, Cg, GLSL, and HLSL are also *streaming languages*, as their programs consist of transformations over streams of input (e.g., vertices). In addition to the shading languages that we will discuss in Section 3.6, this streaming paradigm inspired two subsequent approaches to GPU programming: Vertigo [106], which we discuss in Section 3.4, and Brook [52], which we discuss next.

3.2 Programming with Parallel Patterns

Brook [52] was one of the first languages explicitly intended for GPGPU programming as opposed to being "misused" for that purpose, as many early shading languages were. Originally designed for general stream programming, it was adapted to GPUs via compilation to Cg shaders, leveraging their conceptual similarities. Brook relies on the insight that common structures in graphics programming can naturally be expressed as functions; for example, an image can be represented as a function mapping coordinates to colors. Whole graphics programs can then be represented as transformations and compositions of these functions. However, Brook does not take this idea as far as the functional languages we discuss in Section 3.4; Brook programs are still imperative in style, resembling an extension of C. Many GPU languages descend from Brook, and we chart their genealogy in Figure 19.

Brook programs are built using three key ideas: *streams*, *reductions*, and *kernels*. Streams are Brook's term for collections of data, similar to arrays in a conventional programming model. Reductions

behave like folds (in the functional programming sense), aggregating a stream into a single scalar result. Kernels are stream transformers that accept any number of input streams and produce any number of outputs, which can be streams or scalars. Crucially, kernels are written on a per-element basis: programmers specify how to compute one element, and Brook implicitly applies this computation pointwise over the entire stream. In Figure 20, the input `a` is a scalar, while `x` and `y` are streams. However, in the body of the kernel, `x` and `y` refer to elements of their respective streams; the kernel will be invoked once per pair of input elements to compute each component of the output. Each Brook kernel compiles to a single shader in Cg.

```
kernel void saxpy (float a, float4 x<>, float4 y<>, out float4 r<>) {
    r = a * x + y;
}
```

Fig. 20. A basic vector scaling and addition in Brook, from Buck et al. [52].

The **Accelerator** [331] language built on Brook but presented a higher-level programming model: programmers specify the behavior of their programs as single functions, as opposed to multiple kernels like in Brook. Accelerator automatically decomposes functions into kernels during compilation, which happens just in time rather than ahead of time, increasing portability. Accelerator's key data type is the parallel array, conceptually equivalent to Brook's streams; in Figure 21, the `array` input is explicitly parallelized on line 3, causing the computations on lines 8 and 14 to be processed element-wise but expressed as bulk operations on a single value.

```
1  static float[,] Blur(float[,] array, float[] kernel) {
2      float[,] result;
3      DFPA parallelArray = new DFPA(array);
4
5      FPA resultX = new FPA(0f, parallelArray.Shape);
6      for (int i = 0; i < kernel.Length; i++) {
7          int[] shiftDir = new int[] { 0, i};
8          resultX += PA.Shift(parallelArray, shiftDir) * kernel[i];
9      }
10
11     FPA resultY = new FPA(0f, parallelArray.Shape);
12     for (int i = 0; i < kernel.Length; i++) {
13         int[] shiftDir = new int[] { i, 0 };
14         resultY += PA.Shift(resultX, shiftDir) * kernel[i];
15     }
16
17     PA.ToArray(resultY, out result);
18     parallelArray.Dispose();
19     return result;
20 }
```

Fig. 21. A 2D matrix convolution in Accelerator, from Tarditi et al. [331].

Following Brook's and Accelerator's success, **StreamIt** [341], a contemporary of Brook also originally designed for other hardware, was later adapted to target GPUs using integer linear programming (ILP) [348]. StreamIt views programs as graphs, with nodes representing stream functions and edges representing data dependencies between them. Constraints—for example, that each function maps to one SM or that functions that depend on each other must execute sequentially—are encoded as ILP inequalities; the system's solution denotes a mapping from the graph to the hardware.

Copperhead [59] simplified the stream model with the aim of increasing productivity; it is embedded in Python [284], and computations are expressed as compositions of classic data-parallel combinators like `map` and `reduce` operating over arrays of input. **HiDP** [379] took a similar approach but exposed the hierarchical, parallel structure of the unified shader model without being specialized to any one architecture. Using annotations, programmers indicate where in the GPU hierarchy their combinators should execute, and HiDP's compiler concretely maps these combinators to hardware, fusing combinators where possible to improve performance.

The **Kokkos** [102, 103] library, meanwhile, adapted the data-parallel model to target GPUs (with a particular focus on CPU-GPU interoperation) and other high-performance hardware. Kokkos is embedded in C++'s template system and has an explicit `parallel_for` construct for launching computations across many threads; it also features a unique separation of *execution spaces*, which specify where code runs, from *memory spaces*, which specify where data is stored. These spaces allow Kokkos to model, for example, code running on a GPU that stores data on the host. Kokkos also influenced the development of polymorphic data layouts, which we discuss in Appendix B.

While languages like Brook and Copperhead presented an element-wise programming model over arrays and streams, others realized that the same approach extended to other structures when

operations over their elements obeyed a fixed pattern. These languages generalized the streaming approach by allowing users to specify the behavior of their programs via actions over *stencils*, local regions of a data structure that expose only an element and its immediate neighbors. Parallel maps over streams can be viewed as a very simple stencil over arrays or lists where the stencil consists only of the element being mapped, while a stencil over a graph would allow users to specify operations per edge and per node. Stencil programs are parallelized over the components of the underlying data structure and draw heavily from *parallel patterns*, or *algorithmic skeletons* [77]—a technique pioneered in the late 1980s to express program behavior without relying on a single implementation. Stencil programs are often highly portable across data structures and architectures.

Since many problem domains are tightly coupled with specific data structures, the parallel-patterns approach became particularly popular for designing DSLs. The **Delite** [63, 323] framework used parallel patterns to allow programmers to implement their own DSLs. Delite is embedded in Scala [384] and has a deferred execution model inherited by every DSL implemented within it; programs are compiled to a dependency graph of operations that the compiler is free to reorder and optimize. Nodes in the graph that do not depend on each other are implicitly parallelized. Users create new DSLs by defining new operations (i.e., nodes), which can be compiled to CUDA or associated with handwritten microkernels as desired.

Several DSLs were implemented in Delite, including OptiML [324] and **Liszt** [62], the latter of which was later expanded into an independent mesh-based DSL for partial differential equation (PDE) solvers [95]. *Meshes* are data structures common in graphics applications that encode shapes as collections of vertices and edges (and faces, for 3D shapes), but they can also be used in other domains; PDEs in particular can be embedded in a mesh, enabling graphics techniques to solve them. Operations on meshes are typically uniform over their components, with data dependencies only between neighboring elements, so kernels have abundant opportunities for parallelism. However, mesh elements cannot be accessed arbitrarily: they must instead be obtained from their neighbors via topological relationships encoded into the mesh's structure, imposing a particular order upon any iterations over it. Liszt's stencil-based design reflects these concerns, exposing no details of its execution order. Operations like the `for`-comprehension in Figure 22 are stenciled over the mesh and implicitly parallelized by Liszt's compiler.

```
1 for (v <- vertices(mesh)) {
2   if (ID(v) == 1)
3     Temperature(v) = 1000.0f
4   else
5     Temperature(v) = 0.0f
6 }
```

Fig. 22. A `for`-comprehension stencil over the vertices of a mesh in Liszt, from DeVito et al. [95].

While Liszt dealt with dense meshes, **MeshTaichi** [374] handled sparse meshes by extending Taichi (discussed in Section 3.7), using its ideas to express kernels over sparse meshes as if they were dense. However, during its evolution from Taichi, many of that language's user-scheduling capabilities were removed, making the result more portable but offering users less control.

In physical simulation, diverse data structures can be useful depending on the level of abstraction at which one wishes to view the simulated system. Reflecting this, the **Simit** [185] simulation DSL used a programming model over graphs for local reasoning. However, it also allows users to switch from graphs to vector- and matrix-based representations for viewing simulations globally. Meanwhile, the contemporaneous **Ebb** [37] DSL used a multilayer model: its lower layer allows programmers to describe arbitrary data structures upon which to base their kernels, encoding the underlying topology of these structures as relations (in the sense of databases) to make definitions extensible and customizable. The upper layer abstracts over the details of these relations, allowing programmers to specify kernels element-wise, in a stencil-like fashion.

3.3 SIMT Languages

The modern GPU landscape owes an enormous amount to Brook—for example, it popularized the now ubiquitous term "kernel" for a GPU program. More tangibly, though, the ideas underlying Brook

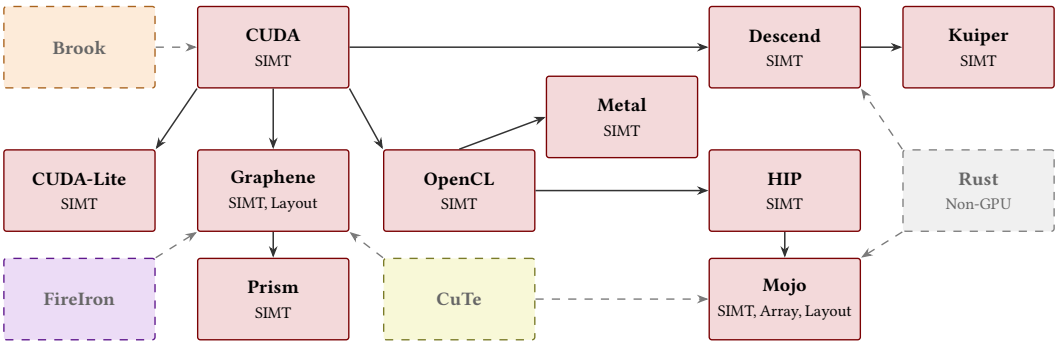


Fig. 23. Descendants of CUDA [239].

led directly to the development of Tesla, CUDA, and the SIMT programming model [51, 238], which extended Brook's model by giving programmers more control over how work is distributed over the compute hierarchy. Instead of implicitly parallelizing kernels pointwise over an input stream, programmers specify exactly how to divide their kernels' workloads among their constituent threads and blocks. We have already discussed CUDA and SIMT at length in Section 2, so we focus here on the languages that adapted or extended CUDA to evolve the SIMT model, shown in Figure 23.

One of the first issues identified with SIMT programming was its complexity, so some early research attempted to simplify the model. **CUDA-Lite** [349], for example, attempted to flatten the memory hierarchy by only exposing programmers to global memory, rewriting kernels to use shared memory at compile time. In practice, however, CUDA-Lite's compiler could not do this transformation automatically, so programmers were expected to insert annotations on memory accesses or control flow describing where in the hierarchy they wanted data to live, thereby guiding the compiler. Figure 24 shows some examples; the annotations on lines 3 and 4 indicate to the compiler that arrays a and b live in global memory, while the annotation on line 14 specifies the iteration bounds of the loop.

```
1  void kernel (float *a, float *b) {
2     __annotation (L"__global__ TPB 1");
3     __annotation (L"garray a 2 4 ASIZE ASIZE");
4     __annotation (L"garray b 2 4 ASIZE ASIZE");
5     int thi = threadIdx.x;
6     int bki = blockIdx.x;
7     float t = (float) thi + bki;
8     int i;
9
10    __annotation (L"BoundChk");
11    if (bki * TPB + thi >= ASIZE)
12       return;
13    for (i = 0; i < ASIZE; i++) {
14       __annotation (L"loop i 0 ASIZE 1");
15       b[(bki*TPB+thi)*ASIZE + i] =
16       a[(bki*TPB+thi)*ASIZE + i] * t;
17    }
18 }
```

Fig. 24. Scaling a matrix in CUDA-Lite, from Ueng et al. [349].

Other languages attempted to grapple with SIMT's complexity by providing users with more guardrails and safety guarantees. **Mojo** [232] took inspiration from Rust's [340] ownership model to rule out memory leaks and use-after-free errors, supporting multiple execution models, including SIMT for targeting GPUs. Mojo also incorporates ideas from user-schedulable languages (discussed in Section 3.7) along with an algebraic layout system (discussed in Appendix B) inspired by CuTe [254]. **Descend** [194] built further on ownership to guarantee full data-race freedom; its key idea is *views*, composable array-access operators that are memory-safe by construction. Descend also extends Rust's borrow checker to account for the GPU hierarchy, capturing how ownership of a resource by a block can result in each of that block's threads owning a different portion of that resource. It further reflects this hierarchy in the form of type-level *execution resources*, through which users assign computations to threads and blocks to ensure nonoverlapping ownership of memory.

Kuiper [227], meanwhile, is a dependently typed SIMT language embedded in F* [328]; it uses separation logic [296] to allow programmers to prove properties about Kuiper kernels within the language itself. Kuiper extends the Pulse [101] separation logic with *located resources*, which describe where in the hierarchy resources live; they can be used in predicates to assert ownership of a resource

by a given thread or block. Other predicates encode notions of visibility for different regions of memory, allowing Kuiper to describe how, for example, global memory is visible to all threads in a grid, while shared memory is visible only to threads in the same block. An opportunity exists to further develop the ideas underlying Descend and Kuiper, as neither has yet formalized its metatheory.

In response to the changes to GPUs we discussed in Section 2.6, some work has attempted to evolve the SIMT model to reflect new hardware. One such attempt was **Graphene** [132], a SIMT IR with three main novel features. The first is a representation format for tensors that allows users to express them as combinations of shapes and element types. Shapes contain a dimension and a stride and describe how tensors are laid out in memory, while the element type describes what is stored at each index; this can itself be another shape, allowing tensors to be defined hierarchically. These shapes are examples of *tensor layouts*, a language feature also seen in tile-based languages; we discuss these in more detail in Appendix B.

Graphene's second feature is *logical thread groups*, which represent active compute resources as run-time objects that can be manipulated and decomposed to model collectively executing hardware like Tensor Cores. Finally, inspired by FireIron [131], Graphene programs contain specifications that describe the required input and output shapes of their data tensors, as well as the configurations of threads required to run them. These language features allow Graphene to express the tensor-level programming model of Tensor Cores; later work on Async Graphene [133] added support for Hopper's and Blackwell's asynchronous hardware features by extending the language with concurrency primitives like futures and async references.

Recently, **Prism** [25] has attempted to improve SIMT's handling of collective operations by introducing *typed perspectives*, which reify the compute and memory hierarchies into the language's type system, allowing users to express the level of those hierarchies at which they are working. Perspectives are similar to Graphene's logical thread groups, but, because they exist statically rather than at run time, they can be enforced by Prism's compiler. In a Prism kernel, each code point has an associated perspective that dictates which operations it can perform: for example, writes to memory must happen from the perspective of a single thread, while a collective operation like a Tensor Core requires a larger perspective—a warp for a `wmma` on Ampere hardware or four warps for a Hopper `wgmma`. In Figure 25, the write to `y_thread` happens at a single-thread perspective, as indicated by the `group` operation preceding it.

```
1 @requires(grid[1], thread[1])
2 def add_m_thread(x: ptr(const(int)) @ grid[1],
3                  y: ptr(int) @ grid[1],
4                  m: int @ grid[1]):
5     tid: int @ thread[1] = id()
6     with partition(x, p=thread[1], offset=tid) as x_thread:
7         with partition(y, p=thread[1], offset=tid) as y_thread:
8             with group(thread[1]):
9                 y_thread[0] = x_thread[0] + m
```

Fig. 25. Adding `m` to a vector in Prism, from Bansal et al. [25].

To keep code perspectives coherent in the face of data-dependent control flow, Prism's perspectives also track how frequently data varies across threads. In Figure 25, the `tid` variable is annotated with a `thread[1]` perspective, indicating that it differs across every thread. On the other hand, the `m` parameter of the kernel has a `grid[1]` perspective, meaning that every thread in the computation agrees on its value. Prism uses principles from the literature on dependency tracking [1] and information flow [93] to manage the interplay between the perspectives of code and of data. Specifically, perspectives form a lattice, with broad perspectives—those representing large groupings of compute resources or data that varies infrequently—at the top and narrow perspectives at the bottom. Data and control are only allowed to flow down the lattice, from broader perspectives to narrower ones. Functions are annotated with the perspectives they require, allowing safe encapsulation and modular composition of collective operations; Prism programs are guaranteed to have the necessary compute resources to invoke these operations safely.

3.4 Functional Array Languages

While Brook envisioned GPUs as stream processors, other approaches leaned further into the idea of graphics programs as transformations of functions, leading to a functional programming model for GPUs. Many languages in this space employed arrays as their primary data structure, borrowing from APL [168] and Single Assignment C (SAC) [125, 301]. The latter's *functional arrays* were particularly significant, combining the efficient access patterns of arrays with high-level, compositional functional programming. The functional array paradigm gave SAC and its descendants a host of safety benefits, including memory safety and data-race freedom, by adding more structure to array accesses and enabling more expressive type systems.

The first GPU language in this vein was **Vertigo** [106], a high-level, domain-specific shading language embedded in Haskell [144]. Vertigo's embedding is shallow, resembling a library more than a language; programs are expressed as combinations of shading primitives, manipulated using typical higher-order functions. Due to its domain specificity, Vertigo itself did not survive the transition to GPGPU programming, but its ideas spawned a new line of research into functional array languages for GPUs, depicted in Figure 26.

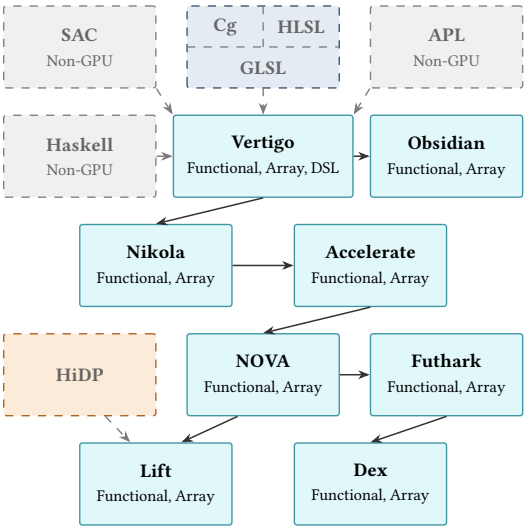


Fig. 26. Functional array languages for GPUs.

Following Vertigo was **Obsidian** [327], another Haskell-embedded language, this time designed for general-purpose computation. Unlike Vertigo and many of the functional languages that would follow, Obsidian exposes the structure of the GPU, giving programmers direct control over its hardware resources; for example, Obsidian users can choose whether to store data to shared memory or global memory. Obsidian programs—despite being embedded in Haskell—are imperative in spirit, making heavy use of Haskell's monadic do notation to emulate stateful operations. The primary benefit of Obsidian is access to Haskell's type system, which has stronger guardrails than CUDA's, but performance is the trade-off for this increased safety: Obsidian kernels are slower than their CUDA counterparts. Subsequent work [326] attempted to bridge the gap by compiling a set of array combinators to efficient CUDA, but not all programs can be expressed using these combinators alone.

```
1 dotp :: Vector Float -> Vector Float -> Acc (Scalar Float)
2 dotp xs ys = let xs' = use xs
3                  ys' = use ys
4              in
5              fold (+) 0 (zipWith (*) xs' ys')
```

Fig. 27. Vector dot product in Accelerate, from Chakravarty et al. [64].

Aiming to be simple and portable, other languages instead chose to obscure the hardware. **Nikola** [224] and **Accelerate** [64] deepened the Haskell embedding, allowing programmers to write kernels that resemble Haskell but execute imperatively like CUDA. Unlike Obsidian, these do not rely on do notation to mimic stateful behavior. The performance of both languages trailed that of CUDA, but the path forward was clear: more compiler optimizations were necessary, particularly automatic kernel fusion.

These optimizations were implemented in **NOVA** [81], which was also the first language in this line of research not embedded in Haskell. While usable as a source language in its own right, it had the dual goal of being a useful IR for compiling DSLs to GPUs; certain design decisions, like the explicit use of fixed-point combinators to implement recursion, reflect this goal. In either use case,

NOVA achieves competitive performance via extensive optimization during compilation to CUDA. Its optimizations include combinator fusion and a pass to automatically flatten and unflatten nested lists; as differently sized sublists can lead to load imbalances when they are allocated to different blocks, this pass improves parallelism by more uniformly distributing work.

Lift [319, 321], meanwhile, aimed to balance performance with portability, combining many of NOVA's ideas with the parallel-patterns-based approach seen in languages like HiDP and libraries like SkelCL [320]. Lift is compiled to OpenCL via a series of rewrite rules, which are proven sound with respect to a dependently typed core calculus. While Lift programs are architecture-agnostic, the rewrite rules compiling them are not; once given a particular hardware target, Lift's compiler selects appropriate rewrites for it, thus achieving portability. Moreover, the set of rewrite rules is extensible in the compiler, allowing Lift to be forward-compatible with new hardware via new rewrite rules. This combination of hardware-agnostic source code with hardware-sensitive rewrites also appears in user-schedulable languages, but Lift is not such a language itself because users cannot directly interact with the rewriting process. Subsequent work on RISE/ELEVATE [134] and Exo [163, 164] evolved Lift's ideas by giving users control over application and extension of rewrites; we discuss these further in Section 3.7.

A particularly influential recent work in the functional lineage is **Futhark** [151], which, like NOVA, uses compiler optimizations like combinator fusion and automatic coalescing to achieve CUDA-competitive performance while maintaining expressiveness. Its key innovation, however, was support for in-place array updates, whose observational purity is enforced by Clean- [27, 28] and Rust-inspired *uniqueness types*. Futhark's type system guarantees several properties beyond the memory safety and data-race freedom common to functional array languages, most notably freedom from run-time shape errors—that is, a guarantee that inputs to array and tensor operations have the right dimensions. Futhark has been a popular substrate for functional programming and type systems research on GPUs since its release; recent work has added efficient array bounds checking [150], rank-polymorphic [160] type inference [300], a higher-order module system [107], and an Agda [333] front end to execute formally verified tensor algebra on GPUs [385]. Other languages like **Dex** [273] have built on Futhark's ideas with dependent value types and a monadic effect system to support automatically differentiable array programming, which is important for deep learning.

```
1 fun main(m : [n][m]f32): ([n][m]f32, [n]f32) =
2   map (λrow : ([m]f32, f32) →
3     let row' = map (λx : f32 → x+1.0) row
4     let s = reduce (+) 0 row
5     in (row',s)) m
```

Fig. 28. Mapping and summing a vector in Futhark, from Henriksen et al. [151].

3.5 Machine Learning DSLs

The functional languages discussed in the previous section were not the only GPU languages to borrow array-programming ideas from APL; **TensorFlow** [2] and **PyTorch** [272], enormously influential deep-learning DSLs that make up a significant fraction of current GPU programming, both feature an array-programming model over tensor primitives. These languages are not functional, however, and unlike most of the languages in this survey, they are used for programming entire ML pipelines, rather than individual kernels. Prior to PyTorch and TensorFlow, these workloads were commonly handled using libraries like NumPy [268], Caffe [172], Theano [36], Chainer [344], and Torch [82].

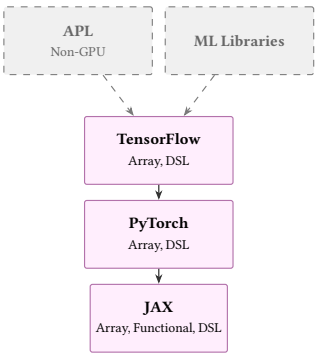


Fig. 29. Machine learning DSLs.

Due to their high-level abstractions, in which programs are expressed as compositions of primitive tensor operations with efficient GPU implementations, PyTorch and TensorFlow have achieved wide adoption among users who are not otherwise GPU experts. The two languages once again

demonstrate a trade-off between flexibility and efficiency: TensorFlow's original design emphasized laying out models as static sequences of operators, offering improved performance via operator fusion at the cost of a more restrictive API. PyTorch, on the other hand, allows programmers to manage their models' control flow dynamically within Python, offering a more permissive programming interface at the cost of making performance optimizations more challenging.

The design of these tensor languages informed more principled DSLs like **JAX** [119], which stripped away PyTorch's and TensorFlow's deep-learning-specific features to reveal a pure, functional, array-based core amenable to whole-program optimizations like auto-differentiation and vectorization. Ultimately, these DSLs and their array-programming model influenced another important family of GPU programming languages: *tile* languages, which we will see in Section 3.8.

3.6 Modern Shading DSLs

Machine learning is hardly the only domain supported by specialized DSLs; despite the general shift toward GPGPU programming, shading DSLs continued to develop, with a particular focus on languages for rendering 3D images, some of which are depicted in Figure 30.

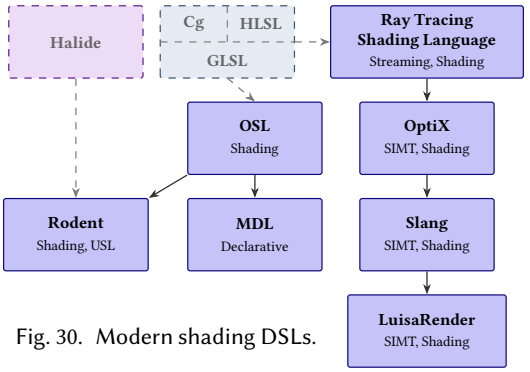


Fig. 30. Modern shading DSLs.

One common technique for rendering is *ray tracing* [361], which colors a scene by simulating the paths of rays between sources of light and an imaginary eye or camera. On its face, ray tracing is a natural fit for the SIMT programming model: each thread independently computes the path of a single ray. In practice, however, the path taken by light is often affected by the materials and surfaces it encounters as it travels, violating the common principle of control-data independence from Theme II; in particular, angled or reflective surfaces can cause rays to take wildly different paths from their neighbors. This makes ray tracers hard to efficiently schedule on GPUs, where adjacent threads want to execute similar code. A common remedy is to partition rays with similar paths into *packets* that execute together; however, this can incur run-time overhead.

Another common ray-tracing problem involves *bounding volumes*—closed regions that encapsulate the union of objects in a set. Bounding volumes are organized into trees in which the leaves encapsulate individual objects. Leaves are grouped with their neighbors in the tree and enclosed within a larger bounding volume; the tree's root ultimately encloses all objects in a scene. Ray tracers must navigate these hierarchies to determine which scene geometry rays intersect. But tree traversal is a highly divergent operation and therefore inefficient on SIMD hardware [10]; many modern GPUs targeted at consumers (rather than datacenters) therefore come equipped with dedicated Ray Tracing Cores to accelerate these workloads [18, 180].

An early ray-tracing language was the **Ray Tracing Shading Language** [271]—also abbreviated RTSL but not to be confused with the Real-Time Shading Language from Section 3.1—which built on GLSL, adding Java-style classes and interfaces. It also has several ray-tracing-specific features, like a dedicated construct to iterate over all the light sources in a scene. RTSL expands upon RenderMan's computation rates, adding a handful of new variable qualifiers, and its compiler targets early SIMD GPUs using an algorithm to automatically group rays into packets. With some run-time overhead, this allows kernels to be written for a single ray and parallelized automatically.

This single-ray model proved successful and was adopted by many subsequent ray-tracing languages. **OptiX** [270], for instance, made the ray-tracing pipeline programmable using a multistage SIMT model. Developers write kernels for each event an individual ray could experience during its

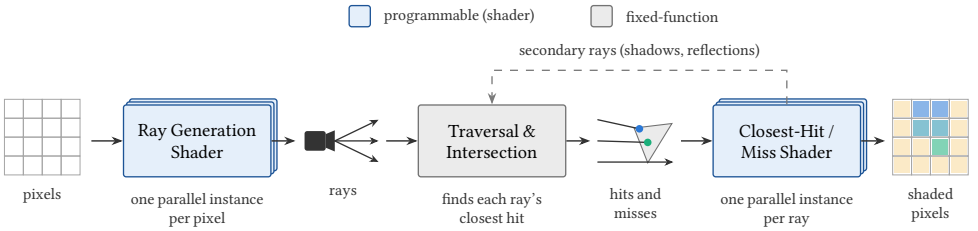


Fig. 31. A (simplified) ray-tracing pipeline, as exposed by languages like OptiX [270] and HLSL [229]. Like the rasterization pipeline in Figure 1, it consists of both programmable stages and hardware-accelerated fixed-function stages.

lifetime, such as encountering an object. OptiX parallelizes these kernels over all the rays and light sources in a scene, invoking the appropriate ones as each ray traverses the scene; this process is depicted in Figure 31. The GPU's structure and hierarchy, along with the process of scene traversal, are opaque to the user; they are instead managed by OptiX's runtime. OptiX is compiled to PTX [260], CUDA's instruction set architecture (ISA), to run on GPUs.

In competition with GLSL, HLSL modernized itself with features like ray-tracing support [229], and a successor DSL called **Slang** [148] added the classes and interfaces that RTSL had brought to GLSL alongside new features like generics, which had not previously appeared in shading languages. Other DSLs, however, diverged from existing approaches by abstracting the details of rendering, thereby simplifying their programming model. Sony's Open Shading Language (**OSL**) [126] implicitly performs ray tracing at run time, but users do not interact with this process directly; the language makes no distinction between light sources and other kinds of surfaces, nor does it expose the structure of the graphics pipeline. Instead, shaders are connected to each other in a graph and evaluated lazily as data is demanded from them by downstream nodes. OSL's simple interface has made it the industry standard for visual effects (VFX) and animation tasks.

Unlike other shaders whose final output is usually a set of colors describing the scene as viewed from a particular angle, OSL shaders compute *radiance closures*, which encode details of scene elements as well as symbolic descriptions of how those elements interact with light; these closures can then be evaluated from various angles to render images. This idea was adopted by other languages for physically based rendering, including the Material Definition Language (**MDL**) [178], a DSL specifically for creating closure-like structures that describe how different materials interact with light. MDL programs are pure and declarative so as to be more easily parallelized on GPU hardware.

Some rendering DSLs aimed for portability by obscuring hardware details. **LuisaRender** [382], for example, presents a hardware-agnostic SIMT model targeting a variety of GPUs, while still taking advantage of bespoke hardware like Ray Tracing Cores via its JIT compiler. **Rodent** [276] uses partial-evaluation techniques along with ideas from user-schedulable languages like Halide [288, 289] (discussed in the next section) to give users a declarative programming model for their renderers. Users specify a scene to be rendered, abstracting the details of which rendering procedures are used. Then, Rodent applies the first Futamura projection [120], partially evaluating its interpreter on the provided scene to yield a specialized renderer for that scene; to execute the renderer on a given hardware target, one need only supply implementations of each rendering procedure required by the scene. Zheng et al. [382] note that, while elegant, this compilation process is lengthy.

While these shading languages make up a sizable fraction of modern GPU programs, they have received less attention from the research community than their general-purpose cousins; the formal-methods tools we discuss in Section 4, for example, mostly do not target shading kernels. Whether shading DSLs might benefit from some of the language and type-system features we discuss elsewhere in this survey is a question for future work.

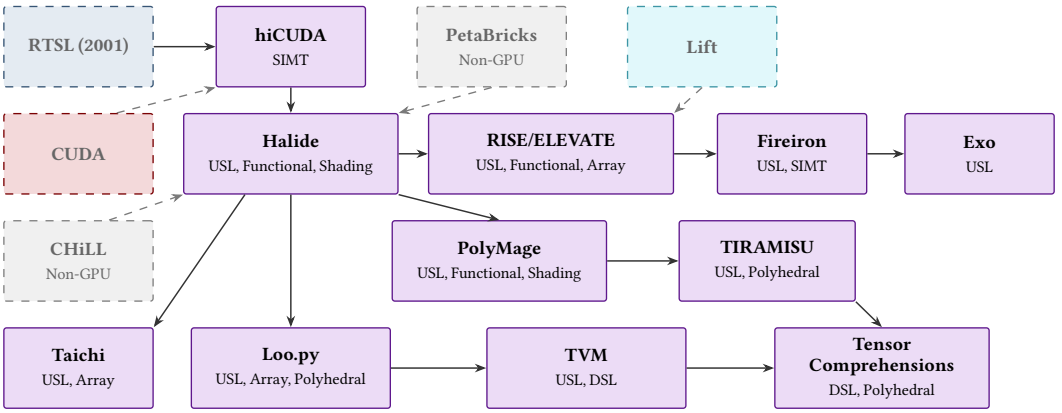


Fig. 32. Descendants of Halide [288, 289].

3.7 User-Schedulable Languages

High-level languages rely on extensive optimization to achieve competitive performance. Their compilers can match or even outperform hand-optimized low-level kernels in the best case, but, as we discussed in Theme II, the complexity of the GPU optimization space makes this approach brittle; when programmers deviate too far from the problems these high-level languages are designed to solve, performance frequently suffers. Expert performance engineers are therefore often skeptical of optimizing compilers. The key idea of *user-schedulable languages* (USLs), whose genealogy appears in Figure 32, is to trust that these experts are more capable of optimizing their code than a compiler, while also separating the optimization process from program behavior.

The idea that users should have some control over optimization is old: it appears in RenderMan's computation rates and the Real-Time Shading Language's computation frequencies, which function as hints to the compiler about how values change within a program. The ***hi*CUDA** [137] language—a high-level, sequential variant of CUDA—built upon these annotations with *directives*, which allow users to specify how to map code to hardware. These come in two forms: computation-model directives and data-model directives. The former specify which code should be executed on the GPU and how to parallelize it: a `loop_partition` directive, for example, informs the compiler how to distribute a sequential loop over the threads and blocks of the grid. Directives for the data model, on the other hand, dictate the movement of data between levels of the memory hierarchy.

The **PetaBricks** [17, 279] framework, meanwhile, gave programmers control over *algorithmic choice*—the ability to specify multiple versions of their kernels and the conditions under which each version should be used. A simple motivating example for this idea is sorting an array; for small inputs, a quadratic algorithm like insertion sort outperforms merge sort despite the latter's superior asymptotic complexity. Accordingly, many languages' standard libraries use quadratic sorts for inputs below a certain hard-coded size. PetaBricks, however, argues that this size is hardware dependent and no single choice will perform well on every machine. Instead, the right size should be identified at compile time; PetaBricks extends this idea beyond list sorting by allowing users to specify general conditions—such as specific sizes, dimensions, or types of input data—under which different versions of their programs should be executed, filling in concrete values via autotuning.

Ultimately, like computation rates and frequencies before them, PetaBricks's algorithmic choices and *hi*CUDA's directives are still just guidance to the compiler; they do not give users direct control over optimization. One of the first fully user-controlled optimizers appeared outside the GPU space in the form of **CHiLL** [66, 299], a compilation framework for sequential loop programs in C that allows programmers to directly control how their code is optimized and compiled via *transformation*

recipes. These recipes are user-written scripts—an example appears in Figure 33—comprising many fine-grained transformation primitives for operations like tiling, loop unrolling, and data movement. Developers compile their programs by executing these scripts.[3] This approach is in some ways the inverse of PetaBricks's: there, users specify multiple algorithms and rely on the compiler to choose one based on the hardware, whereas CHiLL asks users to write one algorithm and then specify how to map it to hardware. There is much more to be said about CHiLL and its origins, but we lack space to cover the full history of USLs—interested readers are directed to Hall et al. [136]'s comprehensive survey; we focus here on their application to GPUs.

USLs achieved maturity with **Halide** [288, 289], a graphics DSL that has seen extensive industry adoption [5] and coined much of the terminology used in the literature. In particular, it codified the distinction between the *algorithm* (i.e., what a kernel is doing, sometimes called the "application logic") and the *schedule* (i.e., how the algorithm is performed and in particular how work is assigned to hardware resources). Halide's algorithms are pure functions from inputs to outputs; purity is what makes it safe for scheduling operations like tiling to reorder parallel loops [136]. Schedules, as in CHiLL, are sequences of built-in operators like `tile` and `vectorize`.

```
permute ([3, 1, 2])
tile (0, 2, TJ)
tile (0, 2, TI)
tile (0, 5, TK)
datacopy (0, 3, 2, [1])
datacopy (0, 4, 3)
unroll (0, 4, UI)
unroll (0, 5, UJ)
```

Fig. 33. A transformation recipe in CHiLL, from Hall et al. [136].

Halide also advanced the USL paradigm by unifying the languages for the algorithm and the schedule. Unlike in CHiLL, where transformation recipes are separate from the C code they compile, Halide's schedules and algorithms live in the same file, allowing scheduling operators to directly inspect and modify algorithm data. Figure 34 depicts a simple Halide kernel; the algorithm defines two functions, `blur_x` and `blur_y`, while the schedule tiles, parallelizes, and vectorizes these functions. As in Brook, images are functions from coordinates to data, and Halide algorithms do not specify the bounds of the images they operate over. Instead, Halide's compiler infers these by symbolically evaluating indices into functions and arrays to determine the intervals over which they apply. Later work on **TIRAMISU** [24] argues this use of bounds is a limitation of Halide; in particular, Halide requires rectangular iteration spaces—a result of its focus on graphics. TIRAMISU is similar to Halide in both design and purpose, but it uses a *polyhedral* [166, 199, 214] representation for its optimizations (we discuss polyhedral compilation further in Appendix A), which has different trade-offs. In particular, the polyhedral model supports more complex iteration spaces, and it can identify more opportunities for loop fusion than Halide's bounds inference. However, it requires loop bounds to be linear functions of static values or surrounding loop variables; symbolic bounds inference, meanwhile, supports more complex, nonlinear loop bounds and symbolically sized tiling transformations, which the polyhedral model cannot express [289].

```
Func blur_3x3(Func input) {
    Func blur_x, blur_y;
    Var x, y, xi, yi;

    // The algorithm - no storage or order
    blur_x(x, y) = (input(x-1, y) + input(x, y) + input(x+1, y))/3;
    blur_y(x, y) = (blur_x(x, y-1) + blur_x(x, y) + blur_x(x, y+1))/3;

    // The schedule - defines order, locality; implies storage
    blur_y.tile(x, y, xi, yi, 256, 32)
          .vectorize(xi, 8).parallel(y);
    blur_x.compute_at(blur_y, x).vectorize(x, 8);

    return blur_y;
}
```

Fig. 34. A three-by-three image blur in Halide, from The Halide Team [336].

Halide conceived of schedules as inhabiting a 2D space: programs are loop nests over the dimensions of images, and transformations on those images occur at a particular granularity, similar to the Real-Time Shading Language's computation frequencies. The deeper into the nest an operation

[3] A note on terminology: a *user-schedulable language*, or USL, is one whose optimization (or schedule) can be controlled by the program author, while a *scheduling language* is a DSL for writing schedules for a USL. In the case of CHiLL, transformation recipes are the scheduling language and make C user-schedulable.

appears, the finer its *compute granularity*; if the operation's result is stored in memory, the level of the nest where that occurs is its *storage granularity*. The space of schedules plots these two granularities against each other, with the constraint that an operation's compute granularity must be finer than its storage granularity—every computed value must have a location into which it can be stored. Each operation (or call) in an algorithm is given a call schedule along these two dimensions that places it within the loop nest. Finer compute granularity results in higher locality (data needs to persist for less time before it is used), but it can also cause more recomputation. Coarser storage granularity, meanwhile, increases reuse but may introduce sequential dependencies, thereby reducing parallelism.

A wide body of work builds directly on Halide: formalizing its semantics [294], validating the correctness of its scheduling transformations [75], automatically deriving efficient schedules via autotuning [234, 310, 311], and using machine learning to improve that process [4]. Other work has used Halide in domains beyond graphics, like tensor algebra [378] and machine learning [210], or extended its ideas to different programming models. For example, **Loo.py** [186] uses an imperative approach embedded in Python.

Many DSLs have built on Halide's ideas, leveraging the doctrine of *efficiency from specificity* [136] to simplify the scheduling search space and identify new optimizations. The **PolyMage** [170, 235] graphics DSL, for instance, employs a handful of domain-specific schedules, most notably *overlapped tiling*. Overlapped tiling differs from usual tiling strategies because overlapped tiles share some of their data with adjacent ones—useful for certain graphics operations wherein pixels require data from the region surrounding them. However, when an overlap is too coarse, there may be a large synchronization overhead between threads; when an overlap is too fine, some computation may be redundant. PolyMage's approach is to perform overlapped tiling at the level of warps, rather than threads or blocks, storing part of the tile in the warp's registers instead of shared memory; it then employs *warp shuffling* [85] to exchange data between threads within the warp without going through shared memory, improving efficiency.

TVM [69], meanwhile, brought user scheduling to deep learning. It comprises a high-level DSL for tensor operations (similar to Loo.py's front end) and an optimizing compiler that uses learning-based search to identify optimal schedules on various types of hardware. TVM implements schedules beyond those from Halide, such as automatic tensorization and explicit latency hiding, which are specialized to deep-learning workloads and which allow TVM to target new architectures like TPUs [129]. Other compilers for TVM target specific use cases, such as Cortex [112] for recursive models. IRs like Relay [297], Relax [195], and TensorIR [113], meanwhile, sit higher up the stack, presenting a common target for ML frameworks that wish to leverage TVM's ecosystem.

Even higher level than TVM, **Tensor Comprehensions** (TC) [353] was designed to present the most abstract possible model for tensor programming. Kernels are specified mathematically; the entire TC definition of SGEMM, for example, is given in four lines in Figure 35. Despite its abstract programming model, TC achieves competitive performance by leveraging its extreme domain specificity: it is lowered first to Halide's IR, then to a polyhedral representation, autotuning its parameters all the way down. The polyhedral optimization space is well understood, and TC exploits this to generate highly optimized kernels.

```
1 def sgemm(float a, float b, float(N, M) A, float(M, K) B) → (C) {
2   C(i, j) = b * C(i, j)
3   C(i, j) += a * A(i, k) * B(k, j)
4 }
```

Fig. 35. SGEMM using Tensor Comprehensions, from Vasilache et al. [353].

USLs have also proved useful in the domain of sparse tensor algebra. Sparse tensors require special storage formats to avoid storing vast quantities of trivial data; in general, efficient access is only possible along dimensions in which tensors are dense. Algorithms over tensors therefore differ depending on their dense and sparse dimensions, and efficient tensor-algebra libraries suffer from a combinatorial explosion of implementations—one for each combination of sparse and dense inputs.

One influential solution to this problem was pioneered by the Tensor Algebra Compiler, or Taco [184], an optimizing compiler that originally featured no user-scheduling capabilities. Taco automatically generates implementations of a given tensor operation for inputs of various sparsities; users specify which computations to apply to which tensors through a library API, along with a storage format that describes the dimensions and sparsity characteristics of each input. Taco then synthesizes an *iteration graph* that describes how to efficiently co-iterate over the inputs.

To offer more control over performance, over time Taco introduced user-scheduling elements [183], in particular allowing users to explicitly reorder loops or materialize intermediate results in temporary workspaces without changing the meaning of computations. These schedules are captured in an IR called *concrete index notation*, which sits between Taco's original declarative notation and its final compilation target. Taco remains an active line of work, having been extended with further scheduling features to target both individual GPUs [73, 304] and multi-GPU systems [368].

Taichi [159] took the opposite trajectory to Taco, beginning its life as a USL but ultimately transitioning away from that style. Initially, Taichi took a user-scheduling view of sparse tensor programming, separating data from its actual arrangement in memory; concretely, its programming model allowed developers to program against sparse data structures as though they were dense. Taichi accomplished this by including a mini-language for describing data structures as compositions of sparse and dense components; based on these definitions and guided by a set of user-provided schedules, Taichi's compiler transformed iterations over logically dense, user-facing data structures into iterations over their underlying sparse representations in memory. However, as choosing a schedule often involves specializing code to specific hardware, Taichi ultimately removed most of its user-scheduling features in the pursuit of portability [330].

The USLs we have discussed so far have been broadly successful, but they do have some drawbacks. By giving users precise control over compilation, USLs dramatically raise the complexity of their programming model and present a steep learning curve for nonexpert users. Additionally, they are not extensible; users cannot add to or modify the set of scheduling operations available to them. This limitation has two dimensions: first, users cannot implement optimizations outside what the built-in operators allow [134], and second, users are dependent on the compiler to update the implementation of hardware-specific operators when new microarchitectures are released [163]. The latter problem is far more pressing on GPUs than CPUs due to the fast and visible cadence of hardware evolution discussed in Theme III; some work exists on updating USLs to target Tensor Cores, such as **FireIron** [131], but these approaches only address a specific instance of a more general problem.

Work addressing the first of these dimensions made the set of scheduling operators user-extensible by reconceptualizing them as term rewrites. This approach, which drew from earlier work on Lift [319] (discussed in Section 3.4), was first realized by a pair of programming languages: **RISE** and **ELEVATE** [134]. The algorithm language, RISE, is a functional array language, while ELEVATE is based on *strategy languages* [355]—a kind of language for specifying rules for term-rewriting systems, which allow ELEVATE to be fully programmable. Strategies encode particular ways of rewriting a program and can be composed using combinators. These combinators range in complexity: the `;` in Figure 36 performs simple sequencing, whereas the `match` in Figure 37 conditionally applies one strategy based on the result of another.

```
val dot = fun(as,
          fun(bs,
            zip(as)(bs) |>
            map(fun(ab, mult(fst(ab))(snd(ab)))) |>
            reduce(add)(0)))
val mm = fun(a : M.K.float,
          fun(b : K.N.float,
            a |> map(fun(arow,              // iterating over M
               transpose(b) |> map(fun(bcol, // iterating over N
                  dot(arow)(bcol)))))))     // iterating over K
```

```
val tiledMM = (tile(32,32) `@` outermost(mapNest(2)) `;` lowerToC)(mm)
```

Fig. 36. Matrix multiplication in RISE/ELEVATE, from Hagedorn et al. [134]. The top box is the algorithm in RISE, while the bottom is the optimization strategy in ELEVATE.

One of the main risks of allowing users to specify their own rewrites is that their optimizations may not be sound—that is, they may not actually preserve the semantics of the original program. RISE's semantics are formally modeled in Agda, and ELEVATE's built-in rewrites and strategy combinators are proven correct against those semantics. However, this does not guarantee the correctness of any new rewrites a user might implement. The ATL [217, 218] language addressed this issue by embedding rewrites as theorems in the Rocq [332] proof assistant, guaranteeing their correctness by construction, but it has not yet been extended to target GPUs. Shoggoth [286], meanwhile, modeled the semantics of strategies in Isabelle [240] and formulated a calculus for proving properties of rewrites beyond correctness, such as termination or composability.

Strategic rewriting has a few limitations, however. For one, it scales poorly to large search spaces; later research sought to replace it in RISE's compiler with an approach based on *equality saturation* [188]. Additionally, it does not address the latter of the extensibility challenges mentioned earlier: ELEVATE's rewrites are source-to-source, and the process of lowering optimized programs to executables is still opaque to the user. Exposing this process to the programmer is the key idea of *exocompilation*. Introduced by the **Exo** [163, 164] language, exocompilation identifies the boundary that every compiler implicitly draws between what the compiler does on its own ("automation") and what users can affect directly ("control"), shifting hardware-specific code generation to the user side of the boundary. In languages like Halide, hardware-specific compilation or optimization steps like register allocation and instruction selection are performed automatically by the compiler; Ikarashi et al. [164] argue that this decision is why Halide has struggled to keep up with new GPU hardware features like Tensor Cores. In Exo, by contrast, hardware-specific compilation steps can be specified as user-extensible libraries, allowing performance engineers to target whatever hardware they desire without needing to modify the compiler. As in ELEVATE, Exo's schedules are expressed as compositions of rewrite rules; Exo also features an effect system and formal semantics to argue the correctness of its rewrite system, although its metatheory is not yet formalized. Exo has seen some industry adoption [164], and it has been used to generate portable GEMM kernels that outperform hand-optimized ones on various accelerators [57, 58]. Recent work has extended its ideas to GPUs, with a focus on Tensor Cores [11].

```
def tileND(n: List[Int]): Strategy[Rise] = DFNF ';' (n.size match {
  case 1 => function(split(n.head)) // loop-blocking
  case i => fmap(tileND(d-1)(n.tail)) ';' // recurse
            function(split(n.head))   ';' // loop-blocking
            interchange(i) })             // loop-reorder
```

Fig. 37. A conditional strategy definition in ELEVATE, from Hagedorn et al. [134].

3.8 Tile-Based Languages

As we discussed in Section 2.6, the advent of Tensor Cores gave rise to a new programming model for GPUs—one in which tiles, rather than scalars, were the new fundamental unit of computation. On first release, Tensor Cores were only fully programmable in PTX [260] bytecode; CUDA had little support for the new hardware. Programmers were limited to libraries like cuDNN [253] or cuBLAS [252], if those libraries supported their use case, or to the unstable `wmma` API [20, 226] otherwise. This situation was improved by **CUTLASS** [334], a library for recursive decomposition of linear-algebra computations along the GPU's block-warp-thread hierarchy [176]; a GEMM, for example, can be

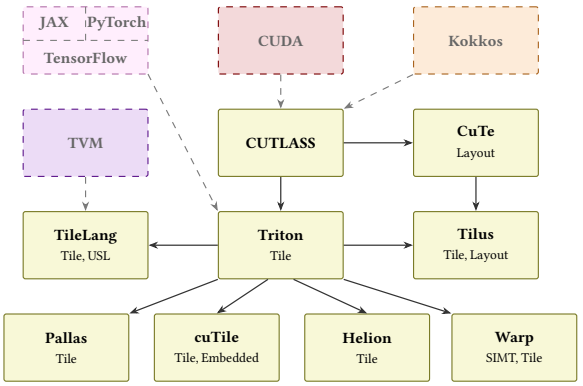


Fig. 38. Descendants of CUTLASS [334] and Triton [343].

recursively decomposed into multiplication of block tiles, which can then be further decomposed into multiplication of warp tiles, and ultimately into thread tiles, each of which is accumulated into a result. CUTLASS is embedded in C++'s template system and automatically generates kernels from *policies*, wherein users specify tensor dimensions and tiling strategies via templates. While originally designed for more than just Tensor Cores, it proved a popular medium for programming them: CUTLASS's warp-tile-level GEMM can be implemented with Tensor Cores, with other parts of the API abstracting the process of massaging input data into their expected format.

However, while CUTLASS was a marked improvement over `wmma`, it lacked the structure and composability of a full-fledged programming language. **Triton** [343] soon filled this gap, offering the first language-level support for tile-based programming and reifying the previously implicit structure of tiles into the language. Instead of performing scalar operations per thread, Triton programs are conceptually single-threaded, require no manual synchronization, and undertake computations one tile at a time, operating at the granularity of an entire block; this programming model is an abstraction of SIMT and is sometimes referred to as Single Instruction, Multiple Block (SIMB) [128]. Programmers can focus on the application logic of their kernels, leaving concerns like recursive decomposition and Tensor Core-input swizzling to the compiler, while still achieving performance competitive with hand-optimized CUDA on common tensor workloads like matrix multiplication. Triton's type system statically tracks the shapes of tiles, enforcing that inputs to tile-level operations like tensor addition or multiplication have the correct dimensions; when tile sizes are incompatible, they are automatically padded or replicated until they have the right shape, similar to broadcast semantics in languages like PyTorch and TensorFlow. In the same way that SIMT is an effective model for programming CUDA Cores, the tile-based model envisioned by Triton and its descendants (depicted in Figure 38) is a natural fit for the synchronous Tensor Cores of Volta and Ampere.

In Figure 39, the `x` and `y` variables are initialized with a (one-dimensional, in this case) tile of data, based on the `offsets` computed from the size of the block. The sum of these tiles is stored in `output`, with addition performed at tile granularity. The `+` operation requires the types of its inputs to be compatible; if either `x` or `y` were longer than the other, the shorter tile would be reshaped to match.

Triton developed the tile-based programming paradigm, but it did so in a domain-specific way: its ecosystem and optimization strategies are primarily oriented toward deep-learning workloads. Other work—such as **Warp** [222]—aimed to bring tile-based programming to other areas by adapting Triton's ideas to general differentiable computing. Warp was initially designed with a SIMT programming model but added tile-based programming support early in its lifespan [223].

```
def add(x_ptr, y_ptr, output_ptr, BLK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)

    block_start = pid * BLK_SIZE
    offsets = block_start + tl.arange(0, BLK_SIZE)

    x = tl.load(x_ptr + offsets)
    y = tl.load(y_ptr + offsets)

    output = x + y

    tl.store(output_ptr + offsets, output)
```

Fig. 39. Vector addition in Triton, from Gupta [130].

Some languages built on Triton's foundation while further abstracting its programming model. **Helion** [285], for example, sought to be "higher-level Triton" [228] by obscuring details like kernel-launch dimensions, instead using autotuning to search for the best values for these factors at compile time. **TileLang** [356] incorporated ideas from TVM, asking developers to focus solely on their kernel's data flow and leaving scheduling to the compiler. TileLang programs are built from tile-level operators that encapsulate common patterns like data copying or matrix multiplication, as well as a small set of scheduling operators and annotations for use when the compiler's automatic optimizations fall short. The **cuTile** [105, 246] family of embedded

```
fn add<const B: i32>(
    z: &mut Tensor<f32, {[B]}>, // exclusive write
    x: &Tensor<f32, {[-1]}>, // shared read
    y: &Tensor<f32, {[-1]}>, // shared read
) {
    let tx = load_tile_like(x, z);
    let ty = load_tile_like(y, z);
    z.store(tx + ty);
}
```

Fig. 40. Tensor addition in cuTile Rust, from Elibol et al. [105].

languages, meanwhile, allows programmers to write Triton-like kernels in familiar languages like Python and Rust, compiling them to a common IR called TileIR [248].

Other languages went the opposite direction, lowering Triton's abstraction level to squeeze more performance out of the hardware. **Pallas** [337], for example, requires explicit use of Tensor Cores and other hardware features but adds source-level metaprogramming and the ability to target TPUs [129]—another kind of accelerator for tensor operations with similar (but, crucially, not identical) kernels. **CuTeDSL** [254], meanwhile, began as an extension of the CUTLASS library with algebraic layouts [61, 255]—discussed in Appendix B—but evolved into a full-fledged, Python-embedded language. Layouts complement the tile-based model by allowing programmers finer control over how tiles are arranged in memory; **Tilus** [97], a language for low-precision computation [118], uses them to expose more details of the GPU hierarchy and grant more control over data movement. CUDA libraries such as ThunderKittens [316] also emerged to support tile-based patterns while allowing programmers to fall back on standard CUDA as needed.

However, while tile-based programming is an elegant abstraction for synchronous, warp-level Tensor Cores, it is not clear that it is the right model for programming asynchronous hardware units like TMAs and the warp-specialized pipelines they encourage. Due to their single-threaded programming model, languages like Triton and cuTile must rely on their compilers to determine how to specialize warps and when to use Tensor Cores or the TMA [295]; the heuristic methods they employ do not always make the optimal choice. Hopper and Blackwell benchmarks [367] indicate that Triton can underperform handwritten kernels—including on some of its original benchmarks, like attention—while cuTile's performance varies wildly across machines, even within the same microarchitecture generation. Languages like Pallas and extensions to Triton like Gluon [346] and TLX [128] have attempted to address this problem by giving programmers more control, allowing them to program at the granularity of a warp or warpgroup rather than a whole block, manually specialize warps, and explicitly specify when to use Tensor Cores and TMAs using inline IR. However, giving users this much control sacrifices much of what makes Triton an elegant abstraction in the first place: these languages cannot be conceptually single-threaded because they must handle resources like TMEM and cluster-level Tensor Cores that are shared across warpgroups and blocks. Thus, programmers are forced to grapple with the full complexity of communication on asynchronous hardware, making it easy to write buggy code.

```
def tlx_kernel(x_ptr, y_ptr, n_elements, BLOCK: tl.constexpr):
    ...
    with tlx.async_tasks():
      with tlx.async_task("default"):
        while tile_id != -1:
          tlx.barrier_wait(empty[0], prod_phase)
          x = tl.load(x_ptr+...)
          tlx.local_store(smem[0], x)
          tlx.barrier_arrive(full[0])
          tlx.clc_producer(...)
          tile_id = tlx.clc_consumer(...)
      with tlx.async_task(num_warps=1):
        while tile_id != -1:
          tlx.barrier_wait(full[0], cons_phase)
          x_local = tlx.local_load(smem[0])
          y = x_local * 2.0 + cta_rank
          tl.store(y_ptr+..., y)
          tlx.barrier_arrive(empty[0])
          tile_id = tlx.clc_consumer(...)
```

Fig. 41. Warp specialization in Triton using TLX, from Guan et al. [128].

Other work has attempted to preserve the simplicity of Triton's model by enhancing its compiler. In particular, Tawa [68] attempts to automatically warp-specialize Triton kernels, allowing them to use Hopper's TMAs, which Triton had previously struggled to wield effectively. Tawa's core idea is the asynchronous reference, or `aref`, a communication primitive that allows warps to have a formal producer-consumer relationship. One warp can publish values to an `aref`, which has operations to get, set, and consume its value, while others can asynchronously wait for data to arrive. The `aref`'s semantics enforce the producer-consumer relationship between these warps: it cannot be read while empty or written while full, and each warp possesses a *credit* (effectively a capability) allowing producers to write to the `aref` while empty and consumers to read from it while full.

While Tawa focused on the problem of applying a given warp-specialization strategy to a Triton kernel, the Twill [313] framework tackled the related question of deriving the optimal such strategy,

arguing that heuristic methods of warp specialization are not portable across GPU generations. Twill is based on the idea of software pipelining [196], which uses software to mimic the way that CPU hardware pipelines dynamically exploit instruction-level parallelism. Specifically, Twill statically reorders instructions to overlap independent computation, maximizing utilization of hardware resources. While prior compilers performed warp specialization and software pipelining independently, Twill argues that these two processes are intrinsically linked and must be considered together to derive optimal schedules for either. The usual algorithm for identifying optimal software pipelines is called *modulo scheduling* [290, 291]; by parameterizing this algorithm over certain hardware details, Twill can derive provably optimal warp-specialized schedules for a variety of deep-learning kernels, including FlashAttention on Hopper [305] and Blackwell [375].

However, Chen et al. [68] note that, while tools like Tawa achieve competitive performance, the ergonomics of warp specialization remain challenging, clashing with fundamental assumptions about SIMT execution baked into both GPU hardware and its software ecosystem. Once again, hardware evolution has broken the existing programming model; whether updated abstractions can be found that maintain Triton's ergonomics in the asynchronous setting without surrendering performance is an ongoing area of research.

3.9 Task-Based Languages

While some attempted to adapt the tile-based model to asynchronous GPU hardware, others abandoned it in favor of a new *task-based* programming model. Task-based programming has its origins in high-performance computing (HPC) languages—namely, Sequoia [110] and Legion [34]—for large-scale multiprocessor machines. These languages have since been extended to target GPUs, but their original targets were more heterogeneous; in the task-based model, programmers specify small "tasks" or "micro-operations" that each describe part of a larger computation, then assign these tasks to the various components of their distributed or heterogeneous hardware. Yadav et al. [369] assert that this is the natural way to manage the complexity introduced by the various fixed-function components of modern GPUs, like Tensor Cores or TMAs, and scales as chips continue to evolve—Blackwell, for example, has more specialized subcomponents than Hopper, which has more than Ampere.

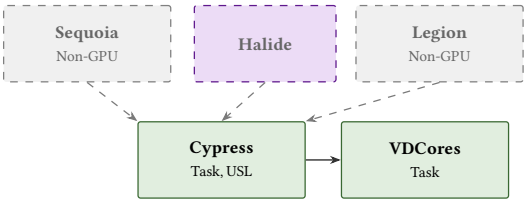


Fig. 42. Task-based languages for GPU programming.

```
1  @task ("gemm", Inner, reads=["A","B"], writes=[ "C" ])
2  def gemm_block (C: tensor[2, f16],
3                  A: tensor[2, f16],
4                  B: tensor[2, f16]) :
5    W = tunable(int)
6    M , N , K = C.shape[0], C.shape[1],  A.shape[1]
7    # Break A and B into tiles of width W.
8    Ap = partition_by_blocks(A, (M, W))
9    Bp = partition_by_blocks(B, (W, N))
10   Cacc : tensor[2, f16] = make_tensor(M, N)
11   launch("clear" ,Cacc)
12   # Launch a sequential group of tasks over each tile.
13   for k in srange (cdiv (K, W)):
14     launch("gemm", Cacc, Ap [0, k], Bp [k, 0])
15   launch("copy", Cacc, C)
16
17 # Mapping Specification
18 TaskMapping (
19   instance="gemm_block",
20   variant="gemm_block",
21   proc=BLOCK,
22   mems=[GLOBAL, GLOBAL, GLOBAL],
23   tunables={"W": 64},
24   calls=[ "clear_tile", "gemm_tile", "copy_tile"],
25   warpspecialize=True,
26   pipeline=3
27 )
```

Fig. 43. Block-level GEMM in Cypress with its associated mapping, from Yadav et al. [369]. The `launch` keyword invokes a new task; the version invoked depends on the mapping specification. The `gemm` on line 14, for example, is mapped to the tile-level `gemm_tile` task on line 24.

An early work in this area, depicted in Figure 42, is **Cypress** [369], which attempted to unify task-based programming with user scheduling. Cypress programs comprise a *logical specification* and a *mapping specification*—essentially an algorithm-schedule distinction, except that Cypress's mapping specifications assign tasks to different groups of threads. The logical specification of a GEMM, for example, is initially mapped to the entire grid; this mapping is then decomposed into

subtasks for blocks, warpgroups, warps, and eventually threads. Cypress's logical specifications elide the tricky details of synchronization and inter-thread communication, allowing the programmer to focus purely on their kernel's logic; these details instead appear in the mapping specification, which the compiler uses to generate code and coordinate data movement to maximize use of GPU resources. Each level of the hierarchy has an associated mapping with its own properties; all mappings can specify how to move data between hierarchy levels as tasks are decomposed, for instance, but only the block-level mapping can specify if and how to perform warp specialization.

Meanwhile, Virtual Decoupled Cores, or **VDCores** [149], is a task-based IR that models the execution units of the GPU as independent, isolated cores; interactions between cores are expressed through data dependencies between their tasks. For example, CUDA Cores and Tensor Cores are coupled together into an Async Compute Unit and modeled as one core, while the TMA is an Async Memory Unit modeled as a different core. Cores are abstract and extensible, allowing the system to easily adapt to further evolution of GPU design. VDCores programs are dependency graphs of small operations called *μops*. These μops are assigned to cores dynamically by the runtime; they can be reordered to hide latency as long as their dependencies allow it. It is not yet clear whether this new line of research into task-based languages will be as effective for asynchronous GPUs as the tile-based model is for synchronous Cores, but it seems a promising avenue for future work.

4 A Taxonomy of GPU Formal Methods

Having discussed their alternatives at length, we now return focus to CUDA and OpenCL. These languages' extensive ecosystems and support from chip manufacturers mean they are still widely used, both directly by programmers and as compilation targets for many of the other languages we have discussed. However, their low-level programming model and basis in C++ mean they lack many of the features and guardrails of modern languages, and so it falls to formal methods to fill the gap. In this section, we discuss Table 2: a taxonomy of existing GPU formal-methods tools across various dimensions, including methodology, languages and issues targeted, purpose, and limitations.

4.1 Preliminaries: GPU Semantics

Before diving into the taxonomy, it is worth discussing a question faced by all the tools discussed in this section: how do GPUs actually behave? Any tool wishing to analyze a program must have a model of how that program will execute, but manufacturer-provided specifications of languages like CUDA and OpenCL, as well as ISAs like PTX [260] and SPIR-V [338] (OpenCL's IR), are often unclear or even contradictory. Hardware details like floating-point rounding modes [124, 212] and memory-consistency models [12, 314, 345] are likewise frequently underspecified. A wide body of research exists on empirically evaluating and formalizing the behavior of GPUs [12, 212, 314, 315, 345], but such work necessarily lags behind the cadence of hardware releases.

In the absence of a formal specification of GPU behavior, program analyses must choose an approximate model; the two most common models are *interleaving semantics* and *lockstep semantics*. In the interleaving model, tools assume that threads execute independently but have a *canonical schedule* [72]: a sequential interleaving of threads that is equivalent in behavior to the parallel execution of the program. In reality, most GPUs have a weak memory-consistency model [12, 314], so this assumption only holds for programs that lack data races. When races occur, execution may not be sequentially consistent: as an example, consider the program using shared memory in Figure 44. Weak memory means it is possible for `a` to be `1` and `b` to be `0` after execution of this program, even though no sequential interleaving of these instructions could yield this state.

Lockstep semantics, on the other hand, treats all the threads under analysis as executing simultaneously, more accurately describing the behavior of threads within the same warp on pre-Volta hardware. However, the lockstep model struggles to reason about threads in different warps, and the

Table 2. A taxonomy of formal-methods techniques for GPU programs, ordered chronologically. The heavy line separates analyses developed before (above) and after (below) the release of the Volta [243] architecture. The **Purpose** column describes what each tool aims to achieve. Legend: ⊘ indicates that the tool verifies the absence of bugs (or conformance to a specification); ⚠ indicates that the tool identifies the presence of bugs (or violations of a specification); ◔ indicates that the tool quantifies the performance cost of bugs.
The **Issues** column describes which tasks or problems each tool targets. Legend: ⁝⁝ bank conflicts; ‖ barrier synchronization; ⛶ block-size independence; ϟ data races; 〰 determinism; ✓ functional correctness; = kernel equivalence checking; ⛾ memory coalescing; ⛨ memory safety; ■ termination; ⋔ warp divergence.

Tool	Methodology	Target	Purpose	Issues	Limitations
PUG [203, 204]	Model Checking	CUDA	⚠⊘	⁝⁝‖ϟ✓	Only targets races in SHMEM; verification for functional correctness only, scaling poorly with increasing numbers of threads
KLEE-CL [79]	Symbolic Execution [182]	OpenCL	⚠	ϟ=	Nonparametric in kernel configuration
Leung et al. [201]	Test Amplification	CUDA LLVM [198]	⚠	ϟ〰	Requires test cases for properties to be exercised
GPUVerify [26, 38, 39]	Operational Semantics in Boogie [29]	CUDA OpenCL	⊘	‖ϟ✓	Nonparametric; unsound for certain features; models blocks as having one warp
GKLEE [72, 205, 207]	Symbolic Execution	CUDA	⚠	⁝⁝‖ϟ⛾⋔	Execution model is only valid up to the first occurrence of a data race
SESA [208]	Symbolic Execution	CUDA	⚠	ϟ	Limited to data races only
VerCors [16, 43]	Hoare Logic [153] Concurrent Sep. Logic [269]	OpenCL	⊘	‖ϟ✓	High annotation burden [352]; unsound assumptions about barrier divergence
WEFT [306]	Symbolic Execution	PTX	⊘	‖ϟ	Only handles warp-specialized code; predates PTX spec. [220]
CIVL [308]	Symbolic Execution Model Checking	CUDA	⊘	‖ϟ✓=⛨	No support for atomics [352]
GPUCSL [21]	Concurrent Sep. Logic Information Flow [93]	OpenCL	⊘	‖✓	Inherits high annotation burden from VerCors
Vericuda [189–191]	Hoare Logic	CUDA	⊘	✓	Requires data-race freedom as a precondition [352]
ESBMC-GPU [233, 277]	Model Checking	CUDA	⚠⊘	ϟ✓⛨	Can only verify correctness properties with user annotations
Ketema and Donaldson [177]	Term Rewriting	CUDA OpenCL	⊘	■	Requires data-race freedom as a precondition
GPUDrano [13]	Abstract Interpretation [87]	CUDA	⚠	⛾	Assumes warp-synchronicity
Alur et al. [14]	Abstract Interpretation Symbolic Execution	CUDA	⊘	⛶	Requires sync-free, race-free input (no shared memory); assumes warp-synchronicity
Xing et al. [366]	Model Checking	PTX	⊘	ϟ	Unsoundly unrolls loops [352]
CUDA au Coq [114]	Machine Validation	PTX	⊘	✓■	Requires manual proofs in Rocq
RaCUDA [236, 237]	Hoare Logic Quantitative Logic	CUDA	◔	⁝⁝⛾⋔	Cannot handle thread-divergent loops; assumes warp-synchronicity
Faial [76]	Memory-Access Protocols	CUDA	⚠	ϟ	Reports benign races as errors; does not support atomics
FaialAA [213]	Memory-Access Protocols	CUDA	⚠⊘	ϟ	True Positives guarantee requires control-data independence
Dartagnan [345]	Model Checking	PTX SPIR-V [338]	⊘	✓	Cannot reason about barrier divergence
Volta [99]	Symbolic Execution	PTX	⊘	‖ϟ=	Specialized to ML programs; analysis limited to one thread block
Pico [42]	Memory-Access Protocols Relational Cost Analysis [74]	CUDA	◔	⁝⁝⛾⋔	Certain loops and conditional patterns cause cost to be overestimated
Dirigo [41]	Symbolic Execution	PTX	⚠	‖ϟ⛨=	Specialized to ML programs; no Tensor Core support despite ML focus

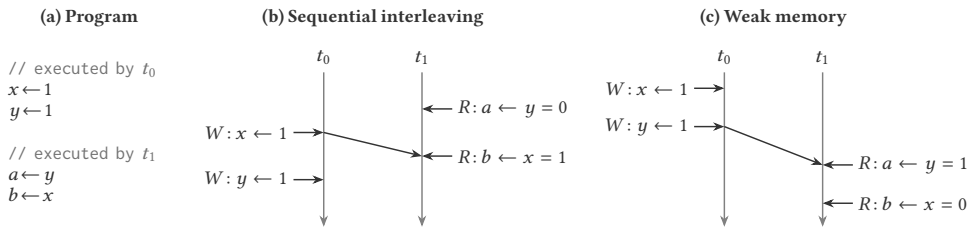


Fig. 44. In (a), a program where t_0 executes $x \leftarrow 1$; $y \leftarrow 1$ and t_1 executes $a \leftarrow y$; $b \leftarrow x$. Initially $x = y = 0$. In (b) and (c), two possible parallel executions of this program; diagonal arrows link writes to reads that observe them. In (b), threads are interleaved sequentially, and t_1 reads y before it is written. In (c), weak-memory behaviors cause t_1 to observe the write to y but not the write to x, an outcome no sequential interleaving can produce.

advent of Independent Thread Scheduling [256] meant that lockstep semantics no longer accurately modeled intra-warp execution either. Work exists proving the equivalence of lockstep semantics to interleaved ones [80, 190], but these proofs also require data-race freedom, making this property a prerequisite to sound program analysis for either choice of model.

Some work has proposed new models of execution for GPUs, particularly focusing on capturing the semantics of collective operations like Tensor Cores to allow tools to analyze their behavior. Valpey et al. [351] developed an SMT-based formal model for Volta- and Ampere-generation Tensor Cores, which they used to characterize architectural differences and fix incorrect assumptions about Tensor Core behavior. SIMT-Step execution [71], meanwhile, proposed a new operational semantics for GPUs with configurable lockstep granularity, enabling analyses to specify which groups of threads must execute together. This flexibility makes the semantics portable across GPUs, where the sizes and synchronicity of thread groups may differ (e.g., NVIDIA warps vs. AMD wavefronts). SIMT-Step can also track which threads enter each basic block in a program's control-flow graph, allowing it to describe the collective execution of Tensor Cores and providing a promising foundation for future tools that wish to analyze them.

4.2 Early Bug-Finding Approaches

One of the earliest tools for analyzing GPU programs was **PUG** [203], which was released shortly after the Tesla architecture and uses SMT-based model checking to find data races, bank conflicts, and synchronization errors in CUDA kernels, as well as verify user-provided assertions. PUG uses an interleaving semantics with standard techniques for model checking sequential programs but extends them with new ways to encode thread interleavings and barrier semantics as SMT formulas. One particularly important insight was that race detection could be reduced to reasoning about an arbitrary pair of threads, rather than every possible combination thereof.

PUG is an early exemplar of several common trends in the analysis of GPU kernels; many of its techniques (e.g., the interleaving semantics), simplifying assumptions (e.g., the two-threaded approach to races), and limitations are endemic in later work. **ESBMC-GPU** [233, 277], for example, is a GPU-focused extension to the Efficient SMT-Based Context-Bounded Model Checker (ESBMC) [84] framework—originally designed for multithreaded CPU programs—that built upon PUG's ideas and uses many of its techniques, applying them to new problem domains like memory safety.

A key challenge faced by formal-methods tools for GPUs is managing the launch configuration of the kernel being analyzed. This configuration can vary across several factors—including number of blocks, threads per block, and dimensionality of both threads and blocks—all of which can affect the properties under analysis. PUG's initial solution was to require the user to specify the launch configuration at each invocation of the tool, a *nonparametric* approach that limited the classes of bugs it could identify. If the tool reported no bank conflicts on a kernel launched with two threads,

for example, this indicated nothing about whether bank conflicts might be present in the same kernel launched with three threads instead. The nonparametric approach also adversely impacted PUG's performance; since PUG frequently timed out when analyzing kernels with as few as four threads [207], it could not hope to check most functional-correctness properties of interest. An alternative *parametric* approach was implemented in a later extension to PUG, called PUG_{para} [204], which allowed it to reason about kernels regardless of their particular configuration by drawing from a parallel stream of contemporaneous work on symbolic execution.

That stream began with **KLEE-CL** [79], an extension of the KLEE [53] bug-finding tool, which uses symbolic-execution [182] techniques to find data races and behavioral mismatches between kernels and the C++ code from which they were generated. KLEE had previously been ported to the SIMD model in the form of KLEE-FP [78]; KLEE-CL further extended those ideas to SIMT by targeting OpenCL. Symbolic execution allows tools to explore all the possible execution paths through a program by evaluating code on *symbolic inputs*. Initially, these inputs are unconstrained, but as evaluation proceeds, assumptions and constraints on input values are collected and propagated through the analysis. Whenever input-dependent control is encountered, the analysis forks and checks all branches, recording assumptions about the input reflecting the control path taken.

GKLEE [205] also built on KLEE, using *concolic execution*, a hybrid analysis technique combining concrete and symbolic execution [54, 122, 303], to detect both correctness and performance bugs in GPU kernels. Concolic execution modifies the pure symbolic approach by holding some inputs concrete; its main advantage is that searching for a bug also produces a test case that demonstrates it. Unlike that of many other race-detection tools, GKLEE's analysis of data races is prescient, prefiguring the arrival of ITS by handling both inter- and intra-warp data races. However, GKLEE uses an interleaving semantics, meaning that its model of GPU semantics is only valid for the race-free prefix of the canonical schedule it analyzes; it can reliably detect the first data race in a kernel, but it will not necessarily catch any subsequent races or other bugs.

Like PUG before it, GKLEE was originally conceived as a nonparametric tool, although it could scale to far more threads—around 2000. GKLEE's approach to parametric analysis came in the form of GKLEE_{p} [207], which developed the idea of *parametric flows*, also used later by PUG_{para}. Parametric flows rely on the assumption that most GPU kernels are behaviorally symmetric—that is, most threads do similar things—as a consequence of the control-data independence property we discussed in Theme II. Thus, instead of analyzing arbitrary pairs of threads, analyses can sort threads into equivalence classes based on divergence behavior; the data-race detection problem can then be reduced to finding races between arbitrary threads within each class and between representatives of each class, as depicted in Figure 45. Whereas previous concolic-execution tools needed to use concrete values for thread IDs, parametric flows allow IDs to be held symbolic during evaluation, further reducing the number of threads that must be considered. A subsequent tool called **SESA** [208] used information flow to combine equivalence classes that only differ on non-control data. Its authors note, however, that the parametric approach to symbolic execution does not scale well to complex functional-correctness properties, as symbolic thread IDs require many manual annotations. Indeed, Table 2 shows that few symbolic-execution tools after GKLEE attempt to target functional correctness.

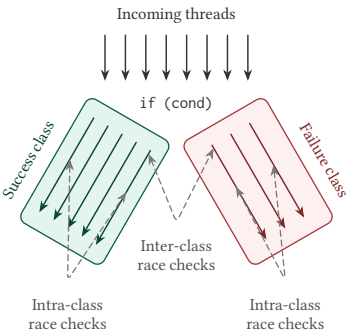


Fig. 45. Parametric flows handling a conditional in GKLEE_{p} [207]. Races are checked between pairs of arbitrary threads within the classes that succeed and fail the conditional and between representatives of the two classes.

Even with a parametric approach, GKLEE still suffered another limitation: it could not handle programs that used *atomic operations* [249], which are guaranteed to execute without interference from other threads. The interaction between atomics and barriers had previously been dismissed as too complex for practical analysis, but an extension to GKLEE called $\text{GKLEE}_{\text{atm}}$ [72] used *conflict-directed delay-bounding* to find canonical schedules even in the presence of atomics. Given a naïve schedule that does not consider atomics, the delay-bounding approach records conflicts between the naïve schedule and any atomic operations encountered during evaluation. These conflicts are then used to delay threads to generate candidate schedules for the kernel; an error encountered by any such schedule is a potential bug in execution.

A separate stream of early research used information flow to perform *test amplification* [201] on CUDA kernels, expanding the coverage of individual tests by extrapolating their results to entire classes of input. The technique works as follows: First, a user runs a test to dynamically check whether a given property of a program holds for some concrete input. Then, information-flow-based reasoning determines which parameters of the program can influence the property under test (also called the *property-integrity inputs*). The test result can then be extrapolated to any other execution with equal property-integrity inputs; any other parameters can differ, and the property will still hold or not hold as in the original test. Test amplification is effective on GPU kernels due to control-data independence: configuration data like thread IDs or tensor dimensions are typically property-integrity inputs, while tensor contents are not.

4.3 Early Verification Approaches

While tools like PUG and GKLEE focused on finding bugs in incorrect kernels, other work focused on verifying the absence of bugs in correct ones. An early such tool was **GPUVerify** [38, 39], which aimed to guarantee the absence of data races and barrier divergence in CUDA and OpenCL kernels, while also checking some functional-correctness properties. GPUVerify's approach is based on *synchronous, delayed visibility* (SDV) semantics, in which kernels are executed by a single, synchronous warp. To express the possibility of data races, writes to memory are delayed until the warp reaches a synchronization barrier, after which writes are propagated to their respective locations in memory; in this semantics, depicted in Figure 46, reading a recently written value before synchronizing (i.e., a data race) causes a run-time error. Betts et al. [38] used SDV to model GPU execution independently of its underlying hardware structure, making GPUVerify portable across architectures. Collingbourne et al. [80] later proved an equivalence between GPUVerify's semantics and an interleaved one for race-free programs, allowing the tool to later support Independent Thread Scheduling. However, recent hardware developments have posed a challenge for SDV, as its single-warp semantics does not capture the warpgroup- and block-level collective operations that are now hallmarks of performant kernels.

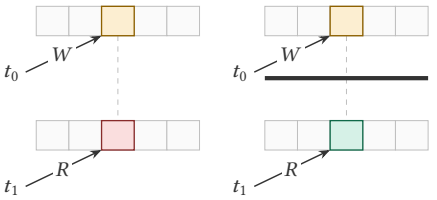


Fig. 46. The SDV semantics of GPUVerify [38]. Left: reading a pending write causes a run-time error. Right: after synchronizing, the read succeeds.

To analyze a kernel, GPUVerify first infers loop invariants for it, then encodes both the program and a description of its evaluation in the Boogie [29] intermediate verification language (IVL); Boogie then uses the Z3 [92] SMT solver to determine whether these invariants hold. To avoid a combinatorial explosion of SMT formulas, GPUVerify employs a technique similar to PUG's, reducing the number of threads under consideration to just two. Among all the static verifiers in their survey, van den Haak et al. [352] evaluate GPUVerify as the most practical, citing its ability to automatically infer loop invariants in many cases. They also note, however, that its analysis is sometimes unsound—for example, it assumes array accesses are always in bounds. Later work by Bardsley and Donaldson

[26] extended GPUVerify to handle atomics. Unlike GKLEE$_{\text{atm}}$, which explores possible canonical schedules heuristically, GPUVerify reasons about thread interleavings abstractly, treating the values returned by shared-memory reads as arbitrary. Sometimes, however, these values cannot occur in a real execution of the kernel under analysis, making impossible states appear reachable and leading to false positives.

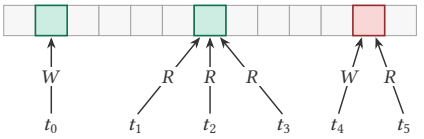


Fig. 47. VerCors [43] allows each memory location an exclusive writer or shared readers. A writer and reader at the same location conflict.

VerCors [43] was another approach to verification that used Hoare logic [153] and *permission-based separation logic* (PSL) [49] to prove a variety of correctness properties about OpenCL kernels. PSL extends concurrent separation logic [269] by adding permissions to its annotation language; correctness properties must contain the appropriate read or write permissions to be verified. Specifically, at most one thread can have a write permission to a memory location, while threads can share read permissions to a location so long as no thread can write to it. These restrictions, depicted in Figure 47, mean that any program verified by PSL is inherently data-race free.

To use VerCors, users begin by describing the initial permissions associated with a kernel when it is launched. At each barrier synchronization, they then specify pre- and postconditions about program state and the distribution of permissions across threads. van den Haak et al. [352] identify VerCors as the most versatile verification tool for functional correctness but note that it imposes a heavy annotation burden on users. However, recent work on ProofWright [65] has used LLMs to automatically generate these annotations, potentially alleviating some of this burden. In addition to using VerCors to find races and prove memory safety, ProofWright attempts to automatically translate kernels into Rocq to generate proofs of functional correctness.

Like GPUVerify, VerCors was subsequently extended with support for atomics [16], this time by distinguishing two kinds of permissions at synchronization barriers: kernel resource invariants, which are properties that hold for entire kernels, and group resource invariants, which hold only for each work-group (block). Each barrier's kernel and group specifications can be derived by the universal separating conjunction over the specifications of all the threads in the kernel and in the group it synchronizes, respectively. This extension allowed VerCors to verify programs using barriers and atomics, but later work by Asakura et al. [21] identified a critical issue: VerCors assumes that thread-ID independence—the condition that control flow and the values of variables are independent of thread ID—is sufficient to guarantee the absence of barrier divergence. In fact, both thread-ID independence and data-race freedom are necessary to guarantee barrier-divergence freedom. Asakura et al. [21] also note that VerCors's formalization of its separation logic is nonstandard; specifically, it lacks a frame rule, which complicates proving soundness. They rectify both these issues with **GPUCSL**, which builds upon VerCors, proves the corrected logic's soundness in Rocq, and uses an information-flow type system to identify when data is thread-ID independent.

Another program logic for CUDA was developed by Kojima and Igarashi [189, 190] and later implemented in the **Vericuda** [191] verification tool. Rather than building on separation logic, Vericuda's approach extends Hoare logic directly, adding a new *mask* element specifying the set of active threads to the usual Hoare triple. Users decorate their programs with assertions and loop invariants (such a program is depicted in Figure 48), and Vericuda encodes these as input to an SMT solver; for the class of programs it can handle, Vericuda can efficiently and parametrically verify functional-correctness properties. However, Vericuda's

```
1  /*@ requires len == m * N;
2      ensures \forall int j; 0 <= j < len
3        ==> b[j] == a[j]; */
4  void ArrayCopy(int *a, int *b, int len) {
5    int i = tid;
6    /*@ loop invariant i == N * loop_count + tid;
7        loop invariant
8          \forall int j; 0 <= j < N * loop_count
9            ==> b[j] == a[j]; */
10   while (i < len) {
11     b[i] = a[i];
12     i = i + N;
13 }}
```
Fig. 48. Copying an array in Vericuda, from Kojima et al. [191].

semantics assumes that all threads execute in lockstep, so it could not originally handle Independent Thread Scheduling. Like Collingbourne et al. [80], however, Kojima and Igarashi [190] later proved their lockstep semantics equivalent to an interleaved one in the absence of data races, enabling Vericuda to soundly verify post-Volta kernels in that restricted setting.

While verifiers like GPUVerify and VerCors focused on a wide class of common GPU bugs, others either broadened or narrowed their focus. The Concurrency Intermediate Verification Language, or **CIVL** [308], aimed to be a common intermediate representation for verifying concurrency properties across a wide variety of parallel architectures. Languages like CUDA and OpenMP are compiled to CIVL's IR, and model checking and symbolic execution are used to verify the absence of many concurrency bugs.

WEFT [306], on the other hand, was a symbolic-execution tool that focused solely on warp-specialized code. WEFT was unusual among early tools in that it analyzed PTX rather than CUDA itself, which allowed it to support operations that did not exist in CUDA. In particular, WEFT analyzes programs that use *named barriers*. Unlike `__syncthreads()`, which synchronizes an entire block, these PTX primitives can synchronize groups of threads at finer granularity, which is necessary for warp specialization. WEFT posits a formal operational semantics for PTX and evaluates programs symbolically, leaving data values abstract but computing control flow and array accesses; it therefore requires kernels to be control-data independent. This use of PTX proved influential, inspiring later tools that sought to support low-level operations unavailable in CUDA. However, WEFT predates the official PTX memory model [220], which was released shortly after Volta, and its model of PTX's semantics may therefore not reflect how the ISA actually behaved at the time.

Few early formal-methods approaches considered the termination behavior of kernels, which is particularly important on GPUs because the result of a kernel cannot be viewed by its host until the kernel has terminated. Ketema and Donaldson [177]'s tool was the first to do so, building on the termination analyzer KITTeL [109], which translates single-threaded C programs into a term-rewriting system that it uses to automatically synthesize termination proofs. Their approach deals with the multithreaded nature of GPU programs by only considering the behavior of a single arbitrary thread, assuming control-data independence. Ketema and Donaldson [177] prove that if an arbitrary thread can be shown to terminate, then so must every thread; however, like Vericuda, they require data-race freedom as a prerequisite to analysis.

4.4 Formal Methods After Volta

Much as it had for programming languages, Volta changed the landscape for formal methods on the GPU, invalidating old assumptions and laying the groundwork for new approaches. With the advent of Independent Thread Scheduling, tools could no longer assume warp-synchronicity, and supporting Tensor Cores required them to analyze more than one or two threads at a time. On the other hand, the release of an official consistency model for PTX [220], spurred by research into GPUs' weak memory models [12, 314], opened the door to analyzing NVIDIA's ISA directly.

Despite the arrival of ITS, much of the early effort following the release of Volta continued to assume warp-synchronous execution. Examples of such work included **GPUDrano** [13], a tool based on abstract interpretation [87] that identifies uncoalesced memory accesses, as well as an analysis by Alur et al. [14] that verifies that the behavior of CUDA kernels is independent of the block size with which they are launched. As Kojima and Igarashi [190] argued earlier, assuming warp-synchronicity restricts sound analysis to kernels without data races, and the latter tool is further limited to kernels that use neither synchronization nor shared memory.

A particularly notable new tool for identifying data races in GPU kernels is **Faial** [76], which built upon the SDV semantics of GPUVerify to present *memory-access protocols*, a new abstraction of array accesses that eschews information about what is being read from or written to memory,

focusing instead on where the reads and writes are happening. For example, `A[tid + i] = B[tid * X + j]` would translate to the protocol `read[tid * X + j]; write[tid + i]`. Faial encodes these protocols as SMT formulas, which it uses to identify whether a race is possible in a given protocol; like many other tools before it, by abstracting the details of the data being read from memory, Faial implicitly assumes control-data independence. Consequently, Faial cannot distinguish between benign and harmful data races, as shown in Figure 49. Faial was later extended by **FaialAA** [213], which addressed this issue, added support for atomics, and reduced its rate of false alarms. In particular, FaialAA can prove a True Positives theorem (i.e., that all reported data races are real) for control-data-independent kernels.

Separately, a strand of research arose measuring the *cost* of performance bugs in kernels. The first such tool was **RaCUDA** [236, 237], which developed a program logic based on *amortized resource analysis* [155]. RaCUDA infers symbolic resource bounds by assigning each program state a resource budget, called its *potential*. Operations consume potential as the program executes, with inefficient events like bank conflicts and thread divergence costing more. The cost of all kernel operations across all threads is called the *work*, and the maximum cost of any single warp is the *span*; RaCUDA estimates the cost of a kernel as its work divided by the maximum number of warps the GPU can schedule at a time, plus its span. Later, **Pico** [42] addressed a few of RaCUDA's limitations; in particular, RaCUDA cannot handle thread-divergent loops and tends to overestimate the cost of thread-divergent conditionals. Pico's approach is primarily based on Faial's memory-access protocols, but it also uses *relational cost analysis* [74] to compare the costs of programs. Specifically, Pico translates GPU kernels to sequential programs and uses this translation to bound the originals' costs. RaCUDA's and Pico's implementations both assume control-data independence to make their analyses more tractable.

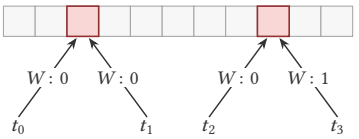


Fig. 49. In a *benign* data race (left), threads write the same value to the same location. In a *harmful* data race, threads write different values. Faial [76] cannot distinguish between the two; later work on FaialAA [213] is more precise.

Spurred by PTX's new memory model, several other projects began exploring the possibility of analyzing the ISA directly. Work by Xing et al. [366] uses model checking to identify data races; like WEFT, their work supports inline PTX instructions, which have become increasingly essential with the emergence of new hardware features without CUDA support. However, Xing et al. [366]'s tool unsoundly unrolls loops as part of its analysis [352]. **CUDA au Coq** [114] is another PTX-level approach, embedding the ISA's semantics in Rocq and providing a framework for proving functional-correctness properties and termination.

Instead of avoiding weak-memory behaviors by assuming data-race-free programs, working at the level of PTX allows tools to reason about GPU behavior even in the presence of races. In particular, **Dartagnan** [345] encodes the memory-consistency models of both PTX and SPIR-V and can thus soundly verify properties about GPU programs that have data races. During development, Dartagnan also uncovered a handful of bugs in the specifications of both PTX and SPIR-V.

As machine learning has become a primary application for GPUs, tools have emerged specifically targeting ML kernels; one such tool is **Dirigo** [41], which checks the equivalence of a kernel pipeline against a reference PyTorch program, evolving GKLEE's parametric concolic-execution approach for the modern era. Dirigo executes kernels with fully symbolic thread IDs and array values but spot-checks equivalence with their reference programs by concretizing thread IDs at specific output locations. This approach often does not scale to checking every location, so the tool is aimed at bug finding rather than full verification.

A related tool, **Volta** [99] (not to be confused with the microarchitecture), takes a complementary approach. Volta checks the equivalence of ML kernels, making the problem tractable by only comparing pairs of individual kernels and only executing the first block of each. Unlike Dirigo, Volta aims to verify kernel equivalence (up to the stated bounds), rather than just find bugs. Both tools assume control-data independence.

Notably, Volta is the only tool we surveyed that supports Tensor Core operations, modeling the warp-level semantics of Ampere's Cores. However, both Dirigo and Volta position themselves as tools motivated by a recent rise in LLM-driven optimization of ML kernels [30, 358], many of which use Hopper- and Blackwell-era Tensor Cores. These Cores are the workhorses of modern training and inference, but their asynchronous programming models are not supported by Dirigo, Volta, or any other existing tools. As discussed in Theme I, correctness is of little use without efficiency, so the lack of analyses that can handle modern hardware features is a major gap in the current formal-methods literature and an important opportunity for future work.

4.5 Dynamic Approaches

Many dynamic analyses for GPU programs also exist; full enumeration and in-depth discussion of these are out of scope for this survey, but we list a representative sampling as a starting point for interested readers. BARRACUDA [104], CURD [275], HAccRG [156], HiRace [169], GRace [380], GMRace [381], LD [206], NVIDIA's Compute Sanitizer [251], Oclgrind [282], iGuard [174], ScoRD [175], cuFuzz [162], SuperCollider [318], and the work of Boyer et al. [50] detect data races. Simulee [365], CURD, BARRACUDA, and the Compute Sanitizer can also be used to find sources of barrier divergence, and the Compute Sanitizer, cuCatch [161], cuFuzz, and Boyer et al. [50]'s work also identify inefficient or invalid memory accesses. Hayes et al. [147]'s work finds information leaks. NVIDIA's Nsight [257] suite is commonly used to profile kernels and analyze their performance. Guardian [274] enforces safe memory sharing between multiple kernels running on the same GPU. GLFuzz [98] identifies bugs and security vulnerabilities in compilers for shading languages. GPU-FPX [212] identifies floating-point bugs in kernels that use Tensor Cores. Some dynamic tools like GRace [380] use static techniques to prune cases from their analyses when they can verify that these cases cannot cause a data race. We also direct readers toward van den Haak et al. [352]'s survey of formal methods for GPU programming for further discussion of both static and dynamic analyses.

5 Conclusion

In this work, we have surveyed the research on language design and formal methods for GPUs. We traced the genealogy of programming languages for GPUs, tracking how ideas spread between languages and led to new paradigms, and we taxonomized formal-methods approaches, describing the various approaches to kernel analysis and identifying gaps in the literature. Along the way, we identified three major themes of GPU programming and observed their impact on the programming-languages and formal-methods landscape.

Theme I—that performance is critical on the GPU—was a constant refrain. Performance is a key factor in GPU language design, and subpar performance frequently motivates further research and evolution [81, 128, 326, 346]. This theme also points to a major gap in current formal-methods research: the lack of tools that support modern hardware features like Tensor Cores, TMAs, and TMEM. Moreover, there is an urgent need for tools that can reason about kernel performance, whether by quantifying the cost of suboptimal patterns or by guaranteeing their absence.

Theme II—that kernel logic is often simple while its optimization is complex—has spawned an ecosystem of programming languages that expose precise control over hardware or compilation. GPUs have a vast optimization space, and searching it is hard; many languages target specialized domains to narrow the search space [235, 353], while others shift the burden of optimization onto the

programmer [25, 134, 239, 288]. Another consequence is that formal-methods tools widely assume control-data independence [76, 99, 207, 236]. While this property does not hold in all domains, it is common in important ones like graphics and deep learning.

Theme III—that GPU hardware is diverse and constantly evolving—is reflected in a high rate of kernel churn, as developers reimplement their code across multiple architectures to improve performance. Performance portability is thus a common goal in language design, but some argue that it is futile and that programmers would be better served by languages giving them more control over how their code is compiled and optimized [163, 164].

The GPU's rapid evolution has also influenced the design and usability of its programming languages and formal-methods tools. For example, the move from warp-synchronous execution to Independent Thread Scheduling has challenged formal methods, and the evolution of GPUs from bulk-synchronous scalar processors to an asynchronous tensor model has upended both language design and program analysis. In particular, Triton and the tile-based programming model were an elegant design for the synchronous, warp-level era of Tensor Cores, but GPUs' increasingly asynchronous, heterogeneous hardware demands new solutions. These may entail smarter compilation for tile-based languages [68, 313], more powerful types for SIMT [25, 132, 194], adaptations of high-performance computing paradigms like task-based programming [149, 369], or as-yet-undiscovered innovations.

Acknowledgments

We thank Zain Aamer, Andrew Adams, Manya Bansal, Gilbert Bernstein, Jonathan Chan, Tiago Cogumbreiro, Joe Cutler, Joe Devietti, Alastair Donaldson, Ganesh Gopalakrishnan, Vinod Grover, Troels Henriksen, Fredrik Kjolstad, Bastian Köpcke, Ernest Ng, Alexander Root, Adrian Sampson, Tyler Sorensen, Michel Steuwer, Stephanie Weirich, Rohan Yadav, and Laura Zielinski for thoughtful advice, reading suggestions, and feedback on early drafts. Alexander Root in particular was instrumental to the discussion of the shading languages in Section 3.6 and gave us valuable suggestions about the structure of the PL genealogy.

This material is based on work supported by the National Science Foundation CISE Graduate Fellowships under Grant No. 2313998. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

A Polyhedral Compilation

Polyhedral compilation originated outside the GPU space in the compilation of *loop-nest* [111] programs, whose primary structure is a series of nested `for` loops. The polyhedral model envisions the iteration domain of a loop nest of depth n as an n-dimensional polyhedron, or *polytope*. Each statement within a loop nest corresponds to a polytope, and the coordinates of points within that polytope correspond to the values of the index variables of the loop nest. A program can be *perfectly nested*, meaning that all its statements appear in the body of the innermost loop, or *imperfectly nested*, meaning that some statements appear at higher levels of the nest or multiple loops appear sequentially within an outer loop. As an example, the assignment on line 3 of Figure 50 can be envisioned as a triangle in 2D space, with base and height equal to $N-1$. Each pair of i and j that are valid loop indices—that is, when $j < N$ and $i \leq j$—corresponds to a point within this triangle, while the iteration bounds of the loop nest define the edges of the statement's *iteration-space polytope*.

```
1 for (int i = 0; i < N; i++) {
2   for (int j = i; j < N; j++) {
3     arr[i][j] = arr[i][j] * 2;
4   }
5 }
```

Fig. 50. A simple loop nest.

The polyhedral model is a convenient way of reasoning about program optimizations that restructure loop nests (e.g., tiling), as such optimizations can be represented as affine functions on the polytopes of a loop nest that preserve key geometric properties and dependencies. Polyhedral

```
// a length 2 vector, encoded as a 2x1 matrix
local(2, 1)
// a 2x3 matrix, distributed over the same
// address in 6 adjacent threads instead of 6
// adjacent memory addresses
spatial(2, 3)
// a 2x2 matrix via layout composition
local(2, 1).local(1, 2)
// A Tensor Core input layout
local(2, 1).spatial(8, 4).local(1, 2)
```

Fig. 51. Example layouts in Tilus, from Ding et al. [97].

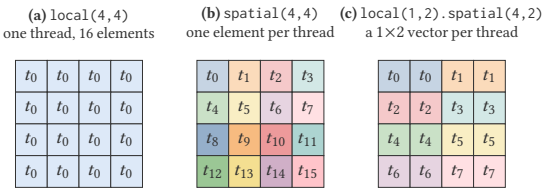


Fig. 52. Three layouts assigning the elements of a 4×4 tile to threads, in the notation of Figure 51; each cell is labeled by the thread that holds it in its registers. Composition (c) yields the distributed fragments: each thread holds a small contiguous vector, and consecutive threads hold adjacent vectors.

compilation has a long history that we cannot cover in detail; we focus here on its applications to GPUs and direct readers to Thangamani et al. [335]'s survey. In particular, Courant and Leroy [86]'s work on verifying the correctness of a simple polyhedral compiler may be of interest to a PL audience.

The polyhedral model is a natural fit for programs that iterate deeply over multidimensional structures like images and tensors. Consider the matrix-multiplication algorithm we saw in Figure 11: it is a doubly nested loop, implicitly wrapped in a third parallel loop following the SIMT intuition that kernels execute inside a parallel `for` loop whose indices are thread IDs. In addition, the polyhedral model requires the control flow of its loop nests to be statically determinable; that is, loop bounds and conditional tests must be affine functions of compile-time constants and the iteration variables of surrounding loops. This is a similar requirement to the control-data independence property that is already common in GPU kernels, so it is no surprise that the ideas behind polyhedral compilation were adapted to the SIMT model. One of the first such adaptations was that of Baskaran et al. [31], which built on the existing PLuTo [47, 48] polyhedral compiler; their work automatically unrolls and tiles loops to improve data locality and parallelism. However, one cannot achieve optimal performance by naïvely applying parallel CPU techniques to GPUs; as discussed in Theme II, few optimizations are uniformly beneficial on GPUs, so Baskaran et al. [31]'s compiler must search for the best parameters for tiling dimensions and loop-unrolling depth to balance concerns like parallelism with register pressure. Their compiler also attempts to automatically coalesce global-memory accesses and eliminate bank conflicts by reordering iteration, rearranging independent accesses, or inserting shared-memory indirection.

Many of the languages we discuss in this survey, like TIRAMISU [24] and Tensor Comprehensions [353], rely on the polyhedral model for their compilation, whether for parallelizing sequential code or for domain-specific loop transformations.

B Algebraic Layouts

CUTLASS, CuTeDSL, Graphene, Mojo, and Tilus feature systems of algebraic tensor layouts designed to streamline tile programming by abstracting tile shapes and giving operations over them an elegant compositional theory. A tensor's *layout* is a bijection between the tensor's logical indices and the actual locations that hold its elements. Layouts can represent tensors compactly in memory, and operations such as tiling or transposition can then modify the layout rather than transforming the data itself. This is quicker to compute, and, unlike direct transformation, its cost is independent of tensor size. The most basic layout stores a multidimensional tensor as a flat array; when accessing an element, it computes the element's array position from its indices and the tensor's dimensions. More complex layouts originate from libraries and compilers for tensor algebra, such as Kokkos [102, 103] and Taco [184], and were later reified as language features by PyTorch [272] and TensorFlow [307].

The CUTLASS library incorporated CuTe [255], a system of layouts for Tensor Core programming that lets programmers specify how tensors are distributed across both memory and compute resources. Tilus [97] supports these two methods of distribution with two primitive kinds of layouts: in *local* layouts, the tensor exists within a single thread, whereas in *spatial* layouts, it is shared among threads. Ding et al. [97] show that many common layouts can be constructed by composing these primitives. Figure 51 shows some Tilus layouts; the layout on line 10 exactly describes the shape expected by 16×8 Tensor Core instructions. Figure 52 visualizes how local and spatial layouts (and their compositions) assign the elements of a tile to threads. CUTLASS and CuTeDSL [61] also provide operators for composing layouts, and later work gives a full category-theoretic treatment of their theory [56]. These layouts influenced an extension of Triton [383] that added linear algebraic layouts.

References

[1] Martín Abadi, Anindya Banerjee, Nevin Heintze, and Jon G. Riecke. 1999. A Core Calculus of Dependency. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Antonio Texas USA, 1999-01). ACM, 147–160. doi:10.1145/292540.292555

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USENIX Association, USA, 265–283.

[3] Hamdy Abdelkhalik, Yehia Arafa, Nandakishore Santhi, and Abdel-Hameed Badawy. 2022. Demystifying the Nvidia Ampere Architecture through Microbenchmarking and Instruction-level Analysis. (2022). https://arxiv.org/abs/2208.11174v1

[4] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, and Jonathan Ragan-Kelley. 2019. Learning to Optimize Halide with Tree Search and Random Programs. *ACM Transactions on Graphics* 38, 4 (Aug. 2019), 1–12. doi:10.1145/3306346.3322967

[5] Adobe. 2025. Inside Halide, the open source language engineers use to make imaging tools faster. https://research.adobe.com/news/inside-halide-the-open-source-language/ Accessed: 2026-06-20.

[6] Advanced Micro Devices, Inc. 2025. Introducing AMD CDNA 4 Architecture: Breakthrough AI and HPC Acceleration with Enhanced AI Capabilities, Advanced Precisions, and High Efficiency. (2025). https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-4-architecture-whitepaper.pdf Accessed: 2026-06-01.

[7] Advanced Micro Devices, Inc. 2025. RDNA4 Instruction Set Architecture Reference Guide. https://docs.amd.com/v/u/en-US/rdna4-instruction-set-architecture Accessed: 2026-07-25.

[8] Advanced Micro Devices, Inc. 2026. Introduction to the HIP programming model. https://rocm.docs.amd.com/projects/HIP/en/latest/understand/programming_model.html Accessed: 2026-05-29.

[9] Alex Aiken, Michael Bauer, and Sean Treichler. 2014. Realm: An Event-Based Low-level Runtime for Distributed Memory Architectures. In *2014 23rd International Conference on Parallel Architecture and Compilation Techniques (PACT)*. 263–275. doi:10.1145/2628071.2628084

[10] Timo Aila and Samuli Laine. 2009. Understanding the Efficiency of Ray Traversal on GPUs. In *Proceedings of the Conference on High Performance Graphics 2009* (New Orleans, Louisiana) *(HPG '09)*. Association for Computing Machinery, New York, NY, USA, 145–149. doi:10.1145/1572769.1572792

[11] David Zhao Akeley, Yuka Ikarashi, and Jonathan Ragan-Kelley. 2026. Exo-GPU: Safe, Imperative, User-schedulable Programming for Tensor Cores. arXiv:2609.16389 [cs.PL] doi:10.48550/arXiv.2609.16389

[12] Jade Alglave, Mark Batty, Alastair F. Donaldson, Ganesh Gopalakrishnan, Jeroen Ketema, Daniel Poetzl, Tyler Sorensen, and John Wickerson. 2015. GPU Concurrency: Weak Behaviours and Programming Assumptions. *ACM SIGARCH Computer Architecture News* 43, 1 (May 2015), 577–591. doi:10.1145/2786763.2694391

[13] Rajeev Alur, Joseph Devietti, Omar S. Navarro Leija, and Nimit Singhania. 2017. GPUDrano: Detecting Uncoalesced Accesses in GPU Programs. In *Computer Aided Verification*. Springer International Publishing, Cham, 507–525. doi:10.1007/978-3-319-63387-9_25

[14] Rajeev Alur, Joseph Devietti, and Nimit Singhania. 2018. Block-Size Independence for GPU Programs. In *Static Analysis* (Cham, 2018). Springer International Publishing, 107–126. doi:10.1007/978-3-319-99725-4_9

[15] Amazon Web Services. 2026. Amazon EC2 G4 Instances. https://aws.amazon.com/ec2/instance-types/g4/ Accessed: 2026-07-16.

[16] Afshin Amighi, Saeed Darabi, Stefan Blom, and Marieke Huisman. 2015. Specification and Verification of Atomic Operations in GPGPU Programs. In *Software Engineering and Formal Methods* (Cham, 2015). Springer International

Publishing, 69–83. doi:10.1007/978-3-319-22969-0_5

[17] Jason Ansel, Cy Chan, Yee Lok Wong, Marek Olszewski, Qin Zhao, Alan Edelman, and Saman Amarasinghe. 2009. PetaBricks: A Language and Compiler for Algorithmic Choice. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Dublin Ireland, 38–49. doi:10.1145/1542476.1542481

[18] Apple Inc. 2023. Apple unveils M3, M3 Pro, and M3 Max, the most advanced chips for a personal computer. https://www.apple.com/newsroom/2023/10/apple-unveils-m3-m3-pro-and-m3-max-the-most-advanced-chips-for-a-personal-computer/

[19] Apple, Inc. 2026. Metal Shading Language Specification. https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf Accessed: 2026-08-27.

[20] Jeremy Appleyard and Scott Yokim. 2017. Programming Tensor Cores in CUDA 9. (2017). https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/ Accessed: 2026-06-03.

[21] Izumi Asakura, Hidehiko Masuhara, and Tomoyuki Aotani. 2016. Proof of Soundness of Concurrent Separation Logic for GPGPU in Coq. *Journal of Information Processing* 24, 1 (2016), 132–140. doi:10.2197/ipsjjip.24.132

[22] Joshua Auerbach, David F. Bacon, Ioana Burcea, Perry Cheng, Stephen J. Fink, Rodric Rabbah, and Sunil Shukla. 2012. A Compiler and Runtime for Heterogeneous Computing. In *Proceedings of the 49th Annual Design Automation Conference* (San Francisco, California) *(DAC '12)*. Association for Computing Machinery, New York, NY, USA, 271–276. doi:10.1145/2228360.2228411

[23] Joshua Auerbach, David F. Bacon, Perry Cheng, and Rodric Rabbah. 2010. Lime: A Java-Compatible and Synthesizable Language for Heterogeneous Architectures. *ACM SIGPLAN Notices* 45, 10 (Oct. 2010), 89–108. doi:10.1145/1932682.1869469

[24] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. 2019. Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 193–205. doi:10.1109/CGO.2019.8661197

[25] Manya Bansal, Daniel Sainati, Joseph W. Cutler, Saman Amarasinghe, and Jonathan Ragan-Kelley. 2026. Modular GPU Programming with Typed Perspectives. *Proc. ACM Program. Lang.* 10, PLDI, Article 212 (June 2026), 25 pages. doi:10.1145/3808290

[26] Ethel Bardsley and Alastair F. Donaldson. 2014. Warps and Atomics: Beyond Barrier Synchronization in the Verification of GPU Kernels. In *NASA Formal Methods* (Cham, 2014). Springer International Publishing, 230–245. doi:10.1007/978-3-319-06200-6_18

[27] Erik Barendsen and Sjaak Smetsers. 1993. Conventional and Uniqueness Typing in Graph Rewrite Systems. In *Proceedings of the 13th Conference on Foundations of Software Technology and Theoretical Computer Science*. Springer-Verlag, Berlin, Heidelberg, 41–51.

[28] Erik Barendsen and Sjaak Smetsers. 1996. Uniqueness Typing for Functional Languages with Graph Rewriting Semantics. *Mathematical Structures in Computer Science* 6, 6 (1996), 579–612. doi:10.1017/S0960129500070109

[29] Mike Barnett, Bor-Yuh Evan Chang, Robert DeLine, Bart Jacobs, and K. Rustan M. Leino. 2005. Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In *Proceedings of the 4th International Conference on Formal Methods for Components and Objects* (Amsterdam, The Netherlands) *(FMCO'05)*. Springer-Verlag, Berlin, Heidelberg, 364–387. doi:10.1007/11804192_17

[30] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. 2025. Kevin: Multi-Turn RL for Generating CUDA Kernels. arXiv:2507.11948 doi:10.48550/arXiv.2507.11948

[31] Muthu Manikandan Baskaran, Uday Bondhugula, Sriram Krishnamoorthy, J. Ramanujam, Atanas Rountev, and P. Sadayappan. 2008. A Compiler Framework for Optimization of Affine Loop Nests for GPGPUs. In *Proceedings of the 22nd Annual International Conference on Supercomputing*. ACM, Island of Kos, Greece, 225–234. doi:10.1145/1375527.1375562

[32] Michael Bauer, Henry Cook, and Brucek Khailany. 2011. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *SC '11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–11. doi:10.1145/2063384.2063400

[33] Michael Bauer, Sean Treichler, and Alex Aiken. 2014. Singe: Leveraging Warp Specialization for High Performance on GPUs. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Orlando, Florida, USA) *(PPoPP '14)*. Association for Computing Machinery, New York, NY, USA, 119–130. doi:10.1145/2555243.2555258

[34] Michael Bauer, Sean Treichler, Elliott Slaughter, and Alex Aiken. 2012. Legion: Expressing Locality and Independence with Logical Regions. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–11. doi:10.1109/SC.2012.71

[35] Eli Ben-Sasson, Matan Hamilis, Mark Silberstein, and Eran Tromer. 2016. Fast Multiplication in Binary Fields on GPUs via Register Cache. In *Proceedings of the 2016 International Conference on Supercomputing* (Istanbul, Turkey) *(ICS '16)*. Association for Computing Machinery, New York, NY, USA, Article 35, 12 pages. doi:10.1145/2925426.2926259

[36] James Bergstra, Olivier Breuleux, Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, Guillaume Desjardins, Joseph Turian, David Warde-Farley, and Yoshua Bengio. 2010. Theano: A CPU and GPU Math Compiler in Python. *SciPy 2010* (2010). doi:10.25080/Majora-92bf1922-003

[37] Gilbert Louis Bernstein, Chinmayee Shah, Crystal Lemire, Zachary Devito, Matthew Fisher, Philip Levis, and Pat Hanrahan. 2016. Ebb: A DSL for Physical Simulation on CPUs and GPUs. *ACM Transactions on Graphics* 35, 2 (May 2016), 1–12. doi:10.1145/2892632

[38] Adam Betts, Nathan Chong, Alastair Donaldson, Shaz Qadeer, and Paul Thomson. 2012. GPUVerify: A Verifier for GPU Kernels. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. ACM, Tucson Arizona USA, 113–132. doi:10.1145/2384616.2384625

[39] Adam Betts, Nathan Chong, Alastair F. Donaldson, Jeroen Ketema, Shaz Qadeer, Paul Thomson, and John Wickerson. 2015. The Design and Implementation of a Verification Technique for GPU Kernels. *ACM Transactions on Programming Languages and Systems* 37, 3 (June 2015), 1–49. doi:10.1145/2743017

[40] Somashekaracharya G. Bhaskaracharya, Julien Demouth, and Vinod Grover. 2020. Automatic Kernel Generation for Volta Tensor Cores. arXiv:2006.12645 doi:10.48550/arXiv.2006.12645

[41] Paul Biberstein, Joseph Devietti, and Mayur Naik. 2026. The Output-Space Hypothesis: Enumerative Equivalence Checking for Tensor Programs. arXiv:2609.19611 [cs.PL] doi:10.48550/arXiv.2609.19611

[42] Gregory Blike, Hannah Zicarelli, Udaya Sathiyamoorthy, Julien Lange, and Tiago Cogumbreiro. 2026. A Modular Static Cost Analysis for GPU Warp-Level Parallelism. *Proceedings of the ACM on Programming Languages* 10, POPL (Jan. 2026), 1471–1499. doi:10.1145/3776693

[43] Stefan Blom, Marieke Huisman, and Matej Mihelčić. 2014. Specification and Verification of GPGPU Programs. *Science of Computer Programming* 95 (Dec. 2014), 376–388. doi:10.1016/j.scico.2014.03.013

[44] François Bodin, Toru Kisuki, Peter Knijnenburg, Mike O'Boyle, and Erven Rohou. 1998. Iterative Compilation in a Non-Linear Optimisation Space. *Workshop on Profile and Feedback-Directed Compilation* (03 1998).

[45] Simon Boehm. 2022. How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog. https://siboehm.com/articles/22/CUDA-MMM Accessed: 2026-05-29.

[46] Jeff Bolz, Ian Farmer, Eitan Grinspun, and Peter Schröder. 2003. The GPU as Numerical Simulation Engine. (03 2003). https://www.researchgate.net/publication/2551205_The_GPU_as_Numerical_Simulation_Engine

[47] Uday Bondhugula, Muthu Baskaran, Sriram Krishnamoorthy, J. Ramanujam, Atanas Rountev, and P. Sadayappan. 2008. Automatic Transformations for Communication-Minimized Parallelization and Locality Optimization in the Polyhedral Model. In *Compiler Construction* (Berlin, Heidelberg, 2008). Springer, 132–146. doi:10.1007/978-3-540-78791-4_9

[48] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. 2008. A Practical Automatic Polyhedral Parallelizer and Locality Optimizer. *SIGPLAN Not.* 43, 6 (June 2008), 101–113. doi:10.1145/1379022.1375595

[49] Richard Bornat, Cristiano Calcagno, Peter O'Hearn, and Matthew Parkinson. 2005. Permission Accounting in Separation Logic. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Long Beach, California, USA) *(POPL '05)*. Association for Computing Machinery, New York, NY, USA, 259–270. doi:10.1145/1040305.1040327

[50] Michael Boyer, Kevin Skadron, and Westley Weimer. 2008. Automated Dynamic Analysis of CUDA Programs. (March 2008). https://www.nvidia.com/docs/IO/67190/stmcs08.pdf

[51] Ian Buck. 2009. From Brook to CUDA. https://www.nvidia.com/content/GTC/documents/1408_GTC09.pdf Accessed: 2026-05-29.

[52] Ian Buck, Theresa Foley, Daniel Horn, Jeremy Sugerman, Kayvon Fatahalian, Mike Houston, and Pat Hanrahan. 2004. Brook for GPUs: Stream Computing on Graphics Hardware. *ACM Transactions on Graphics* 23, 3 (Aug. 2004), 777–786. doi:10.1145/1015706.1015800

[53] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-coverage tests for complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation* (San Diego, California) *(OSDI'08)*. USENIX Association, USA, 209–224.

[54] Cristian Cadar, Vijay Ganesh, Peter M. Pawlowski, David L. Dill, and Dawson R. Engler. 2008. EXE: Automatically Generating Inputs of Death. *ACM Trans. Inf. Syst. Secur.* 12, 2, Article 10 (Dec. 2008), 38 pages. doi:10.1145/1455518.1455522

[55] D. Callahan, B.L. Chamberlain, and H.P. Zima. 2004. The Cascade High Productivity Language. In *Ninth International Workshop on High-level Parallel Programming Models and Supportive Environments, 2004. Proceedings.* 52–60. doi:10.1109/HIPS.2004.1299190

[56] Jack Carlisle, Jay Shah, Reuben Stern, and Paul VanKoughnett. 2026. Categorical Foundations for CuTe Layouts. https://arxiv.org/abs/2601.05972v1

[57] Adrián Castelló, Julian Bellavita, Grace Dinh, Yuka Ikarashi, and Héctor Martínez. 2024. Tackling the Matrix Multiplication Micro-kernel Generation with Exo. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization* (Edinburgh, United Kingdom) *(CGO '24)*. IEEE Press, 182–192. doi:10.1109/CGO57630.2024.10444883

[58] Adrián Castelló, Hèctor Martínez, Sandra Catalán, Jie Lei, Yuka Ikarashi, Grace Dinh, Francisco D. Igual, and Enrique S. Quintana-Ortí. 2025. Portable, High Performance Matrix Multiplication Micro-Kernels for RISC-V with ExO. In *2025 33rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*. 25–32. doi:10.1109/PDP66500.2025.00013 ISSN: 2377-5750.

[59] Bryan Catanzaro, Michael Garland, and Kurt Keutzer. 2011. Copperhead: Compiling an Embedded Data Parallel Language. In *Proceedings of the 16th ACM symposium on Principles and Practice of Parallel Programming*. ACM, San Antonio TX USA, 47–56. doi:10.1145/1941553.1941562

[60] Bryan Catanzaro, Alexander Keller, and Michael Garland. 2014. A Decomposition for In-Place Matrix Transposition. *SIGPLAN Not.* 49, 8 (Feb. 2014), 193–206. doi:10.1145/2692916.2555253

[61] Cris Cecka. 2026. CuTe Layout Representation and Algebra. arXiv:2603.02298 doi:10.48550/arXiv.2603.02298

[62] Hassan Chafi, Zach DeVito, Adriaan Moors, Tiark Rompf, Arvind K. Sujeeth, Pat Hanrahan, Martin Odersky, and Kunle Olukotun. 2010. Language Virtualization for Heterogeneous Parallel Computing. *SIGPLAN Not.* 45, 10 (Oct. 2010), 835–847. doi:10.1145/1932682.1869527

[63] Hassan Chafi, Arvind K. Sujeeth, Kevin J. Brown, HyoukJoong Lee, Anand R. Atreya, and Kunle Olukotun. 2011. A Domain-Specific Approach to Heterogeneous Parallelism. *ACM SIGPLAN Notices* 46, 8 (Sept. 2011), 35–46. doi:10.1145/2038037.1941561

[64] Manuel M.T. Chakravarty, Gabriele Keller, Sean Lee, Trevor L. McDonell, and Vinod Grover. 2011. Accelerating Haskell Array Codes with Multicore GPUs. In *Proceedings of the Sixth Workshop on Declarative Aspects of Multicore Programming*. ACM, Austin Texas USA, 3–14. doi:10.1145/1926354.1926358

[65] Bodhisatwa Chatterjee, Drew Zagieboylo, Sana Damani, Siva Hari, and Christos Kozyrakis. 2026. ProofWright: Towards Agentic Formal Verification of CUDA. arXiv:2511.12294 [cs.SE] doi:10.48550/arXiv.2511.12294

[66] Chun Chen, Jacqueline Chame, and Mary W. Hall. 2008. CHiLL : A Framework for Composing High-level Loop Transformations. https://api.semanticscholar.org/CorpusID:15619135

[67] Ethan Chen, Jiwon Chang, and Yuhao Zhu. 2024. CoolerSpace: A Language for Physically Correct and Computationally Efficient Color Programming. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (Oct. 2024), 846–875. doi:10.1145/3689741

[68] Hongzheng Chen, Bin Fan, Alexander Collins, Bastian Hagedorn, Evghenii Gaburov, Masahiro Masuda, Matthew Brookhart, Chris Sullivan, Jason Knight, Zhiru Zhang, and Vinod Grover. 2026. Tawa: Automatic Warp Specialization for Modern GPUs with Asynchronous References. In *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 255–267. doi:10.1109/CGO68049.2026.11394849

[69] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation* (Carlsbad, CA, USA) *(OSDI'18)*. USENIX Association, Carlsbad, CA, USA, 579–594.

[70] Wentao Chen, Jiace Zhu, Qi Fan, Yehan Ma, and An Zou. 2025. CUDA-LLM: LLMs Can Write Efficient CUDA Kernels. arXiv:2506.09092 doi:10.48550/arXiv.2506.09092

[71] Zheyuan Chen, Naomi Rehman, Guido Martínez, and Tyler Sorensen. 2026. SIMT-Step Execution: A Flexible Operational Semantics for GPU Subgroup Behavior. *Proc. ACM Program. Lang.* 10, PLDI, Article 219 (June 2026), 25 pages. doi:10.1145/3808297

[72] Wei-Fan Chiang, Ganesh Gopalakrishnan, Guodong Li, and Zvonimir Rakamarić. 2013. Formal Analysis of GPU Programs with Atomics via Conflict-Directed Delay-Bounding. In *NASA Formal Methods* (Berlin, Heidelberg, 2013). Springer, 213–228. doi:10.1007/978-3-642-38088-4_15

[73] Atharva Chougule, Alexander J. Root, Rubens Lacouture, Bobby Yan, Rohan Yadav, and Fredrik Kjolstad. 2026. Partitioning Unstructured Sparse Tensor Algebra for Load-Balanced Parallel Execution. arXiv:2604.17198 doi:10.48550/arXiv.2604.17198

[74] Ezgi Çiçek, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Jan Hoffmann. 2017. Relational Cost Analysis. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages* (Paris, France) *(POPL '17)*. Association for Computing Machinery, New York, NY, USA, 316–329. doi:10.1145/3009837.3009858

[75] Basile Clément and Albert Cohen. 2022. End-to-End Translation Validation for the Halide Language. *Proceedings of the ACM on Programming Languages* 6, OOPSLA1 (April 2022), 1–30. doi:10.1145/3527328

[76] Tiago Cogumbreiro, Julien Lange, Dennis Liew Zhen Rong, and Hannah Zicarelli. 2021. Checking Data-Race Freedom of GPU Kernels, Compositionally. In *Computer Aided Verification*. Springer International Publishing, Cham, 403–426. doi:10.1007/978-3-030-81685-8_19

[77] Murray Cole. 1991. *Algorithmic Skeletons: Structured Management of Parallel Computation.* MIT Press, Cambridge, MA, USA.

[78] Peter Collingbourne, Cristian Cadar, and Paul H.J. Kelly. 2011. Symbolic Crosschecking of Floating-Point and SIMD Code. In *Proceedings of the Sixth Conference on Computer Systems* (Salzburg, Austria) *(EuroSys '11)*. Association for

Computing Machinery, New York, NY, USA, 315–328. doi:10.1145/1966445.1966475

[79] Peter Collingbourne, Cristian Cadar, and Paul H. J. Kelly. 2012. Symbolic Testing of OpenCL Code. In *Hardware and Software: Verification and Testing*. Vol. 7261. Springer Berlin Heidelberg, Berlin, Heidelberg, 203–218. doi:10.1007/978-3-642-34188-5_18

[80] Peter Collingbourne, Alastair F. Donaldson, Jeroen Ketema, and Shaz Qadeer. 2013. Interleaving and Lock-Step Semantics for Analysis and Verification of GPU Kernels. In *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings (Lecture Notes in Computer Science, Vol. 7792)*. Springer, 270–289. doi:10.1007/978-3-642-37036-6_16

[81] Alexander Collins, Dominik Grewe, Vinod Grover, Sean Lee, and Adriana Susnea. 2014. NOVA: A Functional Language for Data Parallelism. In *Proceedings of ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming*. ACM, Edinburgh United Kingdom, 8–13. doi:10.1145/2627373.2627375

[82] Ronan Collobert, Koray Kavukcuoglu, and Clément Farabet. 2011. Torch7: A Matlab-like Environment for Machine Learning. In *NIPS Workshop BigLearn*. https://api.semanticscholar.org/CorpusID:14365368

[83] Robert L. Cook. 1984. Shade trees. *ACM SIGGRAPH Computer Graphics* 18, 3 (July 1984), 223–231. doi:10.1145/964965.808602

[84] Lucas Cordeiro and Bernd Fischer. 2011. Verifying Multi-threaded Software using SMT-based Context-Bounded Model Checking. In *Proceedings of the 33rd International Conference on Software Engineering* (Waikiki, Honolulu, HI, USA) *(ICSE '11)*. Association for Computing Machinery, New York, NY, USA, 331–340. doi:10.1145/1985793.1985839

[85] NVIDIA Corporation. 2026. Warp Shuffle Functions. https://docs.nvidia.com/cuda/cuda-c-programming-guide/#warp-shuffle-functions Accessed: 2025-07-01.

[86] Nathanaël Courant and Xavier Leroy. 2021. Verified Code Generation for the Polyhedral Model. *Proceedings of the ACM on Programming Languages* 5, POPL (Jan. 2021), 1–24. doi:10.1145/3434321

[87] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Los Angeles, California) *(POPL '77)*. Association for Computing Machinery, New York, NY, USA, 238–252. doi:10.1145/512950.512973

[88] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. 2007. Stream Fusion: from Lists to Streams to Nothing at All. *SIGPLAN Not.* 42, 9 (Oct. 2007), 315–326. doi:10.1145/1291220.1291199

[89] L. Dagum and R. Menon. 1998. OpenMP: An Industry Standard API for Shared-Memory Programming. *IEEE Computational Science and Engineering* 5, 1 (1998), 46–55. doi:10.1109/99.660313

[90] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 doi:10.48550/arXiv.2307.08691

[91] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 doi:10.48550/arXiv.2205.14135

[92] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems* (Budapest, Hungary) *(TACAS'08/ETAPS'08)*. Springer-Verlag, Berlin, Heidelberg, 337–340.

[93] Dorothy E. Denning and Peter J. Denning. 1977. Certification of Programs for Secure Information Flow. *Commun. ACM* 20, 7 (July 1977), 504–513. doi:10.1145/359636.359712

[94] Rajeshwari Devaramani, Jonathan Bentz, and Tony Scudiero. 2025. Unlock GPU Performance: Global Memory Access in CUDA. https://developer.nvidia.com/blog/unlock-gpu-performance-global-memory-access-in-cuda/ Accessed: 2026-06-05.

[95] Zachary DeVito, Niels Joubert, Francisco Palacios, Stephen Oakley, Montserrat Medina, Mike Barrientos, Erich Elsen, Frank Ham, Alex Aiken, Karthik Duraisamy, Eric Darve, Juan Alonso, and Pat Hanrahan. 2011. Liszt: A Domain Specific Language for Building Portable Mesh-based PDE Solvers. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, Seattle Washington, 1–12. doi:10.1145/2063384.2063396

[96] Zachary Devito, Michael Mara, Michael Zollhöfer, Gilbert Bernstein, Jonathan Ragan-Kelley, Christian Theobalt, Pat Hanrahan, Matthew Fisher, and Matthias Niessner. 2017. Opt: A Domain Specific Language for Non-Linear Least Squares Optimization in Graphics and Imaging. *ACM Transactions on Graphics* 36, 5 (Oct. 2017), 1–27. doi:10.1145/3132188

[97] Yaoyao Ding, Bohan Hou, Xiao Zhang, Allan Lin, Tianqi Chen, Cody Hao Yu, Yida Wang, and Gennady Pekhimenko. 2026. Tilus: A Tile-Level GPGPU Programming Language for Low-Precision Computation. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, Pittsburgh PA USA, 281–297. doi:10.1145/3760250.3762219

[98] Alastair F. Donaldson, Hugues Evrard, Andrei Lascu, and Paul Thomson. 2017. Automated Testing of Graphics Shader Compilers. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 93 (Oct. 2017), 29 pages. doi:10.1145/3133917

[99] Benjamin Driscoll, Kshitij Dubey, Anjiang Wei, Neeraj Kayal, Rahul Sharma, and Alex Aiken. 2026. Equivalence Checking of ML GPU Kernels. *Proc. ACM Program. Lang.* 10, OOPSLA2, Article 338 (Oct. 2026), 29 pages. doi:10.1145/3839470

[100] Christophe Dubach, Perry Cheng, Rodric Rabbah, David F. Bacon, and Stephen J. Fink. 2012. Compiling a High-level Language for GPUs: (via Language Support for Architectures and Compilers). In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Beijing China, 1–12. doi:10.1145/2254064.2254066

[101] Gabriel Ebner, Guido Martínez, Aseem Rastogi, Thibault Dardinier, Megan Frisella, Tahina Ramananandro, and Nikhil Swamy. 2025. PulseCore: An Impredicative Concurrent Separation Logic for Dependently Typed Programs. *Proc. ACM Program. Lang.* 9, PLDI, Article 208 (June 2025), 24 pages. doi:10.1145/3729311

[102] H. Carter Edwards, Daniel Sunderland, Chris Amsler, and Sam Mish. 2011. Multicore/GPGPU Portable Computational Kernels via Multidimensional Arrays. In *2011 IEEE International Conference on Cluster Computing*. 363–370. doi:10.1109/CLUSTER.2011.47

[103] H. Carter Edwards and Christian R. Trott. 2013. Kokkos: Enabling Performance Portability Across Manycore Architectures. In *2013 Extreme Scaling Workshop (xsw 2013)*. 18–24. doi:10.1109/XSW.2013.7

[104] Ariel Eizenberg, Yuanfeng Peng, Toma Pigli, William Mansky, and Joseph Devietti. 2017. BARRACUDA: Binary-level Analysis of Runtime RAces in CUDA Programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Barcelona Spain, 126–140. doi:10.1145/3062341.3062342

[105] Melih Elibol, Jared Roesch, Isaac Gelado, Eric Buehler, and Michael Garland. 2026. Fearless Concurrency on the GPU. arXiv:2606.15991 doi:10.48550/arXiv.2606.15991

[106] Conal Elliott. 2004. Programming Graphics Processors Functionally. In *Proceedings of the 2004 ACM SIGPLAN Workshop on Haskell*. ACM, Snowbird, Utah USA, 45–56. doi:10.1145/1017472.1017482

[107] Martin Elsman, Troels Henriksen, Danil Annenkov, and Cosmin E. Oancea. 2018. Static Interpretation of Higher-Order Modules in Futhark: Functional GPU Programming in the Large. *Proceedings of the ACM on Programming Languages* 2, ICFP (July 2018), 1–30. doi:10.1145/3236792

[108] Eunomia Labs. 2026. A Taxonomy of GPU Bugs: 19 Defect Classes for CUDA Verification. https://eunomia.dev/blog/2026/01/29/a-taxonomy-of-gpu-bugs-19-defect-classes-for-cuda-verification/ Accessed: 2026-07-22.

[109] Stephan Falke, Deepak Kapur, and Carsten Sinz. 2011. Termination Analysis of C Programs using Compiler Intermediate Languages. In *22nd International Conference on Rewriting Techniques and Applications (RTA'11)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

[110] Kayvon Fatahalian, Timothy J. Knight, Mike Houston, Mattan Erez, Daniel Reiter Horn, Larkhoon Leem, Ji Young Park, Manman Ren, Alex Aiken, William J. Dally, and Pat Hanrahan. 2006. Sequoia: Programming the Memory Hierarchy. In *SC '06: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*. 4–4. doi:10.1109/SC.2006.55

[111] Paul Feautrier. 1992. Some Efficient Solutions to the Affine Scheduling Problem: I. One-Dimensional Time. *Int. J. Parallel Program.* 21, 5 (Oct. 1992), 313–348. doi:10.1007/BF01407835

[112] Pratik Fegade, Tianqi Chen, Phillip B. Gibbons, and Todd C. Mowry. 2021. Cortex: A Compiler for Recursive Deep Learning Models. arXiv:2011.01383 doi:10.48550/arXiv.2011.01383

[113] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. 2023. TensorIR: An Abstraction for Automatic Tensorized Program Optimization. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 804–817. doi:10.1145/3575693.3576933

[114] Benjamin Ferrell, Jun Duan, and Kevin W. Hamlen. 2019. CUDA au Coq: A Framework for Machine-validating GPU Assembly Programs. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 474–479. doi:10.23919/DATE.2019.8715160 ISSN: 1558-1101.

[115] Jiří Filipovič, Matúš Madzin, Jan Fousek, and Luděk Matyska. 2015. Optimizing CUDA Code by Kernel Fusion: Application on BLAS. *J. Supercomput.* 71, 10 (Oct. 2015), 3934–3957. doi:10.1007/s11227-015-1483-z

[116] Michael J. Flynn. 1972. Some Computer Organizations and Their Effectiveness. *IEEE Trans. Comput.* C-21, 9 (1972), 948–960. doi:10.1109/TC.1972.5009071

[117] Theresa Foley and Pat Hanrahan. 2011. Spark: Modular, Composable Shaders for Graphics Hardware. *ACM Transactions on Graphics* 30, 4 (July 2011), 1–12. doi:10.1145/2010324.1965002

[118] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323 doi:10.48550/arXiv.2210.17323

[119] Roy Frostig, Matthew Johnson, and Chris Leary. 2018. Compiling machine learning programs via high-level tracing. In *SysML Conference 2018*. https://mlsys.org/Conferences/doc/2018/146.pdf

[120] Yoshihiko Futamura. 1983. Partial Computation of Programs. In *RIMS Symposia on Software Science and Engineering* (Berlin, Heidelberg, 1983). Springer, 1–35. doi:10.1007/3-540-11980-9_13

[121] Dietrich Geisler, Irene Yoon, Aditi Kabra, Horace He, Yinnon Sanders, and Adrian Sampson. 2020. Geometry Types for Graphics Programming. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (Nov. 2020), 1–25. doi:10.1145/3428241

[122] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed Automated Random Testing. *SIGPLAN Not.* 40, 6 (June 2005), 213–223. doi:10.1145/1064978.1065036

[123] Google Cloud. 2026. GPU pricing. https://cloud.google.com/products/compute/gpus-pricing Accessed: 2026-07-16.

[124] Ganesh Gopalakrishnan, Ignacio Laguna, Ang Li, Pavel Panchekha, Cindy Rubio-González, and Zachary Tatlock. 2021. Guarding Numerics Amidst Rising Heterogeneity. In *2021 IEEE/ACM 5th International Workshop on Software Correctness for HPC Applications (Correctness)*. 9–15. doi:10.1109/Correctness54621.2021.00007

[125] Clemens Grelck and Sven-Bodo Scholz. 2006. SAC—A Functional Array Language for Efficient Multi-threaded Execution. *International Journal of Parallel Programming* 34, 4 (2006), 383–427. doi:10.1007/s10766-006-0018-x

[126] Larry Gritz, Clifford Stein, Chris Kulla, and Alejandro Conty. 2010. Open Shading Language. In *ACM SIGGRAPH 2010 Talks* (Los Angeles, California) *(SIGGRAPH '10)*. Association for Computing Machinery, New York, NY, USA, Article 33, 1 pages. doi:10.1145/1837026.1837070

[127] Max Grossman, Mauricio Breternitz, and Vivek Sarkar. 2013. HadoopCL: MapReduce on Distributed Heterogeneous Platforms through Seamless Integration of Hadoop and OpenCL. In *2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and Phd Forum*. 1918–1927. doi:10.1109/IPDPSW.2013.246

[128] Yue Guan, Hongtao Yu, Peng Chen, Daohang Shi, Karthik Manivannan, Nicholas J. Riasanovsky, Manman Ren, Lei Wang, Shane Nay, Partha Kanuparthy, Zaifeng Pan, Zhengding Hu, and Yufei Ding. 2026. TLX: Hardware-Native, Evolvable MIMW GPU Compiler for Large-scale Production Environments. arXiv:2605.10905 doi:10.48550/arXiv.2605.10905

[129] Diwakar Gupta and Sabastian Mugazambi. 2026. Inside the Eighth-Generation TPU: An Architecture Deep Dive. https://cloud.google.com/blog/products/compute/tpu-8t-and-tpu-8i-technical-deep-dive Accessed: 2026-06-24.

[130] Prateek Gupta. 2025. Understanding GPU Programming with Triton: From Hardware to Code. https://www.pgupta.info/blog/2025/12/triton/ Accessed: 2026-06-24.

[131] Bastian Hagedorn, Archibald Samuel Elliott, Henrik Barthels, Rastislav Bodik, and Vinod Grover. 2020. Fireiron: A Data-Movement-Aware Scheduling Language for GPUs. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*. ACM, Virtual Event GA USA, 71–82. doi:10.1145/3410463.3414632

[132] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. 2023. Graphene: An IR for Optimized Tensor Computations on GPUs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. ACM, Vancouver BC Canada, 302–313. doi:10.1145/3582016.3582018

[133] Bastian Hagedorn and Vinod Grover. 2026. It's about Time: Temporal Abstractions for Asynchronous GPU Tensor Computations. In *Proceedings of the 35th ACM SIGPLAN International Conference on Compiler Construction* (Sydney, NSW, Australia) *(CC '26)*. Association for Computing Machinery, New York, NY, USA, 14–26. doi:10.1145/3771775.3786277

[134] Bastian Hagedorn, Johannes Lenfers, Thomas Kœhler, Xueying Qin, Sergei Gorlatch, and Michel Steuwer. 2020. Achieving High-Performance the Functional Way: A Functional Pearl on Expressing High-Performance Optimizations as Rewrite Strategies. *Proceedings of the ACM on Programming Languages* 4, ICFP (Aug. 2020), 1–29. doi:10.1145/3408974

[135] Mary Hall, Jacqueline Chame, Chun Chen, Jaewook Shin, Gabe Rudy, and Malik Murtaza Khan. 2010. Loop Transformation Recipes for Code Generation and Auto-Tuning. In *Languages and Compilers for Parallel Computing*. Springer, Berlin, Heidelberg, 50–64. doi:10.1007/978-3-642-13374-9_4

[136] Mary Hall, Cosmin E. Oancea, Anne C. Elster, Ari Rasch, Sameeran Joshi, Amir Mohammad Tavakkoli, and Richard Schulze. 2025. Scheduling Language Chronology: Past, Present, and Future. *ACM Trans. Archit. Code Optim.* 22, 3, Article 100 (Sept. 2025), 31 pages. doi:10.1145/3743135

[137] Tianyi David Han and Tarek S. Abdelrahman. 2009. *hi*CUDA: A High-level Directive-based Language for GPU Programming. In *Proceedings of 2nd Workshop on General Purpose Processing on Graphics Processing Units*. ACM, Washington D.C. USA, 52–61. doi:10.1145/1513895.1513902

[138] Pat Hanrahan and Jim Lawson. 1990. A Language for Shading and Lighting Calculations. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, Dallas TX USA, 289–298. doi:10.1145/97879.97911

[139] Mark Harris. 2012. How to Optimize Data Transfers in CUDA C/C++. https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/ Accessed: 2026-06-05.

[140] Mark Harris. 2012. How to Query Device Properties and Handle Errors in CUDA C/C++. https://developer.nvidia.com/blog/how-query-device-properties-and-handle-errors-cuda-cc/ Accessed: 2026-06-09.

[141] Mark Harris. 2013. An Efficient Matrix Transpose in CUDA C/C++. https://developer.nvidia.com/blog/efficient-matrix-transpose-cuda-cc/ Accessed: 2026-05-29.

[142] Mark Harris. 2013. Using Shared Memory in CUDA C/C++. https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/ Accessed: 2026-06-05.

[143] Mark J. Harris, Greg Coombe, Thorsten Scheuermann, and Anselmo Lastra. 2002. Physically-based Visual Simulation on Graphics Hardware. In *Graphics Hardware*.
[144] Haskell.org. 2026. Haskell Language. https://www.haskell.org/ Accessed: 2026-06-01.
[145] Akihiro Hayashi, Max Grossman, Jisheng Zhao, Jun Shirako, and Vivek Sarkar. 2014. Speculative Execution of Parallel Programs with Precise Exception Semantics on GPUs. In *Languages and Compilers for Parallel Computing* (Cham, 2014). Springer International Publishing, 342–356. doi:10.1007/978-3-319-09967-5_20
[146] Akihiro Hayashi, Max Grossman, Jisheng Zhao, Jun Shirako, and Vivek Sarkar. 2013. Accelerating Habanero-Java Programs with OpenCL Generation. In *Proceedings of the 2013 International Conference on Principles and Practices of Programming on the Java Platform: Virtual Machines, Languages, and Tools* (Stuttgart, Germany) *(PPPJ '13)*. Association for Computing Machinery, New York, NY, USA, 124–134. doi:10.1145/2500828.2500840
[147] Ari B. Hayes, Lingda Li, Mohammad Hedayati, Jiahuan He, Eddy Z. Zhang, and Kai Shen. 2017. GPU Taint Tracking. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. USENIX Association, Santa Clara, CA, 209–220. https://www.usenix.org/conference/atc17/technical-sessions/presentation/hayes
[148] Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: Language Mechanisms for Extensible Real-time Shading Systems. *ACM Trans. Graph.* 37, 4, Article 141 (July 2018), 13 pages. doi:10.1145/3197517.3201380
[149] Zijian He, Adrian Sampson, Yiying Zhang, and Zhiyuan Guo. 2026. VDCores: Resource Decoupled Programming and Execution for Asynchronous GPU. arXiv:2605.03190 doi:10.48550/arXiv.2605.03190
[150] Troels Henriksen. 2021. Bounds Checking on GPU. *International Journal of Parallel Programming* 49, 6 (Dec. 2021), 761–775. doi:10.1007/s10766-021-00703-4
[151] Troels Henriksen, Niels G. W. Serup, Martin Elsman, Fritz Henglein, and Cosmin E. Oancea. 2017. Futhark: Purely Functional GPU-Programming with Nested Parallelism and In-Place Array Updates. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Barcelona Spain, 556–571. doi:10.1145/3062341.3062354
[152] Pieter Hijma, Stijn Heldens, Alessio Sclocco, Ben Van Werkhoven, and Henri E. Bal. 2023. Optimization Techniques for GPU Programming. *Comput. Surveys* 55, 11 (Nov. 2023), 1–81. doi:10.1145/3570638
[153] Charles Antony Richard Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580.
[154] Kenneth E. Hoff, John Keyser, Ming Lin, Dinesh Manocha, and Tim Culver. 1999. Fast Computation of Generalized Voronoi Diagrams using Graphics Hardware. In *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '99)*. ACM Press/Addison-Wesley Publishing Co., USA, 277–286. doi:10.1145/311535.311567
[155] Martin Hofmann and Steffen Jost. 2003. Static Prediction of Heap Space Usage for First-Order Functional Programs. *SIGPLAN Not.* 38, 1 (Jan. 2003), 185–197. doi:10.1145/640128.604148
[156] Anup Holey, Vineeth Mekkat, and Antonia Zhai. 2013. HAccRG: Hardware-Accelerated Data Race Detection in GPUs. In *2013 42nd International Conference on Parallel Processing*. 60–69. doi:10.1109/ICPP.2013.15 ISSN: 2332-5690.
[157] Mike Houston. 2008. Anatomy of AMD's TeraScale Graphics Engine. https://web.archive.org/web/20100613174446/http://s08.idav.ucdavis.edu/houston-amd-terascale.pdf Accessed: 2026-06-01.
[158] P. Y. T. Hsu and E. S. Davidson. 1986. Highly Concurrent Scalar Processing. In *Proceedings of the 13th Annual International Symposium on Computer Architecture* (Tokyo, Japan) *(ISCA '86)*. IEEE Computer Society Press, Washington, DC, USA, 386–395.
[159] Yuanming Hu, Tzu-Mao Li, Luke Anderson, Jonathan Ragan-Kelley, and Frédo Durand. 2019. Taichi: A Language for High-Performance Computation on Spatially Sparse Data Structures. *ACM Transactions on Graphics* 38, 6 (Dec. 2019), 1–16. doi:10.1145/3355089.3356506
[160] Roger K. W. Hui and Morten J. Kromberg. 2020. APL since 1978. *Proc. ACM Program. Lang.* 4, HOPL, Article 69 (June 2020), 108 pages. doi:10.1145/3386319
[161] Mohamed Tarek Ibn Ziad, Sana Damani, Aamer Jaleel, Stephen W. Keckler, and Mark Stephenson. 2023. cuCatch: A Debugging Tool for Efficiently Catching Memory Safety Violations in CUDA Applications. *Proc. ACM Program. Lang.* 7, PLDI, Article 111 (June 2023), 24 pages. doi:10.1145/3591225
[162] Mohamed Tarek Ibn Ziad and Christos Kozyrakis. 2026. Hunting CUDA Bugs at Scale with cuFuzz. *Proc. ACM Program. Lang.* 10, OOPSLA1, Article 123 (April 2026), 28 pages. doi:10.1145/3798231
[163] Yuka Ikarashi, Gilbert Louis Bernstein, Alex Reinking, Hasan Genc, and Jonathan Ragan-Kelley. 2022. Exocompilation for Productive Programming of Hardware Accelerators. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, San Diego CA USA, 703–718. doi:10.1145/3519939.3523446
[164] Yuka Ikarashi, Kevin Qian, Samir Droubi, Alex Reinking, Gilbert Louis Bernstein, and Jonathan Ragan-Kelley. 2025. Exo 2: Growing a Scheduling Language. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, Rotterdam Netherlands, 426–444. doi:10.1145/3669940.3707218

[165] Intel Corporation. 2022. Introduction to the Xe-HPG Architecture. https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-the-xe-hpg-architecture.html Accessed: 2026-07-25.

[166] F. Irigoin and R. Triolet. 1988. Supernode Partitioning. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages - POPL '88*. ACM Press, San Diego, California, United States, 319–329. doi:10.1145/73560.73588

[167] Kazuaki Ishizaki, Akihiro Hayashi, Gita Koblents, and Vivek Sarkar. 2015. Compiling and Optimizing Java 8 Programs for GPU Execution. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. 419–431. doi:10.1109/PACT.2015.46 ISSN: 1089-795X.

[168] K.E. Iverson. 1962. *A Programming Language*. Wiley.

[169] John Jacobson, Martin Burtscher, and Ganesh Gopalakrishnan. 2024. HiRace: Accurate and Fast Data Race Checking for GPU Programs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14. doi:10.1109/SC41406.2024.00042

[170] Abhinav Jangda and Arjun Guha. 2020. Model-Based Warp Overlapped Tiling for Image Processing Programs on GPUs. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques* (Virtual Event, GA, USA) *(PACT '20)*. Association for Computing Machinery, New York, NY, USA, 317–328. doi:10.1145/3410463.3414649

[171] Aaron Jarmusch and Sunita Chandrasekaran. 2025. Microbenchmarking NVIDIA's Blackwell Architecture: An in-depth Architectural Analysis. (2025). https://arxiv.org/abs/2512.02189v3

[172] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. Caffe: Convolutional Architecture for Fast Feature Embedding. (2014). doi:10.48550/arXiv.1408.5093

[173] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scarpazza. 2018. Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking. (2018). https://arxiv.org/abs/1804.06826v1

[174] Aditya K. Kamath and Arkaprava Basu. 2021. iGUARD: In-GPU Advanced Race Detection. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles* (Virtual Event, Germany) *(SOSP '21)*. Association for Computing Machinery, New York, NY, USA, 49–65. doi:10.1145/3477132.3483545

[175] Aditya K. Kamath, Alvin A. George, and Arkaprava Basu. 2020. ScoRD: A Scoped Race Detector for GPUs. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 1036–1049. doi:10.1109/ISCA45697.2020.00088

[176] Andrew Kerr, Duane Merrill, Julien Demouth, and John Tran. 2017. CUTLASS: Fast Linear Algebra in CUDA C++. https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/ Accessed: 2026-06-23.

[177] Jeroen Ketema and Alastair F. Donaldson. 2017. Termination Analysis for GPU Kernels. *Science of Computer Programming* 148 (2017), 107–122. doi:10.1016/j.scico.2017.04.009 Special Issue on Automated Verification of Critical Systems (AVoCS 2015).

[178] L. Kettner, M. Raab, D. Seibert, J. Jordan, and A. Keller. 2015. The Material Definition Language. In *Proceedings of the Third Workshop on Material Appearance Modeling: Issues and Acquisition* (Darmstadt, Germany) *(MAM '15)*. Eurographics Association, Goslar, DEU, 1–4.

[179] Malik Khan, Protonu Basu, Gabe Rudy, Mary Hall, Chun Chen, and Jacqueline Chame. 2013. A Script-based Autotuning Compiler System to Generate High-Performance CUDA Code. *ACM Transactions on Architecture and Code Optimization* 9, 4 (Jan. 2013), 1–25. doi:10.1145/2400682.2400690

[180] Emmett Kilgariff, Henry Moreton, Nick Stam, and Brandon Bell. 2018. NVIDIA Turing Architecture In-Depth. https://developer.nvidia.com/blog/nvidia-turing-architecture-in-depth/ Accessed: 2026-08-27.

[181] Hansung Kim, Ruohan Richard Yan, Joshua You, Tieliang Vamber Yang, and Yakun Sophia Shao. 2025. Virgo: Cluster-Level Matrix Unit Integration in GPUs for Scalability and Energy Efficiency. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1382–1399. doi:10.1145/3676641.3716281

[182] James C. King. 1975. A New Approach to Program Testing. *SIGPLAN Not.* 10, 6 (April 1975), 228–233. doi:10.1145/390016.808444

[183] Fredrik Kjolstad, Willow Ahrens, Shoaib Kamil, and Saman Amarasinghe. 2019. Tensor Algebra Compilation with Workspaces. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 180–192. doi:10.1109/CGO.2019.8661185

[184] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (Oct. 2017), 1–29. doi:10.1145/3133901

[185] Fredrik Kjolstad, Shoaib Kamil, Jonathan Ragan-Kelley, David I. W. Levin, Shinjiro Sueda, Desai Chen, Etienne Vouga, Danny M. Kaufman, Gurtej Kanwar, Wojciech Matusik, and Saman Amarasinghe. 2016. Simit: A Language for Physical Simulation. *ACM Transactions on Graphics* 35, 2 (May 2016), 1–21. doi:10.1145/2866569

[186] Andreas Klöckner. 2014. Loo.py: Transformation-based Code Generation for GPUs and CPUs. In *Proceedings of ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming*. ACM, Edinburgh United Kingdom, 82–87. doi:10.1145/2627373.2627387

[187] Marcin Knap and Paweł Czarnul. 2019. Performance Evaluation of Unified Memory with Prefetching and Oversubscription for Selected Parallel CUDA Applications on NVIDIA Pascal and Volta GPUs. *The Journal of Supercomputing* 75, 11 (2019), 7625–7645. doi:10.1007/s11227-019-02966-8

[188] Thomas Kœhler, Andrés Goens, Siddharth Bhat, Tobias Grosser, Phil Trinder, and Michel Steuwer. 2024. Guided Equality Saturation. *Proc. ACM Program. Lang.* 8, POPL, Article 58 (Jan. 2024), 32 pages. doi:10.1145/3632900

[189] Kensuke Kojima and Atsushi Igarashi. 2013. A Hoare Logic for SIMT Programs. In *Programming Languages and Systems*. Springer International Publishing, Cham, 58–73. doi:10.1007/978-3-319-03542-0_5

[190] Kensuke Kojima and Atsushi Igarashi. 2017. A Hoare Logic for GPU Kernels. *ACM Transactions on Computational Logic* 18, 1 (Jan. 2017), 1–43. doi:10.1145/3001834

[191] Kensuke Kojima, Akifumi Imanishi, and Atsushi Igarashi. 2018. Automated Verification of Functional Correctness of Race-Free GPU Programs. *Journal of Automated Reasoning* 60, 3 (March 2018), 279–298. doi:10.1007/s10817-017-9428-2

[192] Hsiang Tsung Kung, Charles E. Leiserson, et al. 1979. Systolic Arrays (for VLSI). In *Sparse Matrix Proceedings 1978*, Vol. 1. SIAM Philadelphia, PA, USA, 256–282.

[193] A. Kunimatsu, N. Ide, T. Sato, Y. Endo, H. Murakami, T. Kamei, M. Hirano, F. Ishihara, H. Tago, M. Oka, A. Ohba, T. Yutaka, T. Okada, and M. Suzuoki. 2000. Vector Unit Architecture for Emotion Synthesis. *IEEE Micro* 20, 2 (2000), 40–47. doi:10.1109/40.848471

[194] Bastian Köpcke, Sergei Gorlatch, and Michel Steuwer. 2024. Descend: A Safe GPU Systems Programming Language. *Proceedings of the ACM on Programming Languages* 8, PLDI (June 2024), 841–864. doi:10.1145/3656411

[195] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, Lesheng Jin, Yaxing Cai, Ziheng Jiang, Yong Wu, Sunghyun Park, Prakalp Srivastava, Jared Roesch, Todd C. Mowry, and Tianqi Chen. 2025. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 998–1013. doi:10.1145/3676641.3716249

[196] Monica S. Lam. 2004. Software Pipelining: An Effective Scheduling Technique for VLIW Machines. *SIGPLAN Not.* 39, 4 (April 2004), 244–256. doi:10.1145/989393.989420

[197] E. Scott Larsen and David McAllister. 2001. Fast Matrix Multiplies using Graphics Hardware. In *Proceedings of the 2001 ACM/IEEE Conference on Supercomputing*. ACM, Denver Colorado, 55–55. doi:10.1145/582034.582089

[198] C. Lattner and V. Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* 75–86. doi:10.1109/CGO.2004.1281665

[199] Christian Lengauer. 1993. Loop Parallelization in the Polytope Model. In *CONCUR'93*. Springer, Berlin, Heidelberg, 398–416. doi:10.1007/3-540-57208-2_28

[200] Jed Lengyel, Mark Reichert, Bruce R. Donald, and Donald P. Greenberg. 1990. Real-time Robot Motion Planning using Rasterizing Computer Graphics Hardware. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques* (Dallas, TX, USA) *(SIGGRAPH '90)*. Association for Computing Machinery, New York, NY, USA, 327–335. doi:10.1145/97879.97915

[201] Alan Leung, Manish Gupta, Yuvraj Agarwal, Rajesh Gupta, Ranjit Jhala, and Sorin Lerner. 2012. Verifying GPU Kernels by Test Amplification. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation* (Beijing, China) *(PLDI '12)*. Association for Computing Machinery, New York, NY, USA, 383–394. doi:10.1145/2254064.2254110

[202] Alan Leung, Ondřej Lhoták, and Ghulam Lashari. 2009. Automatic Parallelization for Graphics Processing Units. In *Proceedings of the 7th International Conference on Principles and Practice of Programming in Java* (Calgary, Alberta, Canada) *(PPPJ '09)*. Association for Computing Machinery, New York, NY, USA, 91–100. doi:10.1145/1596655.1596670

[203] Guodong Li and Ganesh Gopalakrishnan. 2010. Scalable SMT-based Verification of GPU Kernel Functions. In *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Santa Fe, New Mexico, USA) *(FSE '10)*. Association for Computing Machinery, New York, NY, USA, 187–196. doi:10.1145/1882291.1882320

[204] Guodong Li and Ganesh Gopalakrishnan. 2012. Parameterized Verification of GPU Kernel Programs. In *2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*. 2450–2459. doi:10.1109/IPDPSW.2012.302

[205] Guodong Li, Peng Li, Geof Sawaya, Ganesh Gopalakrishnan, Indradeep Ghosh, and Sreeranga P. Rajan. 2012. GKLEE: Concolic Verification and Test Generation for GPUs. *ACM SIGPLAN Notices* 47, 8 (Sept. 2012), 215–224. doi:10.1145/2370036.2145844

[206] Pengcheng Li, Xiaoyu Hu, Dong Chen, Jacob Brock, Hao Luo, Eddy Z. Zhang, and Chen Ding. 2017. LD: Low-Overhead GPU Race Detection Without Access Monitoring. *ACM Trans. Archit. Code Optim.* 14, 1, Article 9 (March 2017), 25 pages.

doi:10.1145/3046678

[207] Peng Li, Guodong Li, and Ganesh Gopalakrishnan. 2012. Parametric Flows: Automated Behavior Equivalencing for Symbolic Analysis of Races in CUDA Programs. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–10. doi:10.1109/SC.2012.94 ISSN: 2167-4337.

[208] Peng Li, Guodong Li, and Ganesh Gopalakrishnan. 2014. Practical Symbolic Race Checking of GPU Programs. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 179–190. doi:10.1109/SC.2014.20 ISSN: 2167-4337.

[209] Ruihao Li, Sanjana Yadav, Qinzhe Wu, Krishna Kavi, Gayatri Mehta, Neeraja J. Yadwadkar, and Lizy K. John. 2023. Performance Implications of Async Memcpy and UVM: A Tale of Two Data Transfer Modes. In *2023 IEEE International Symposium on Workload Characterization (IISWC)*. 115–127. doi:10.1109/IISWC59245.2023.00024

[210] Tzu-Mao Li, Michaël Gharbi, Andrew Adams, Frédo Durand, and Jonathan Ragan-Kelley. 2018. Differentiable Programming for Image Processing and Deep Learning in Halide. *ACM Transactions on Graphics* 37, 4 (Aug. 2018), 1–13. doi:10.1145/3197517.3201383

[211] Wei Li, Xiaoming Wei, and Arie Kaufman. 2003. Implementing Lattice Boltzmann Computation on Graphics Hardware. *The Visual Computer* 19, 7 (2003), 444–456. doi:10.1007/s00371-003-0210-6

[212] Xinyi Li, Mark Baranowski, Harvey Dam, and Ganesh Gopalakrishnan. 2025. Array Programming on GPUs: Challenges and Opportunities. In *Proceedings of the 11th ACM SIGPLAN International Workshop on Libraries, Languages and Compilers for Array Programming* (Seoul, Republic of Korea) *(ARRAY '25)*. Association for Computing Machinery, New York, NY, USA, 41–52. doi:10.1145/3736112.3736144

[213] Dennis Liew, Tiago Cogumbreiro, and Julien Lange. 2024. Sound and Partially-Complete Static Analysis of Data-Races in GPU Programs. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (Oct. 2024), 2434–2461. doi:10.1145/3689797

[214] Amy W. Lim and Monica S. Lam. 1997. Maximizing Parallelism and Minimizing Synchronization with Affine Transforms. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Paris, France) *(POPL '97)*. Association for Computing Machinery, New York, NY, USA, 201–214. doi:10.1145/263699.263719

[215] Erik Lindholm, Mark J. Kilgard, and Henry Moreton. 2001. A User-Programmable Vertex Engine. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, 149–158. doi:10.1145/383259.383274

[216] Erik Lindholm, John Nickolls, Stuart Oberman, and John Montrym. 2008. NVIDIA Tesla: A Unified Graphics and Computing Architecture. *IEEE Micro* 28, 2 (March 2008), 39–55. doi:10.1109/MM.2008.31

[217] Amanda Liu, Gilbert Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. 2024. A Verified Compiler for a Functional Tensor Language. *Proceedings of the ACM on Programming Languages* 8, PLDI (June 2024), 320–342. doi:10.1145/3656390

[218] Amanda Liu, Gilbert Louis Bernstein, Adam Chlipala, and Jonathan Ragan-Kelley. 2022. Verified Tensor-Program Optimization via High-level Scheduling Rewrites. *Proceedings of the ACM on Programming Languages* 6, POPL (Jan. 2022), 1–28. doi:10.1145/3498717

[219] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Hongyuan Liu, Qiang Wang, and Xiaowen Chu. 2025. Dissecting the NVIDIA Hopper Architecture through Microbenchmarking and Multiple Level Analysis. (2025). https://arxiv.org/abs/2501.12084v2

[220] Daniel Lustig, Sameer Sahasrabuddhe, and Olivier Giroux. 2019. A Formal Analysis of the NVIDIA PTX Memory Consistency Model. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Providence RI USA, 257–270. doi:10.1145/3297858.3304043

[221] Weiliang Ma, Qian Xiong, Xuanhua Shi, Xiaosong Ma, Hai Jin, Haozhao Kuang, Mingyu Gao, Ye Zhang, Haichen Shen, and Weifang Hu. 2023. GZKP: A GPU Accelerated Zero-Knowledge Proof System. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 340–353. doi:10.1145/3575693.3575711

[222] Miles Macklin. 2024. Warp: Differentiable Spatial Computing for Python. In *ACM SIGGRAPH 2024 Courses*. ACM, Denver CO USA, 1–147. doi:10.1145/3664475.3664543

[223] Miles Macklin, Leopold Cambier, and Eric Shi. 2024. Introducing Tile-Based Programming in Warp 1.5.0. https://developer.nvidia.com/blog/introducing-tile-based-programming-in-warp-1-5-0/ Accessed: 2026-06-23.

[224] Geoffrey Mainland and Greg Morrisett. 2010. Nikola: Embedding Compiled GPU Functions in Haskell. In *Proceedings of the third ACM Haskell symposium on Haskell*. ACM, Baltimore Maryland USA, 67–78. doi:10.1145/1863523.1863533

[225] William R. Mark, R. Steven Glanville, Kurt Akeley, and Mark J. Kilgard. 2003. Cg: A System for Programming Graphics Hardware in a C-like Language. *ACM Transactions on Graphics* 22, 3 (July 2003), 896–907. doi:10.1145/882262.882362

[226] Stefano Markidis, Steven Wei Der Chien, Erwin Laure, Ivy Bo Peng, and Jeffrey S. Vetter. 2018. NVIDIA Tensor Core Programmability, Performance & Precision. arXiv:1803.04014 doi:10.48550/arXiv.1803.04014

[227] Guido Martínez, Bastian Köpcke, Jonáš Fiala, Gabriel Ebner, Tahina Ramananandro, Michel Steuwer, Tyler Sorensen, and Nikhil Swamy. 2026. Kuiper: Correct and Efficient GPU Programming with Dependent Types and Separation Logic.

Proc. ACM Program. Lang. 10, PLDI, Article 202 (June 2026), 25 pages. doi:10.1145/3808280
[228] Meta Platforms, Inc. 2025. Helion Documentation. https://helionlang.com/ Accessed: 2026-06-24.
[229] Microsoft. 2021. Direct3D 12 Raytracing HLSL Shaders. https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-raytracing-hlsl-shaders Accessed: 2026-08-27.
[230] Microsoft. 2021. High-level Shader Language (HLSL). https://learn.microsoft.com/en-us/windows/win32/direct3dhlsl/dx-graphics-hlsl Accessed: 2026-05-29.
[231] Sparsh Mittal, S. B. Abhinaya, Manish Reddy, and Irfan Ali. 2018. A Survey of Techniques for Improving Security of GPUs. *Journal of Hardware and Systems Security* 2, 3 (2018), 266–285. doi:10.1007/s41635-018-0039-0
[232] Modular Inc. 2023. Mojo. https://mojolang.org/ Accessed: 2026-06-22.
[233] Felipe R. Monteiro, Erickson H. da S. Alves, Isabela S. Silva, Hussama I. Ismail, Lucas C. Cordeiro, and Eddie B. de Lima Filho. 2018. ESBMC-GPU A Context-Bounded Model Checking Tool to Verify CUDA Programs. *Science of Computer Programming* 152 (2018), 63–69. doi:10.1016/j.scico.2017.09.005
[234] Ravi Teja Mullapudi, Andrew Adams, Dillon Sharlet, Jonathan Ragan-Kelley, and Kayvon Fatahalian. 2016. Automatically Scheduling Halide Image Processing Pipelines. *ACM Transactions on Graphics* 35, 4 (July 2016), 1–11. doi:10.1145/2897824.2925952
[235] Ravi Teja Mullapudi, Vinay Vasista, and Uday Bondhugula. 2015. PolyMage: Automatic Optimization for Image Processing Pipelines. *SIGARCH Comput. Archit. News* 43, 1 (March 2015), 429–443. doi:10.1145/2786763.2694364
[236] Stefan K. Muller and Jan Hoffmann. 2021. Modeling and Analyzing Evaluation Cost of CUDA Kernels. *Proceedings of the ACM on Programming Languages* 5, POPL (Jan. 2021), 1–31. doi:10.1145/3434306
[237] Stefan K. Muller and Jan Hoffmann. 2024. Modeling and Analyzing Evaluation Cost of CUDA Kernels. *ACM Trans. Parallel Comput.* 11, 1, Article 5 (March 2024), 53 pages. doi:10.1145/3639403
[238] Brad Nemire. 2015. Inside the Programming Evolution of GPU Computing. https://developer.nvidia.com/blog/inside-the-programming-evolution-of-gpu-computing/ Accessed: 2026-06-24.
[239] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. 2008. Scalable Parallel Programming with CUDA. In *ACM SIGGRAPH 2008 classes*. ACM, Los Angeles California, 1–14. doi:10.1145/1401132.1401152
[240] Tobias Nipkow, Markus Wenzel, and Lawrence C. Paulson. 2002. *Isabelle/HOL*. Lecture Notes in Computer Science, Vol. 2283. Springer Berlin Heidelberg. doi:10.1007/3-540-45949-9
[241] NVIDIA Corporation. 2016. NVIDIA Tesla P100 Whitepaper. (2016). https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf Accessed: 2026-06-01.
[242] NVIDIA Corporation. 2017. Independent Thread Scheduling. https://docs.nvidia.com/cuda/volta-tuning-guide/index.html#independent-thread-scheduling Accessed: 2026-07-21.
[243] NVIDIA Corporation. 2017. NVIDIA Tesla V100 GPU Architecture. (2017). https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf Accessed: 2026-06-01.
[244] NVIDIA Corporation. 2020. NVIDIA A100 Tensor Core GPU Architecture. (2020). https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf Accessed: 2026-06-02.
[245] NVIDIA Corporation. 2022. NVIDIA H100 Tensor Core GPU Architecture. (2022). https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c Accessed: 2026-06-01.
[246] NVIDIA Corporation. 2025. cuTile Python. https://docs.nvidia.com/cuda/cutile-python/ Accessed: 2026-05-29.
[247] NVIDIA Corporation. 2025. NVIDIA Blackwell Architecture Technical Brief. (2025). https://resources.nvidia.com/en-us-blackwell-architecture Accessed: 2026-06-01.
[248] NVIDIA Corporation. 2025. Tile IR. https://docs.nvidia.com/cuda/tile-ir/latest/ Accessed: 2026-05-29.
[249] NVIDIA Corporation. 2026. Atomic Functions. https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html#atomic-functions Accessed: 2026-06-26.
[250] NVIDIA Corporation. 2026. Coalesced Access to Global Memory. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/#coalesced-access-to-global-memory Accessed: 2026-06-01.
[251] NVIDIA Corporation. 2026. Compute Sanitizer. https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html Accessed: 2026-06-09.
[252] NVIDIA Corporation. 2026. cuBLAS. https://docs.nvidia.com/cuda/cublas/index.html Accessed: 2025-06-22.
[253] NVIDIA Corporation. 2026. cuDNN. https://docs.nvidia.com/deeplearning/cudnn/latest/ Accessed: 2025-06-23.
[254] NVIDIA Corporation. 2026. CuTe DSL. https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl.html Accessed: 2026-06-23.
[255] NVIDIA Corporation. 2026. CuTe Layouts. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/cute/01_layout.html Accessed: 2026-05-29.
[256] NVIDIA Corporation. 2026. Independent Thread Scheduling. https://docs.nvidia.com/cuda/cuda-programming-guide/03-advanced/advanced-kernel-programming.html#advanced-kernel-programming Accessed: 2026-06-02.
[257] NVIDIA Corporation. 2026. NVIDIA Nsight Compute. https://developer.nvidia.com/nsight-compute Accessed: 2026-09-02.

[258] NVIDIA Corporation. 2026. NVIDIA Tensor Cores. https://www.nvidia.com/en-us/data-center/tensor-cores/ Accessed: 2026-06-02.
[259] NVIDIA Corporation. 2026. Occupancy. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/#occupancy Accessed: 2026-06-01.
[260] NVIDIA Corporation. 2026. Parallel Thread Execution ISA Version 9.3. https://docs.nvidia.com/cuda/parallel-thread-execution/ Accessed: 2026-09-02.
[261] NVIDIA Corporation. 2026. Parallel Thread Execution (PTX) ISA — Asynchronous Warpgroup Level Matrix Multiply-Accumulate Instructions. https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=tcgen05%2520cp#asynchronous-warpgroup-level-matrix-instructions. Accessed: 2026-06-01.
[262] NVIDIA Corporation. 2026. Parallel Thread Execution (PTX) ISA — TensorCore 5th Generation Instructions (tcgen05). https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=tcgen05%2520cp#tensorcore-5th-generation-instructions Accessed: 2026-06-01.
[263] NVIDIA Corporation. 2026. Parallel Thread Execution (PTX) ISA — Warp Level Matrix Multiply-Accumulate Instructions. https://docs.nvidia.com/cuda/parallel-thread-execution/index.html?highlight=tcgen05%2520cp#warp-level-matrix-instructions. Accessed: 2026-06-01.
[264] NVIDIA Corporation. 2026. Shared Memory and Memory Banks. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/#shared-memory-and-memory-banks Accessed: 2026-06-01.
[265] NVIDIA Corporation. 2026. Tensor Memory Accelerator. (2026). https://docs.nvidia.com/cuda/hopper-tuning-guide/index.html#tensor-memory-accelerator Accessed: 2026-06-01.
[266] NVIDIA Corporation. 2026. Unified Memory. https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/unified-memory.html Accessed: 2026-06-10.
[267] Kyoung-Su Oh and Keechul Jung. 2004. GPU implementation of neural networks. *Pattern Recognition* 37, 6 (2004), 1311–1314. doi:10.1016/j.patcog.2004.01.013
[268] Travis E. Oliphant. 2006. *Guide to Numpy.*
[269] Peter W. O'Hearn. 2007. Resources, Concurrency, and Local Reasoning. *Theoretical Computer Science* 375, 1 (2007), 271–307. doi:10.1016/j.tcs.2006.12.035 Festschrift for John C. Reynolds's 70th birthday.
[270] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Trans. Graph.* 29, 4, Article 66 (July 2010), 13 pages. doi:10.1145/1778765.1778803
[271] Steven G. Parker, Solomon Boulos, James Bigler, and Austin Robison. 2007. RTSL: A Ray Tracing Shading Language. In *2007 IEEE Symposium on Interactive Ray Tracing*. 149–160. doi:10.1109/RT.2007.4342603
[272] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703 doi:10.48550/arXiv.1912.01703
[273] Adam Paszke, Daniel D. Johnson, David Duvenaud, Dimitrios Vytiniotis, Alexey Radul, Matthew J. Johnson, Jonathan Ragan-Kelley, and Dougal Maclaurin. 2021. Getting to the Point: Index Sets and Parallelism-Preserving Autodiff for Pointful Array Programming. *Proc. ACM Program. Lang.* 5, ICFP, Article 88 (Aug. 2021), 29 pages. doi:10.1145/3473593
[274] Manos Pavlidakis, Giorgos Vasiliadis, Stelios Mavridis, Anargyros Argyros, Antony Chazapis, and Angelos Bilas. 2024. Guardian: Safe GPU Sharing in Multi-Tenant Environments. (2024). https://arxiv.org/abs/2401.09290v2
[275] Yuanfeng Peng, Vinod Grover, and Joseph Devietti. 2018. CURD: A Dynamic CUDA Race Detector. *ACM SIGPLAN Notices* 53, 4 (Dec. 2018), 390–403. doi:10.1145/3296979.3192368
[276] Arsène Pérard-Gayot, Richard Membarth, Roland Leißa, Sebastian Hack, and Philipp Slusallek. 2019. Rodent: Generating Renderers Without Writing a Generator. *ACM Trans. Graph.* 38, 4, Article 40 (July 2019), 12 pages. doi:10.1145/3306346.3322955
[277] Phillipe Pereira, Higo Albuquerque, Hendrio Marques, Isabela Silva, Celso Carvalho, Lucas Cordeiro, Vanessa Santos, and Ricardo Ferreira. 2016. Verifying CUDA Programs using SMT-Based Context-Bounded Model Checking. In *Proceedings of the 31st Annual ACM Symposium on Applied Computing*. ACM, Pisa Italy, 1648–1653. doi:10.1145/2851613.2851830
[278] Ken Perlin. 1985. An Image Synthesizer. *ACM SIGGRAPH Computer Graphics* 19, 3 (July 1985), 287–296. doi:10.1145/325165.325247
[279] Phitchaya Mangpo Phothilimthana, Jason Ansel, Jonathan Ragan-Kelley, and Saman Amarasinghe. 2013. Portable Performance on Heterogeneous Architectures. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, Houston Texas USA, 431–444. doi:10.1145/2451116.2451162
[280] Roberto Di Pietro, Flavio Lombardi, and Antonio Villani. 2016. CUDA Leaks: A Detailed Hack for CUDA and a (Partial) Fix. *ACM Trans. Embed. Comput. Syst.* 15, 1, Article 15 (Jan. 2016), 25 pages. doi:10.1145/2801153

[281] Philip C. Pratt-Szeliga, James W. Fawcett, and Roy D. Welch. 2012. Rootbeer: Seamlessly Using GPUs from Java. In *2012 IEEE 14th International Conference on High Performance Computing and Communication & 2012 IEEE 9th International Conference on Embedded Software and Systems*. 375–380. doi:10.1109/HPCC.2012.57

[282] James Price and Simon McIntosh-Smith. 2015. Oclgrind: An Extensible OpenCL Device Simulator. In *Proceedings of the 3rd International Workshop on OpenCL* (Palo Alto, California) *(IWOCL '15)*. Association for Computing Machinery, New York, NY, USA, Article 12, 7 pages. doi:10.1145/2791321.2791333

[283] Kekoa Proudfoot, William R. Mark, Svetoslav Tzvetkov, and Pat Hanrahan. 2001. A Real-time Procedural Shading System for Programmable Graphics Hardware. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. ACM, 159–170. doi:10.1145/383259.383275

[284] Python Software Foundation. 2026. Python. https://www.python.org/ Accessed: 2026-06-20.

[285] PyTorch Team. 2025. Helion: A High-level DSL for Performant and Portable ML Kernels. https://pytorch.org/blog/helion/ Accessed: 2026-05-29.

[286] Xueying Qin, Liam O'Connor, Rob van Glabbeek, Peter Höfner, Ohad Kammar, and Michel Steuwer. 2024. Shoggoth: A Formal Foundation for Strategic Rewriting. *Proc. ACM Program. Lang.* 8, POPL, Article 3 (Jan. 2024), 29 pages. doi:10.1145/3633211

[287] Jonathan Ragan-Kelley. 2024. The Future of Fast Code: Giving Hardware What It Wants. https://www.youtube.com/watch?v=vU3ryvZYlkk Accessed: 2026-06-01.

[288] Jonathan Ragan-Kelley, Andrew Adams, Sylvain Paris, Marc Levoy, Saman Amarasinghe, and Frédo Durand. 2012. Decoupling Algorithms from Schedules for Easy Optimization of Image Processing Pipelines. *ACM Transactions on Graphics* 31, 4 (Aug. 2012), 1–12. doi:10.1145/2185520.2185528

[289] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. *SIGPLAN Not.* 48, 6 (June 2013), 519–530. doi:10.1145/2499370.2462176

[290] B. Ramakrishna Rau. 1994. Iterative Modulo Scheduling: An Algorithm for Software Pipelining Loops. In *Proceedings of the 27th Annual International Symposium on Microarchitecture* (San Jose, California, USA) *(MICRO 27)*. Association for Computing Machinery, New York, NY, USA, 63–74. doi:10.1145/192724.192731

[291] B. R. Rau and C. D. Glaeser. 1981. Some Scheduling Techniques and an Easily Schedulable Horizontal Architecture for High Performance Scientific Computing. In *Proceedings of the 14th Annual Workshop on Microprogramming* (Chatham, Massachusetts, USA) *(MICRO 14)*. IEEE Press, 183–198.

[292] Mahesh Ravishankar, Justin Holewinski, and Vinod Grover. 2015. Forma: A DSL for Image Processing Applications to Target GPUs and Multi-core CPUs. In *Proceedings of the 8th Workshop on General Purpose Processing Using GPUs* (San Francisco, CA, USA) *(GPGPU-8)*. Association for Computing Machinery, New York, NY, USA, 109–120. doi:10.1145/2716282.2716290

[293] Prashant Singh Rawat, Miheer Vaidya, Aravind Sukumaran-Rajam, Mahesh Ravishankar, Vinod Grover, Atanas Rountev, Louis-Noël Pouchet, and P. Sadayappan. 2018. Domain-Specific Optimization and Generation of High-Performance GPU Code for Stencil Computations. *Proc. IEEE* 106, 11 (2018), 1902–1920. doi:10.1109/JPROC.2018.2862896

[294] Alex Reinking, Gilbert Louis Bernstein, and Jonathan Ragan-Kelley. 2022. Formal Semantics for the Halide Language. doi:10.48550/arXiv.2210.15740 arXiv:2210.15740.

[295] Manman Ren, Nick Riasanovsky, Neil Dhar, Hongtao Yu, Jie Liu, Partha Kanuparthy, and Shane Nay. 2026. Warp Specialization in Triton: Design and Roadmap. https://pytorch.org/blog/warp-specialization-in-triton-design-and-roadmap/ Accessed: 2026-06-24.

[296] John C. Reynolds. 2002. Separation Logic: A Logic for Shared Mutable Data Structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS '02)*. IEEE Computer Society, USA, 55–74.

[297] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. Relay: A New IR for Machine Learning Frameworks. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Philadelphia, PA, USA) *(MAPL 2018)*. Association for Computing Machinery, New York, NY, USA, 58–68. doi:10.1145/3211346.3211348

[298] Randi J. Rost. 2004. *OpenGL Shading Language*. Addison-Wesley Professional.

[299] Gabe Rudy, Malik Murtaza Khan, Mary Hall, Chun Chen, and Jacqueline Chame. 2011. A Programming Language Interface to Describe Transformations and Code Generation. In *Languages and Compilers for Parallel Computing*. Springer, Berlin, Heidelberg, 136–150. doi:10.1007/978-3-642-19595-2_10

[300] Robert Schenck, Nikolaj Hey Hinnerskov, Troels Henriksen, Magnus Madsen, and Martin Elsman. 2024. AUTOMAP: Inferring Rank-Polymorphic Function Applications with Integer Linear Programming. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (Oct. 2024), 1787–1813. doi:10.1145/3689774

[301] Sven-Bodo Scholz. 2003. Single Assignment C: Efficient Support for High-level Array Operations in a Functional Setting. *J. Funct. Program.* 13, 6 (Nov. 2003), 1005–1059. doi:10.1017/S0956796802004458

[302] Fabian Schuetze. 2024. Reverse-Engineering cuBLAS. https://accu.org/journals/overload/32/181/schuetze/ Accessed: 2025-06-22.

[303] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: A Concolic Unit Testing Engine for C. *SIGSOFT Softw. Eng. Notes* 30, 5 (Sept. 2005), 263–272. doi:10.1145/1095430.1081750

[304] Ryan Senanayake, Changwan Hong, Ziheng Wang, Amalee Wilson, Stephen Chou, Shoaib Kamil, Saman Amarasinghe, and Fredrik Kjolstad. 2020. A Sparse Iteration Space Transformation Framework for Sparse Tensor Algebra. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 158 (Nov. 2020), 30 pages. doi:10.1145/3428226

[305] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision. arXiv:2407.08608 doi:10.48550/arXiv.2407.08608

[306] Rahul Sharma, Michael Bauer, and Alex Aiken. 2015. Verification of Producer-Consumer Synchronization in GPU Programs. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Portland, OR, USA) *(PLDI '15)*. Association for Computing Machinery, New York, NY, USA, 88–98. doi:10.1145/2737924.2737962

[307] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake Hechtman. 2018. Mesh-TensorFlow: Deep Learning for Supercomputers. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 10435–10444.

[308] Stephen F. Siegel, Manchun Zheng, Ziqing Luo, Timothy K. Zirkel, Andre V. Marianiello, John G. Edenhofner, Matthew B. Dwyer, and Michael S. Rogers. 2015. CIVL: The Concurrency Intermediate Verification Language. In *SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–12. doi:10.1145/2807591.2807635

[309] Vivek Singh. 2024. Game-Changer: How the World's First GPU Leveled Up Gaming and Ignited the AI Era. https://blogs.nvidia.com/blog/first-gpu-gaming-ai/ Accessed: 2026-07-07.

[310] Savvas Sioutas, Sander Stuijk, Twan Basten, Henk Corporaal, and Lou Somers. 2020. Schedule Synthesis for Halide Pipelines on GPUs. *ACM Trans. Archit. Code Optim.* 17, 3, Article 23 (Aug. 2020), 25 pages. doi:10.1145/3406117

[311] Savvas Sioutas, Sander Stuijk, Luc Waeijen, Twan Basten, Henk Corporaal, and Lou Somers. 2019. Schedule Synthesis for Halide Pipelines through Reuse Analysis. *ACM Trans. Archit. Code Optim.* 16, 2, Article 10 (April 2019), 22 pages. doi:10.1145/3310248

[312] Gina Sitaraman, Noel Chalmers, Nicholas Malaya, Damon McDougall, Ossan O'Reilly, Rene Van Oostrum, and Joseph Greathouse. 2023. AMD matrix cores. https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-readme/ Accessed: 2026-07-25.

[313] Rupanshu Soi, Rohan Yadav, Fredrik Kjolstad, Alex Aiken, Maryam Mehri Dehnavi, Michael Garland, and Michael Bauer. 2026. Optimal Software Pipelining and Warp Specialization for Tensor Core GPUs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1875–1890. https://www.usenix.org/conference/osdi26/presentation/soi

[314] Tyler Sorensen and Alastair F. Donaldson. 2016. Exposing Errors Related to Weak Memory in GPU Applications. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Santa Barbara CA USA, 100–113. doi:10.1145/2908080.2908114

[315] Tyler Sorensen, Lucas F. Salvador, Harmit Raval, Hugues Evrard, John Wickerson, Margaret Martonosi, and Alastair F. Donaldson. 2021. Specifying and Testing GPU Workgroup Progress Models. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 131 (Oct. 2021), 30 pages. doi:10.1145/3485508

[316] Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré. 2024. ThunderKittens: Simple, Fast, and Adorable AI Kernels. doi:10.48550/arXiv.2410.20399 arXiv:2410.20399.

[317] Stack Exchange Inc. 2026. Stack Overflow. https://stackoverflow.co/ Accessed: 2026-06-08.

[318] Mark Stephenson, Sana Damani, Mohamed Tarek Ibn Ziad, Anis Ladram, and Michael Garland. 2026. SuperCollider: Scalable and Effective Data Race Detection for CUDA. *Proc. ACM Program. Lang.* 10, PLDI, Article 261 (June 2026), 25 pages. doi:10.1145/3808339

[319] Michel Steuwer, Christian Fensch, Sam Lindley, and Christophe Dubach. 2015. Generating Performance Portable Code using Rewrite Rules: From High-level Functional Expressions to High-Performance OpenCL Code. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming*. ACM, Vancouver BC Canada, 205–217. doi:10.1145/2784731.2784754

[320] Michel Steuwer, Philipp Kegel, and Sergei Gorlatch. 2011. SkelCL - A Portable Skeleton Library for High-Level GPU Programming. In *2011 IEEE International Symposium on Parallel and Distributed Processing Workshops and PhD Forum*. 1176–1182. doi:10.1109/IPDPS.2011.269

[321] Michel Steuwer, Toomas Remmelg, and Christophe Dubach. 2017. LIFT: A Functional Data-Parallel IR for High-Performance GPU Code Generation. In *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 74–85. doi:10.1109/CGO.2017.7863730

[322] John E. Stone, David Gohara, and Guochun Shi. 2010. OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems. *Computing in Science & Engineering* 12, 3 (May 2010), 66–73. doi:10.1109/MCSE.2010.69

[323] Arvind K. Sujeeth, Kevin J. Brown, Hyoukjoong Lee, Tiark Rompf, Hassan Chafi, Martin Odersky, and Kunle Olukotun. 2014. Delite: A Compiler Architecture for Performance-Oriented Embedded Domain-Specific Languages. *ACM Transactions on Embedded Computing Systems* 13, 4s (July 2014), 1–25. doi:10.1145/2584665

[324] Arvind K. Sujeeth, HyoukJoong Lee, Kevin J. Brown, Hassan Chafi, Michael Wu, Anand R. Atreya, Kunle Olukotun, Tiark Rompf, and Martin Odersky. 2011. OptiML: An Implicitly Parallel Domain-Specific Language for Machine Learning. In *Proceedings of the 28th International Conference on International Conference on Machine Learning* (Bellevue, Washington, USA) *(ICML'11)*. Omnipress, Madison, WI, USA, 609–616.

[325] Shiv Sundram, Akhilesh Balasingam, Thierry Tambe, Kunle Olukotun, and Fredrik Kjolstad. 2026. Cyclotron: The Streaming Multiprocessor Abstraction is Broken. Pittsburgh, Pennsylvania, 3. https://capra.cs.cornell.edu/latte26/paper/latte26-final28.pdf

[326] Joel Svensson, Koen Claessen, and Mary Sheeran. 2010. GPGPU Kernel Implementation and Refinement using Obsidian. *Procedia Computer Science* 1, 1 (2010), 2065–2074. doi:10.1016/j.procs.2010.04.231 ICCS 2010.

[327] Joel Svensson, Mary Sheeran, and Koen Claessen. 2008. Obsidian: A Domain Specific Embedded Language for Parallel Programming of Graphics Processors. In *Implementation and Application of Functional Languages*. Springer, Berlin, Heidelberg, 156–173. doi:10.1007/978-3-642-24452-0_9

[328] Nikhil Swamy, Juan Chen, Cédric Fournet, Pierre-Yves Strub, Karthikeyan Bhargavan, and Jean Yang. 2011. Secure Distributed Programming with Value-Dependent Types. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming* (Tokyo, Japan) *(ICFP '11)*. Association for Computing Machinery, New York, NY, USA, 266–278. doi:10.1145/2034773.2034811

[329] Ali Taha, Jiexiang Liu, and Hengjie Wang. 2025. Matrix Multiplication on Blackwell. https://www.modular.com/matrix-multiplication-on-blackwell. Accessed: 2026-06-03.

[330] Taichi Lang. 2022. Taichi: High-Performance Parallel Programming in Python. https://www.taichi-lang.org/ Accessed: 2026-07-28.

[331] David Tarditi, Sidd Puri, and Jose Oglesby. 2006. Accelerator: Using Data Parallelism to Program GPUs for General-Purpose Uses. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, San Jose California USA, 325–335. doi:10.1145/1168857.1168898

[332] Rocq Team. 2026. The Rocq Prover. https://rocq-prover.org/ Accessed: 2026-05-29.

[333] The Agda Team. 2026. Agda. https://agda.readthedocs.io/en/latest/index.html Accessed: 2026-06-01.

[334] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Aditya Atluri, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. 2023. *CUTLASS*. https://github.com/NVIDIA/cutlass

[335] Arun Thangamani, Vincent Loechner, and Stéphane Genaud. 2024. A Survey of General-Purpose Polyhedral Compilers. *ACM Trans. Archit. Code Optim.* 21, 4, Article 72 (Nov. 2024), 26 pages. doi:10.1145/3674735

[336] The Halide Team. 2026. Halide: A Language for Fast, Portable Computation on Images and Tensors. https://halide-lang.org/#gettingstarted Accessed: 2026-06-20.

[337] The JAX Authors. 2024. Pallas: a JAX kernel language. https://docs.jax.dev/en/latest/pallas/index.html. Accessed: 2026-09-02.

[338] The Khronos Group. 2026. SPIR: The Standard IR for Parallel Compute and Graphics. https://www.khronos.org/spirv/ Accessed: 2026-06-09.

[339] The MathWorks, Inc. 2026. MATLAB. https://www.mathworks.com/products/matlab.html Accessed: 2026-06-24.

[340] The Rust Team. 2026. Rust. https://rust-lang.org/ Accessed: 2026-06-01.

[341] William Thies, Michal Karczmarek, and Saman P. Amarasinghe. 2002. StreamIt: A Language for Streaming Applications. In *Proceedings of the 11th International Conference on Compiler Construction (CC '02)*. Springer-Verlag, Berlin, Heidelberg, 179–196.

[342] C.J. Thompson, Sahngyun Hahn, and M. Oskin. 2002. Using Modern Graphics Architectures for General-Purpose Computing: A Framework and Analysis. In *35th Annual IEEE/ACM International Symposium on Microarchitecture, 2002. (MICRO-35). Proceedings.* 306–317. doi:10.1109/MICRO.2002.1176259

[343] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*. ACM, Phoenix AZ USA, 10–19. doi:10.1145/3315508.3329973

[344] Seiya Tokui, Ryosuke Okuta, Takuya Akiba, Yusuke Niitani, Toru Ogawa, Shunta Saito, Shuji Suzuki, Kota Uenishi, Brian Vogel, and Hiroyuki Yamazaki Vincent. 2019. Chainer: A Deep Learning Framework for Accelerating the Research Cycle. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) *(KDD '19)*. Association for Computing Machinery, New York, NY, USA, 2002–2011. doi:10.1145/3292500.3330756

[345] Haining Tong, Natalia Gavrilenko, Hernan Ponce De Leon, and Keijo Heljanko. 2024. Towards Unified Analysis of GPU Consistency. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. ACM, Hilton La Jolla Torrey Pines La Jolla CA USA, 329–344. doi:10.1145/3622781.3674174

[346] triton-lang. 2025. "01-intro.py" – Introduction to Gluon tutorial in Triton. https://github.com/triton-lang/triton/blob/main/python/tutorials/gluon/01-intro.py. Accessed: 2025-06-23.

[347] Leonard Truong, Rajkishore Barik, Ehsan Totoni, Hai Liu, Chick Markley, Armando Fox, and Tatiana Shpeisman. 2016. Latte: A Language, Compiler, and Runtime for Elegant and Efficient Deep Neural Networks. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Santa Barbara CA USA, 209–223. doi:10.1145/2908080.2908105

[348] Abhishek Udupa, R. Govindarajan, and Matthew J. Thazhuthaveetil. 2009. Software Pipelined Execution of Stream Programs on GPUs. In *2009 International Symposium on Code Generation and Optimization*. 200–209. doi:10.1109/CGO.2009.20

[349] Sain-Zee Ueng, Melvin Lathara, Sara S. Baghsorkhi, and Wen-mei W. Hwu. 2008. CUDA-Lite: Reducing GPU Programming Complexity. In *Languages and Compilers for Parallel Computing*. Springer, Berlin, Heidelberg, 1–15. doi:10.1007/978-3-540-89740-8_1

[350] Swapneela Unkule, Christopher Shaltz, and Apan Qasem. 2012. Automatic Restructuring of GPU Kernels for Exploiting Inter-thread Data Locality. In *Compiler Construction*. Springer, Berlin, Heidelberg, 21–40. doi:10.1007/978-3-642-28652-0_2

[351] Benjamin Valpey, Xinyi Li, Sreepathi Pai, and Ganesh Gopalakrishnan. 2025. An SMT Formalization of Mixed-Precision Matrix Multiplication. In *NASA Formal Methods* (Cham, 2025). Springer Nature Switzerland, 360–379. doi:10.1007/978-3-031-93706-4_21

[352] Lars B. van den Haak, Anton Wijs, Mark van den Brand, and Marieke Huisman. 2020. Formal Methods for GPGPU Programming: Is the Demand Met?. In *Integrated Formal Methods* (Cham). Springer International Publishing, 160–177. doi:10.1007/978-3-030-63461-2_9

[353] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary Devito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2019. The Next 700 Accelerated Layers: From Mathematical Expressions of Network Computation Graphs to Accelerated GPU Kernels, Automatically. *ACM Transactions on Architecture and Code Optimization* 16, 4 (Dec. 2019), 1–26. doi:10.1145/3355606

[354] Giorgos Vasiliadis, Elias Athanasopoulos, Michalis Polychronakis, and Sotiris Ioannidis. 2014. PixelVault: Using GPUs for Securing Cryptographic Operations. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security* (Scottsdale, Arizona, USA) *(CCS '14)*. Association for Computing Machinery, New York, NY, USA, 1131–1142. doi:10.1145/2660267.2660316

[355] Eelco Visser, Zine-el-Abidine Benaissa, and Andrew Tolmach. 1998. Building Program Optimizers with Rewriting Strategies. *SIGPLAN Not.* 34, 1 (Sept. 1998), 13–26. doi:10.1145/291251.289425

[356] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. 2025. TileLang: A Composable Tiled Programming Model for AI Systems. arXiv:2504.17577 doi:10.48550/arXiv.2504.17577

[357] Qifan Wang and David Oswald. 2026. Confidential Computing on Heterogeneous CPU-GPU Systems: Survey and Future Directions. *ACM Comput. Surv.* 58, 9, Article 230 (Feb. 2026), 35 pages. doi:10.1145/3793532

[358] Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. 2025. Astra: A Multi-Agent System for GPU Kernel Performance Optimization. arXiv:2509.07506 doi:10.48550/arXiv.2509.07506

[359] R. Clint Whaley and Jack J. Dongarra. 1998. Automatically Tuned Linear Algebra Software. In *Proceedings of the 1998 ACM/IEEE Conference on Supercomputing* (San Jose, CA) *(SC '98)*. IEEE Computer Society, USA, 1–27.

[360] R. Clint Whaley, Antoine Petitet, and Jack J. Dongarra. 2001. Automated Empirical Optimizations of Software and the ATLAS Project. *Parallel Comput.* 27, 1 (2001), 3–35. doi:10.1016/S0167-8191(00)00087-9 New Trends in High Performance Computing.

[361] Turner Whitted. 1980. An Improved Illumination Model for Shaded Display. *Commun. ACM* 23, 6 (June 1980), 343–349. doi:10.1145/358876.358882

[362] Sandra Wienke, Paul Springer, Christian Terboven, and Dieter an Mey. 2012. OpenACC — First Experiences with Real-World Applications. In *Euro-Par 2012 Parallel Processing* (Berlin, Heidelberg, 2012). Springer, 859–870. doi:10.1007/978-3-642-32820-6_85

[363] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (April 2009), 65–76. doi:10.1145/1498765.1498785

[364] Nicholas Wilt. 2013. Independent Thread Scheduling. In *The CUDA Handbook*. https://www.cudahandbook.com/book/ch7/independent-thread-scheduling Accessed: 2026-07-16.

[365] Mingyuan Wu, Yicheng Ouyang, Husheng Zhou, Lingming Zhang, Cong Liu, and Yuqun Zhang. 2020. Simulee: Detecting CUDA Synchronization Bugs via Memory-Access Modeling. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. 937–948. doi:10.1145/3377811.3380358

[366] Yue Xing, Bo-Yuan Huang, Aarti Gupta, and Sharad Malik. 2018. A Formal Instruction-level GPU Model for Scalable Verification. In *Proceedings of the International Conference on Computer-Aided Design* (San Diego, California) *(ICCAD '18)*. Association for Computing Machinery, New York, NY, USA, Article 130, 8 pages. doi:10.1145/3240765.3240771

[367] Divakar Kumar Yadav, Tian Zhao, and Deepak Kumar. 2026. Evaluating CUDA Tile for AI Workloads on Hopper and Blackwell GPUs. arXiv:2604.23466 doi:10.48550/arXiv.2604.23466

[368] Rohan Yadav, Alex Aiken, and Fredrik Kjolstad. 2022. DISTAL: The Distributed Tensor Algebra Compiler. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, San Diego CA USA, 286–300. doi:10.1145/3519939.3523437

[369] Rohan Yadav, Michael Garland, Alex Aiken, and Michael Bauer. 2025. Task-Based Tensor Computations on Modern GPUs. *Proceedings of the ACM on Programming Languages* 9, PLDI (June 2025), 396–420. doi:10.1145/3729262

[370] Yonghong Yan, Max Grossman, and Vivek Sarkar. 2009. JCUDA: A Programmer-Friendly Interface for Accelerating Java Programs with CUDA. In *Euro-Par 2009 Parallel Processing* (Berlin, Heidelberg, 2009). Springer, 887–899. doi:10.1007/978-3-642-03869-3_82

[371] Tyler Yandrofski, Jingyuan Chen, Nathan Otterness, James H. Anderson, and F. Donelson Smith. 2022. Making Powerful Enemies on NVIDIA GPUs. In *2022 IEEE Real-Time Systems Symposium (RTSS)*. 383–395. doi:10.1109/RTSS55097.2022.00040

[372] Wenhua Yang, Chong Zhang, and Minxue Pan. 2023. Understanding the Topics and Challenges of GPU Programming by Classifying and Analyzing Stack Overflow Posts. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1444–1456. doi:10.1145/3611643.3616365

[373] Yi Yang, Ping Xiang, Jingfei Kong, and Huiyang Zhou. 2010. A GPGPU Compiler for Memory Optimization and Parallelism Management. In *Proceedings of the 31st ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Toronto Ontario Canada, 86–97. doi:10.1145/1806596.1806606

[374] Chang Yu, Yi Xu, Ye Kuang, Yuanming Hu, and Tiantian Liu. 2022. MeshTaichi: A Compiler for Efficient Mesh-Based Operations. *ACM Transactions on Graphics* 41, 6 (Dec. 2022), 1–17. doi:10.1145/3550454.3555430

[375] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. 2026. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. arXiv:2603.05451 doi:10.48550/arXiv.2603.05451

[376] Wojciech Zaremba, Yuan Lin, and Vinod Grover. 2012. JaBEE: Framework for Object-Oriented Java Bytecode Compilation and Execution on Graphics Processor Units. In *Proceedings of the 5th Annual Workshop on General Purpose Processing with Graphics Processing Units* (London, United Kingdom) *(GPGPU-5)*. Association for Computing Machinery, New York, NY, USA, 74–83. doi:10.1145/2159430.2159439

[377] An Qi Zhang, Andrés Goens, Daniel Sorin, and Vijay Nagarajan. 2026. A Formally Verified Foundation for Compositional Heterogeneous Coherence. *Proc. ACM Program. Lang.* 10, PLDI, Article 272 (June 2026), 25 pages. doi:10.1145/3808350

[378] Yihong Zhang, Derek Gerstmann, Andrew Adams, and Maaz Bin Safeer Ahmad. 2026. Pushing Tensor Accelerators Beyond MatMul in a User-Schedulable Language. In *2026 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 242–254. doi:10.1109/CGO68049.2026.11395214

[379] Yongpeng Zhang and Frank Mueller. 2013. Hidp: A Hierarchical Data Parallel Language. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 1–11. doi:10.1109/CGO.2013.6494994

[380] Mai Zheng, Vignesh T. Ravi, Feng Qin, and Gagan Agrawal. 2011. GRace: A Low-Overhead Mechanism for Detecting Data Races in GPU Programs. In *Proceedings of the 16th ACM symposium on Principles and Practice of Parallel Programming*. ACM, San Antonio TX USA, 135–146. doi:10.1145/1941553.1941574

[381] Mai Zheng, Vignesh T. Ravi, Feng Qin, and Gagan Agrawal. 2014. GMRace: Detecting Data Races in GPU Programs via a Low-Overhead Scheme. *IEEE Transactions on Parallel and Distributed Systems* 25, 1 (Jan. 2014), 104–115. doi:10.1109/TPDS.2013.44

[382] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Trans. Graph.* 41, 6, Article 232 (Nov. 2022), 19 pages. doi:10.1145/3550454.3555463

[383] Keren Zhou, Mario Lezcano, Adam Goucher, Akhmed Rakhmati, Jeff Niu, Justin Lebar, Pawel Szczerbuk, Peter Bell, Phil Tillet, Thomas Raoux, and Zahi Moudallal. 2026. Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$. arXiv:2505.23819 doi:10.48550/arXiv.2505.23819

[384] École Polytechnique Fédérale Lausanne (EPFL). 2026. Scala. https://www.scala-lang.org/ Accessed: 2026-06-24.

[385] Artjoms Šinkarovs and Troels Henriksen. 2025. Correctness Meets Performance: From Agda to Futhark. *Proceedings of the ACM on Programming Languages* 9, ICFP (Aug. 2025), 567–596. doi:10.1145/3747524